

王道考研系列

计算机考研——机试指南

(第2版)

杨泽邦 赵霖 主编

电子工业出版社

Publishing House of Electronics Industry

北京 • BEIJING

内 容 简 介

本书是一本关于计算机及相关专业研究生入学考试复试机试的辅导教材。全书内容分为 12 章，包括从零开始、暴力求解、排序与查找、字符串、数据结构一、数学问题、贪心策略、递归与分治、搜索、数据结构二、图论、动态规划等。本书由从浅入深、从易到难地讲解了机试的相关考点，并精选名校的复试上机真题作为例题和习题，以便给读者提供最可靠的练习指导。书中的代码简洁且规范，希望读者在理解算法的同时，能够学会一些实用的编程技巧。

本书可以作为研究生入学考试复试机试的复习用书、各类算法竞赛的入门教材，也可作为计算机及相关专业学生提高编程水平的指导用书，非常适合渴望学习经典算法的初学者。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。
版权所有，侵权必究。

图书在版编目(CIP)数据

计算机考研：机试指南/杨泽邦，赵霖主编. —2 版. —北京：电子工业出版社，2019.11

(王道考研系列)

ISBN 978-7-121-37485-2

I. ①计… II. ①杨… ②赵… III. ①电子计算机—研究生—入学考试—自学参考资料 IV. ①TP3

中国版本图书馆 CIP 数据核字(2019)第 212629 号

责任编辑：谭海平

印 刷：

装 订：

出版发行：电子工业出版社

北京市海淀区万寿路 173 信箱 邮编：100036

开 本：787×1092 1/16 印张：16.25 字数：416 千字

版 次：2014 年 1 月第 1 版

2019 年 11 月第 2 版

印 次：2019 年 11 月第 1 次印刷

定 价：69.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：(010) 88254888，88258888。

质量投诉请发邮件至 zlt@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式：(010) 88254552，tan02@phei.com.cn。

前言

回想起自己当年学习各类算法的经历，可谓是苦不堪言，各种算法竞赛类书籍写得晦涩难懂，网上的各种技术贴质量也参差不齐，只能照抄别人的代码来学习，在学习的过程中走了不少弯路，但也有了一些自己的见解。在自己考完研后，发现上机编程是很多计算机相关专业考研学生的薄弱环节。

想到很多考生每天早起晚归地复习，好不容易进入复试，却因为机试成绩不佳而无法被心仪的学校录取，实在令人痛惜。为此，我决定分享自己在学习过程中的一些心得体会，避免大家在学习过程中像我一样走很多弯路。于是便诞生了这本机试入门的书籍，希望能为考生提供一本帮助循序渐进地提高编程能力的教材。

为了方便各位考生练习机试，书中绝大多数的机试题都取自牛客网的考研真题模块(www.nowcoder.com/kaoyan)，书中还给出了相应的提交网址，建议读者在使用本书时，能积极地上机练习，通过动手训练，真正做到理解和掌握相应的知识点。牛客网的考研真题模块目前已收录了绝大多数高校的复试上机真题，非常适合大家进行练习和模拟。也欢迎大家向我们提供各高校的上机真题，联系邮箱是 luhuifeedback@outlook.com。

在撰写本书的同时，我不断地反思为什么包括我在内的大多数学生，学习算法的过程总是异常艰辛？我想这可能是因为这些知识点通常都比较抽象，而这些抽象的知识一旦离开了可视化的教学，就变得非常难以理解。因此，我还打算抽时间录制一套配套的教学视频配合本书，帮助广大考生攻克考研机试的难关，进而顺利地考上自己心仪的学校。视频课程日后将在**中国大学 MOOC（慕课）**平台上发售，敬请期待！

由于教学视频需要提前录制，无法做到实时地发布一些新的内容，因此我打算再开设一个 Bilibili 的账号：**看不懂算导的炉灰**。我会在上面实时地更新一些好的题目和解题思路，以方便大家学习和交流，请各位读者多多关注。

最后，由于笔者的水平有限，尽管对本书进行了多次校对，书中仍然会有不少错误和需要改进的地方，恳请读者朋友指正，联系邮箱为 luhuifeedback@outlook.com。

杨泽邦（炉灰）
2019 年于清华大学

第 2 版序

无论结果如何，总算坚持到了最后。

初试考完了，是不是应该好好放松放松？是不是初试考得好，录取就肯定没有问题了？对不起，这不是计算机专业研究生考试的规则。目前已有越来越多的高校采用上机考试的形式来考查学生的实际动手编程能力，并且机试在复试中所占的比例非常高，甚至很多高校规定复试成绩不及格者，一律不得录取。目前国内开展 ACM 教学的高校非常少，因此提早开始准备和练习，对于一个完全没有接触过 ACM 的计算机考研人来说，是必需的！

在使用本书之前，我们建议读者要有一定的 C 语言编程基础或数据结构方面的基础，为方便读者学习与参考，在此免费分享王道训练营的一些课程录像视频。

王道训练营——C 语言录课视频（共 40 课时）：

www.bilibili.com/video/av49228231

王道训练营——数据结构代码讲解（共 6 课时）：

www.bilibili.com/video/av46488512

本书第 1 版于 6 年前出版，后来因为九度 OJ 数据丢失而不得已终止印刷和销售。经常有读者联系我，希望王道论坛能够重新出版《机试指南》。为此，我找到高分考入清华大学计算机系的杨泽邦同学，他也很乐意分享自己的上机学习经验，于是才有了这一版本。

书中的上机题主要源自王道论坛合作伙伴——牛客网，欢迎大家向我们提供各高校上机复试真题（luhuifeedback@outlook.com），我们会更新到牛客网，以帮助更多的考研学子。

考研其实并没有什么诀窍，就是每天比别人早起一点，晚睡一点，比别人早准备一点，勤奋一点。一分耕耘，一分收获。但我坚信，一个写不出合格代码的计算机专业的学生，即便考上了研究生，也只是给未来失业“判了个缓期执行”而已。

王道论坛是同学们考研路上值得信赖的好伙伴，十一年来他陪伴了逾百万的计算机考研人，不离不弃。希望道友们忠实于自己心底的梦想，勇敢地坚持下去，而当下请开始准备复试吧，熬过这两个月，一切就都好了！

站长：风华漫舞

目 录

第 1 章 从零开始	1
1.1 本书介绍	1
1.2 机试的意义与形式	1
1.3 评判结果	3
1.4 语言与 IDE 的选择	4
1.5 在线评测系统	4
小结	5
第 2 章 暴力求解	6
2.1 枚举	6
2.2 模拟	10
小结	28
第 3 章 排序与查找	29
3.1 排序	29
3.2 查找	36
小结	41
第 4 章 字符串	42
4.1 字符串	42
4.2 字符串处理	45
4.3 字符串匹配	54
小结	61
第 5 章 数据结构一	62
5.1 向量	62
5.2 队列	66
5.3 栈	71
小结	80
第 6 章 数学问题	81
6.1 进制转换	81
6.2 最大公约数与最小公倍数	89
6.3 质数	92
6.4 分解质因数	96

6.5	快速幂	99
6.6	矩阵与矩阵快速幂	101
6.7	高精度整数	106
	小结	117
第 7 章	贪心策略	118
7.1	简单贪心	118
7.2	区间贪心	124
	小结	131
第 8 章	递归与分治	132
8.1	递归策略	132
8.2	分治法	135
	小结	139
第 9 章	搜索	140
9.1	宽度优先搜索	140
9.2	深度优先搜索	145
	小结	152
第 10 章	数据结构二	153
10.1	二叉树	153
10.2	二叉排序树	159
10.3	优先队列	164
10.4	散列表	170
	小结	179
第 11 章	图论	180
11.1	概述	180
11.2	并查集	183
11.3	最小生成树	194
11.4	最短路径	201
11.5	拓扑排序	208
11.6	关键路径	214
	小结	221
第 12 章	动态规划	222
12.1	递推求解	222
12.2	最大连续子序列和	224
12.3	最长递增子序列	230
12.4	最长公共子序列	234
12.5	背包问题	236

12.6 其他问题.....	246
小结.....	251

第1章 从零开始

1.1 本书介绍

本书对各类读者都有一定的用处，但主要是为两类读者而写的：一类是正在备战各大高校计算机类专业研究生机试的考生；另一类是没有太多编程经验，但希望能够快速掌握这方面的方法和技巧，并运用到之后的学习和工作中的初学者。

为了循序渐进地介绍经典的算法，本书分为12章。

- 第1章是入门介绍，主要介绍机试的意义和形式，推荐编程语言、集成开发环境IDE和评测系统。
- 第2章~第4章是基础入门，主要介绍枚举、模拟、查找、排序和基础字符串处理的相关知识。这三章较为简单，也是学习后续章节的基础。
- 第5章是基础数据结构，主要介绍向量、队列和栈等常见的线性数据结构，着重介绍各个数据结构的特点及其应用场景。
- 第6章~第9章是进阶部分，介绍常见的数理逻辑问题，以及贪心、递归、分治和搜索等算法思想。
- 第10章是高级数据结构，主要介绍二叉树、优先队列和散列表等非线性数据结构，着重介绍各个数据结构的特点及其应用场景。
- 第11章~第12章是高阶部分，介绍经典图论的相关知识及动态规划的算法思想。

希望读者学完整本书后，能使自己的编程水平上一个台阶。编程能力不仅在机试时需要，而且是日后研究生阶段的一项重要的基本功。对于不参加研究生机试的读者来说，良好的编程能力也能给日后的学习和生活助上一臂之力。

1.2 机试的意义与形式

机试成为计算机类专业考研复试中越来越重要的一个环节，越来越多的高校在计算机类专业研究生入学考试中采用了机试这种考查形式，以便考查考生分析问题和编程解决问题的能力。通过机试，可以考查一名考生从实际问题中抽象出数学模型的能力，利用所学的计算机专业知识对模型进行分析求解的能力，以及利用计算机编程语言，结合数据结构和算法解决实际问题的综合能力。

绝大部分机试形式都是由五部分组成的。

- 第一部分是题目描述：描述问题的题面，题面要么直接告知考生需要求解的数学问题，要么给出一个实际案例，要求考生从中抽象出所要求解的数学模型。
- 第二部分是输入格式：约定将要给出的数据以怎样的顺序和格式输入程序，更重要的是它将给出输入数据中各个数据的数据范围，可以通过给出的这些数据范围确定数据的规模，为设计算法提供重要依据。
- 第三部分是输出格式：明确考生编写的程序以什么顺序和格式输出题面要求的答案。
- 第四部分是样例输入：为考生提供一组简单测试样例的数据。
- 第五部分是样例输出：是第四部分输入样例对应的输出结果。

另外，要特别注意题目中给定的两个重要参数：时间限制和空间限制。这两个参数限制了考生提交的程序在输出答案之前所耗费的时间和空间。但往往需要更加关注时间限制，要求考生能够根据数据范围和时间限制选择合适的方法来求解给出的问题。

下面来看一个典型的题目描述，从而了解机试题的问题形式。

例题 1.1 计算 $a + b$

题目描述：

求整数 a, b 的和。

输入：

测试案例有多行，每行为 a, b 的值， a, b 为 int 范围。

输出：

输出多行，对应 $a + b$ 的结果。

样例输入：

1 2

4 5

6 9

样例输出：

3

9

15

通过该例，读者基本上了解了机试的形式及问题的各个组成部分。得到题目后，考生应在计算机上编写程序，确认无误后，将该程序的源代码提交给评判系统。评判系统编译考生提交的源代码后，将后台预先存储的输入测试数据输入考生的程序，并将该程序输出的数据与预先存储在评判系统上的“答案”进行比对得出结果。评判系统评判考生的程序后，实时地将评判结果返回到考生界面，考生可以根据该结果了解自己的程序是否被评判系统判为正确，从而根据不同的结果继续完成考试。

1.3 评判结果

本节详细说明考生提交程序后评判系统返回的结果，并针对不同的返回结果，对可能出现错误的地方做出初步的界定。

- **Accepted (答案正确)**: 你的程序对所有测试数据都输出了正确的答案，你已经得到了该题的所有分数，恭喜。
- **Wrong Answer (答案错误)**: 评判系统测试到你的程序对若干组（或全部）测试数据未输出正确的结果。出现这种错误时，通常有两种解决方案：一种方案是，如果对所设计算法的正确性有较大的把握，那么你可以重点考虑代码健壮性，即是否存在某些特殊数据使程序出现错误，譬如边界数据、程序中的变量出现溢出等；另一种方案是，如果怀疑算法本身的正确性，那么就需要重新考虑你的算法设计。
- **Presentation Error (格式错误)**: 评判系统认为你的程序输出“好像”是正确的，只是没有严格按照题目中所要求的输出格式来输出你的答案，譬如你忽略了题目要求在每组输出后再输出一个空行。出现这种错误时，往往预示着你离完全正确已经不远，出现错误似乎只是因为多输出了一些空格、换行之类的多余字符。但这不是绝对的，假如在图形排版（后文会有介绍）中出现格式错误，那么你离正确的答案可能仍然有一定的距离。
- **Time Limit Exceeded (超出时间限制)**: 你的程序在输出所有需要输出的答案之前，已经超过了题目中所规定的时间。这种结果出现在你的评判结果中时，依然有两种解决方案：第一种方案是，如果你确定算法的时间复杂度符合题目的要求，那么依旧可以检查程序是否可能在某种情况下出现死循环，是否有边界数据可能会让你的代码不按照预想的那样工作，从而使程序不能正常结束；第二种方案是，如果你设计的算法的时间复杂度高于题目对复杂度的要求，那么就需要重新设计更加高效的算法，或者对现有的算法进行一定的优化。
- **Runtime Error (运行时错误)**: 你的程序在计算答案的过程中，由于某种致命的原因而异常终止。这时，可以参考如下几个要点来排除错误：① 程序是否访问了不该访问的内存地址，比如访问数组下标时越界；② 程序是否出现了被零除，从而使程序异常；③ 程序是否调用了评判系统禁止调用的函数；④ 程序是否会出现因为递归过深或其他原因造成的栈溢出。
- **Compile Error (编译错误)**: 你提交的程序未通过评判系统的编译，这时可根据详细的编译信息修改你的程序。
- **Memory Limit Exceeded (使用内存超出限制)**: 你提交的程序在运行并输出所有的答案之前，所调用的内存超过了题目限定的内存限制。造成这种错误的原因主要有两个方面：① 你的程序申请了过多的内存来完成所要求的工作，即算法空间复杂度过高；② 因为程序本身的某种错误使得程序不断地申请内存，例如因为某种原因出现了死循环，使得队列中不断地被放入元素。当然，也千万别忽略自己的低级错误，比如在声明数组大小时多“打”了一个0。
- **Output Limit Exceeded (输出超出限制)**: 你的程序输出了过多的东西，甚至超出了评判系统为了自我保护而设定的被评判程序输出大小的最高上限。一般来说这种错误并不

常见，即使出现，也很好找原因：要么是你在提交时忘记关闭你在调试时输出的调试信息；要么是程序的输出部分出现了死循环，使得程序不断地输出而超出了系统的限制。

以上就是评判系统可能返回的几个最基本的结果。若返回 `Accepted`，则你可以获得该题的所有分数。若返回其他错误，则根据不同的考试规则，你的得分会有一些的差异。若你参加的考试采用按测试点来给分的规则，则你依然能够获得你通过的测试点（即程序返回正确结果的那部分测试数据）所对应的分数；但是，若你参加的考试采用所有数据通过才能得分的评分规则，则到目前为止你在这道题上的得分依旧是 0 分。

如果评判结果显示你提交的程序是错误的，那么你可以在修改程序后再次提交该题，直到获得满意的分数或放弃作答该题。

1.4 语言与 IDE 的选择

一般来说，各大高校的机试都会允许考生使用多种语言进行解答。常见的语言有 C、C++、Java 和 Python。由于 Java 的运行效率不高，机试一般放宽的时长并不能完全弥补其性能的差异。并且考虑到大部分考生本科期间学习的编程语言都是 C 或 C++，而且 C++ 向下兼容 C 语言的语法，加之 C++ 有非常好用的标准库 STL，所以本书采用 C with STL 的编码风格。

IDE 的选择因人而异，机试现场一般提供 VC++ 6.0、Visual Studio、Dev-C++、Code::Blocks、eclipse 等 IDE。推荐大家使用类似 Code::Blocks 这一类轻量级的 IDE，本书中所有的示例代码都由 Code::Blocks 编写，Code::Blocks 的下载地址是 <http://www.codeblocks.org/downloads/26>。

1.5 在线评测系统

那么应该到哪里进行机试训练和模拟考试呢？写完程序后，如何判断程序是否正确呢？在线评测系统一站式解决这些问题。在线评测系统上可以看到题目的题目描述、输入格式、输出格式、样例输入和样例输出。根据题目要求写出相应的代码后，提交给在线评测系统，在线评测系统评测代码后，会向你返回程序的结果。本书中的绝大部分例题和全部习题都来自牛客网 (<https://www.nowcoder.com/kaoyan>) 这个在线评测系统，它收录了近 300 道各高校近年来的机试真题，为准备机试的考生提供了极大的便利。

读者第一次使用牛客网时，需要先为自己注册一个账号，有了账号后，读者就能在牛客网上进行日常的练习。牛客网已按学校对题目进行了分类，选择一所学校后，选择“复试”按钮，便会出现这所学校的历年机试真题，此时读者便可进行练习。

根据题目的要求完成代码的编写后，将其复制到代码填写区域，单击“保存并调试”按钮，就会返回代码的调试情况。

此外，还有很多非常优秀的网上评测系统，譬如北京大学的网上评测系统 POJ (<http://poj.org/>)、杭州电子科技大学的网上评测系统 HDU (<http://acm.hdu.edu.cn/>)，本书中的一些例题便来自这两个评测系统。然而，这些评测系统偏重于编程竞赛，因此只建议学有余力的读者去上面进行练习。

小结

本章首先简要介绍了全书的内容，然后讨论了机试的意义、形式和评判方法，并向读者推荐了编程语言、IDE 和评测系统。希望本书能够助力读者备考计算机考研机试。同时，由于笔者自身能力的限制及编写时间的不足，书中难免存在一些疏漏和错误，恳请读者指正。

第2章 暴力求解

考研机试题目中的很多问题往往能通过暴力方法来求解，这些题目并不需要进行过多的思考，而只需枚举所有可能的情况，或者模拟题目中提出的规则，便可以得到解答。虽然说这种方法看上并不高明，但对于一些简单的题目来说却是行之有效的好策略。

2.1 枚举

枚举是指对每个可能的解进行逐一判断，直到找到符合题目要求的答案。枚举类的题目本身并不复杂，但在采取枚举策略之前，一定要好好分析题目的枚举量，枚举量过大时，需要选择其他解决方法。即使问题适合用枚举策略来求解，也最好对题目进行一定的分析，以便通过减少部分无效的枚举来使得程序更加简洁和高效。

例题 2.1 *abc*（清华大学复试上机题）

题目描述：

设 a, b, c 均是 0 到 9 之间的数字， abc, bcc 是两个三位数，且有 $abc + bcc = 532$ 。
求满足条件的所有 a, b, c 的值。

输入：

题目没有任何输入。

输出：

请输出所有满足题目条件的 a, b, c 的值。
 a, b, c 之间用空格隔开。
每个输出占一行。

样例输入：

样例输出：

提交网址：

<http://t.cn/E9WMRTE>

【分析】

本题只有 a, b, c 三个从 0 到 9 的数，枚举量仅为 $10 \times 10 \times 10 = 1000$ ，所以本题可让 a, b, c 从 0 枚举到 9，每当遇到符合题目要求的解，就输出它。但在动手写代码之前，可以再做一些分析： $abc = 100a + 10b + c$ ， $bcc = 100b + 11c$ ，于是 $abc + bcc = 100a + 110b + 12c$ 。因此，只需判断前一个等式的右侧是否等于 532 即可，这样便简化了加法。

代码 2.1

```
#include <iostream>
#include <cstdio>

using namespace std;

int main() {
    for (int a = 0; a <= 9; ++a) {
        for (int b = 0; b <= 9; ++b) {
            for (int c = 0; c <= 9; ++c) {
                if (a * 100 + b * 10 + c * 12 == 532) {
                    printf("%d %d %d\n", a, b, c);
                }
            }
        }
    }
    return 0;
}
```

例题 2.2 反序数（清华大学复试上机题）

题目描述：

设 N 是一个 4 位数，它的 9 倍恰好是其反序数（如 1234 的反序数是 4321），求 N 的值。

输入：

题目没有任何输入。

输出：

输出题目要求的 4 位数，如果结果有多组，那么每组结果之间以回车隔开。

样例输入：

样例输出：

提交网址：

<http://t.cn/E9WBrut>

【分析】

题目中所求的 N 为 4 位数，因此 N 的取值范围只可能是 1000~9999，枚举量不到 10000，对于每个数，只要求它的反序数即可。要求一个数的反序数，只需进行该数的位数次运算便可得到；例如对于数 1234，只需进行 4 次运算便可得到 4321。于是，本题枚举所有 N 的运算次数不超过 $4 \times 10000 = 40000$ ，所以本题可以采用枚举法来求解。

代码 2.2

```
#include <iostream>
```



```

#include <stdio>

using namespace std;

int Reverse(int x) {    //求反序数
    int revx = 0;
    while (x != 0) {
        revx *= 10;
        revx += x % 10;
        x /= 10;
    }
    return revx;
}

int main() {
    for (int i = 1000; i <= 9999; ++i) {
        if (i * 9 == Reverse(i)) {
            printf("%d\n", i);
        }
    }
    return 0;
}

```

其中 *Reverse* 函数用来求反序数。这种通过对 *x* 不断进行除运算，然后对 *revx* 不断进行乘运算的方法很常见，在后续学习进制转换时还会经常遇到。

例题 2.3 对称平方数 1（清华大学复试上机题）

题目描述：

打印所有不超过 256，其平方具有对称性质的数。如 2 和 11 就是这样的数，因为 $2 \times 2 = 4$ ， $11 \times 11 = 121$ 。

输入：

题目没有任何输入。

输出：

输出具有题目要求的性质的数。如果输出数据不止一组，那么各组数据之间以回车隔开。

样例输入：

样例输出：

提交网址：

<http://t.cn/E9lUYRn>

【分析】

了解如何处理反序数后，读者在面对这道求对称性质数的题目时，是否会觉得很简单呢？只需要比较一个数的反序数和这个数本身的值是否相等来判断数字是否具有对称性质。通过上题可知，要求一个数的反序数，只需进行该数位数的运算次数就可得到。这道题只需从 0 的平方枚举到 256 的平方可以得到答案，其中枚举到的最大数是 $256 \times 256 = 65536$ ，而最大数 65536 也只需要 5 次运算便可以得到它的反序数。于是，运算次数必定小于 $257 \times 5 = 1285$ ，所以本题可以通过枚举法求解。

代码 2.3

```
#include <iostream>
#include <cstdio>

using namespace std;

int Reverse(int x) {
    int revx = 0;
    while (x != 0) {
        revx *= 10;
        revx += x % 10;
        x /= 10;
    }
    return revx;
}

int main() {
    for (int i = 0; i <= 256; ++i) {
        if (i * i == Reverse(i * i)) {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

相信读者通过本节的学习，已经初步了解了枚举类问题。这类题目的特点是，数据量较小，可以逐个地判断数据是否符合题目的要求。这类题目属于机试题中较简单的，希望读者多加练习，以便日后在机试中遇到这类题目时能够轻松应对。

习题 2.1 与 7 无关的数（北京大学复试上机题）

一个正整数，如果它能被 7 整除，或者它的十进制表示法中某个位数上的数字为 7，那么称其为与 7 相关的数。现求所有小于等于 n ($n < 100$) 的与 7 无关的正整数的平方和。

提交网址:

<http://t.cn/E9l0OZQ>

习题 2.2 百鸡问题（哈尔滨工业大学复试上机题）

用小于等于 n 元去买 100 只鸡，大鸡 5 元/只，小鸡 3 元/只，还有 1/3 元每只的一种小鸡，分别记为 x 只、 y 只和 z 只。编程求解 x, y, z 所有可能的解。

提交网址:

<http://t.cn/E9ldhru>

习题 2.3 Old Bill（上海交通大学复试上机题）

【题目大意】 N 只火鸡的价格为 $\$ _XYZ_$ ，火鸡的总数 N 在 1 到 99 之间。价格由五位数组成，两边的数字由于褪色而看不清，所以只能看到中间的三位数。假设第一个数字非零，每只火鸡的价格是整数，并且所有火鸡的价格相同。给定 N, X, Y 和 Z ，编写一个程序来猜测两边褪色的数字和火鸡的原始价格。如果有多个价格符合题意，那么输出最昂贵的那个。

提交网址:

<http://t.cn/E9jqijR>

2.2 模拟

模拟类题目是机试题中出现频率很高的一种类型，这类题目的特点是并不涉及特别高深的知识，只需利用程序实现题目的要求。由于这类题目通常不需要经过太多的思考，所以能够很纯粹地考查考生的编程能力。

1. 图形排版

图形排版是最为常见的模拟类题型，这类题目要求考生按照特定规则输出字符，主要考查考生对输出格式的把握。通常来说这类题目的规律性比较强，掌握了题目中的规律，题目便能迎刃而解。

例题 2.4 输出梯形（清华大学复试上机题）

题目描述:

输入一个高度 h ，输出一个高度为 h 、上底边长度为 h 的梯形。

输入:

一个整数 h ($1 \leq h \leq 1000$)。

输出:

h 所对应的梯形。

样例输入:

4

样例输出:

```
*****
*****
```

【分析】

这道题是考查输出格式的典型真题。观察输出图形可发现其具有较强的规律性：

- ① 梯形的高度是 h ，故图形的行数为 h 。
- ② 首行有 h 个“*”，下一行中总是比上一行中多两个“*”，故第 i 行有 $h+2*i$ 个“*”。
- ③ 最后一行中“*”的数量决定了图像的列数。
- ④ 每行都右对齐，左边空余的位置输出空格，空格数是列数与该行中“*”的数量之差。

代码 2.4

```
#include <iostream>
#include <cstdio>

using namespace std;

int main() {
    int h;
    while (scanf("%d", &h) != EOF) {
        int row = h;                //行数为 h
        int col = h + (h - 1) * 2;    //列数为最底行中*的数量
        for (int i = 0; i < row; ++i) {
            for (int j = 0; j < col; ++j) {
                if (j < col - (h + 2 * i)) { //输出空格
                    printf(" ");
                } else {                    //输出*
                    printf("*");
                }
            }
            printf("\n");
        }
        return 0;
    }
}
```

这道图形排版题的图形具有较强的规律性，并且这一规律顺序往往与输出顺序一致，即可以从上至下、从左至右应用规律。只需仔细观察图形，把握其中的规律，并将其量化后直接写入程序的输出部分，便可输出题面所要求的图形。

相信读者能够理解这道题的输出过程。然而，很多读者可能对主循环体的循环条件 `while (scanf("%d",&h) != EOF)` 大为不解。在用 `scanf` 完成对 h 的输入的同时，又完成了什么看起来

令人迷惑的条件判断呢？要回答这个问题，首先要明确以下两点：

- ① `scanf` 函数是有返回值的（尽管大多时候都会被人忽略），它将返回被输入函数成功赋值的变量个数。在此例中，若成功完成输入并对 `h` 赋值，则 `scanf` 函数的返回值是成功赋值的变量个数，也就是 `1`。可以通过该函数的返回值来判断循环条件是否成立。
- ② 应该注意到该题的题面中并未明确说明数据只有单组，因此输入数据可能会有多组，这就需要对每组输入都输出其相应的结果。然而，事先并不知道会有多少组数据。因此，可以使用这个循环测试条件对输入是否结束进行判断。

了解上述两点后，便可解读这个循环条件。如果仍有测试数据未被测试完，那么 `scanf` 函数会将该数据赋值给变量 `h`，并返回成功赋值的变量个数 `1`，而 `1` 并不等于 `EOF(-1)`，循环条件成立，程序进入循环体继续执行程序；如果输入已到达结尾，那么 `scanf` 函数无法再为变量 `h` 赋值，于是 `scanf` 函数返回 `EOF` (End Of File)。此时，循环条件不成立，程序跳过循环体，执行 `return 0`，使程序正常结束。该循环判断条件既能够保证对多组测试数据进行处理，又使程序在输入结束后能够正常退出。反之，如果不使用循环，那么程序在处理完一组数据后就会退出，造成后续测试数据无法被正常处理。然而，如果使用死循环而不加以任何退出条件，那么程序在处理完所有测试数据后仍不能正常退出，虽然程序已经输出了所有测试数据的答案，但评判系统还是会认为程序在限定时间内无法运行完毕，于是返回超时的评判结果。因此，初学者很容易因为这个原因出现莫名其妙的程序超时。

除上面这种图形排版题外，还有另一个类排版题，它要求的图形不具有明显的规律性或者规律性较难直接应用到输出中。求解此类问题的常用方法是，首先将图形按照题目的要求构造出来，然后进行输出。

例题 2.5 叠筐

题目描述：

把一个个大小差一圈的筐叠上去，使得从上往下看时，边筐花色交错。这个工作现在要让计算机来完成，得看你的了。

输入：

输入是一个个三元组，分别是：外筐尺寸 n (n 为满足 $0 < n < 80$ 的奇整数)，中心花色字符，外筐花色字符，后者都为 ASCII 可见字符。

输出：

输出叠在一起的筐图案，中心花色与外筐花色字符从内层起交错相叠，多筐相叠时，最外筐的角总是被打磨掉。叠筐与叠筐之间应有一行间隔。

样例输入：

```
11 B A
5 @ W
```

样例输出：

```
AAAAAAAAA
ABBBBBBBBBA
```



```

ABAAAAAABA
ABABBBBABA
ABABAAABABA
ABABABABABA
ABABAAABABA
ABABBBBABA
ABAAAAAABA
ABBBBBBBBA
AAAAAAAAA

```

```

@@@
@WWW@
@W@W@
@WWW@
@@@

```

【分析】

如本题的样例输出所示，输出图形的规律性主要体现在由外而内的各个圈上，而这与输出顺序又不太契合（从上至下，从左至右），不容易将该图形存在的规律直接应用到输出中，所以需要使用刚才提到的办法，即先构造后输出。利用观察到的“圈形规律”构造图形，构造完成后再按照先行后列的方式输出图形。构造的理念如下：

- ① 先将 4 个角补上，成为一个正方形。
- ② 从外圈向内圈逐步构造。
- ③ 用圈的左上角和右下角坐标来表示圈。
- ④ 决定圈的填充字符是中心字符还是外筐字符。
- ⑤ 确定圈填充字符的边长。
- ⑥ 构造完成后再将 4 个角剔除。

代码 2.5

```

#include <iostream>
#include <cstdio>

using namespace std;

char matrix[80][80];

int main() {
    int n; // 叠筐大小
    char a, b; // 输入的两个字符
    bool firstCase = true; // 第一组数据标志
}

```

```

while (scanf("%d%c%c", &n, &a, &b) != EOF) {
    if (firstCase == true) {
        firstCase = false;
    } else {
        printf("\n");
    }
    for (int i = 0; i <= n / 2; ++i) {           //(i, i)是每圈左上角坐标
        int j = n - i - 1;                       //(j, j)是每圈右下角坐标
        int length = n - 2 * i;                   //求当前圈的边长
        char c;                                    //求当前圈填充字符
        if ((n / 2 - i) % 2 == 0) {
            c = a;
        } else {
            c = b;
        }
        for (int k = 0; k < length; ++k) {        //为当前圈赋值
            matrix[i][i + k] = c;                 //上边赋值
            matrix[i + k][i] = c;                 //下边赋值
            matrix[j][j - k] = c;                 //右边赋值
            matrix[j - k][j] = c;                 //左边赋值
        }
    }
    if (n != 1) {                                //剔除4个角
        matrix[0][0] = ' ';
        matrix[0][n - 1] = ' ';
        matrix[n - 1][0] = ' ';
        matrix[n - 1][n - 1] = ' ';
    }
    for (int i = 0; i < n; ++i) {                 //逐行逐列打印
        for (int j = 0; j < n; ++j) {
            printf("%c", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

如上面的代码所示，并不是在输出时使用得到的规律，而用另一种更容易的方法完成排版。利用一个缓存数组来表示将要输出的字符阵列，对该字符阵列的坐标做如下规定：规定阵列左

上角字符的坐标为 $(0,0)$ ，阵列右下角字符的坐标为 $(n-1,n-1)$ ，其他坐标可由此推得。程序按照由最外圈至最内圈的顺序来完成图形的排列。完成每圈排列时，需要注意以下几个要点：

- ① 需要确定该圈左上角和右下角的坐标，将以这两个坐标为参照点来完成该圈的其他字符位置的确定（也可选用其他点）。观察图形得知，从最外圈的左上角 $(0,0)$ 到最中间圈的左上角 $(n/2,n/2)$ 都是 (i,i) 的格式，从最外圈的右下角 $(n-1,n-1)$ 到最中间圈的右下角 $(n/2,n/2)$ 都是 (j,j) 的格式，所以 i 和 j 的关系为 $i+j=n-1$ 。
- ② 需要确定该圈使用哪个字符来填充，这由该圈是从内向外数的第几个圈来决定的。如果是奇数圈，那么用中心花色字符填充；如果是偶数圈，那么用外筐花色字符填充。对于左上角为 (i,i) 的圈来说，可以通过 $(n/2-i)$ 的奇偶性来判断。
- ③ 需要计算该圈填充字符的边长。这也很容易得出：最外圈为 n ，次外圈为 $n-2$ ，外圈总比内圈长度多 2。于是，对左上角为 (i,i) 的圈来说，长度为 $n-2*i$ 。

另外，上述代码中还有以下两个值得注意的地方：

- ① 输出格式。题面要求在输出的每个叠筐间输出一个空行，即除最后一个叠筐输出后没有额外的空行外，其他叠筐输出后都需要额外输出一个空行。为了完成这一要求，可以将要求形式改变为：除在第一个输出的叠筐前不输出一个空行外，在其他每个输出的叠筐前都需要输出一个额外的空行。两种要求形式是等价的。为完成这一目的，在程序开头声明了变量 *firstCase* 来表示正在处理的数据是否为第一组数据，它的初始值为 *true*。程序读取每组数据后，都测试 *firstCase* 的值，若其为 *true*，则表示当前处理的数据是第一组数据，不输出空行，并将 *firstCase* 变量改为 *false*。之后如果还有数据，那么在数据输入后对 *firstCase* 变量进行测试，而 *firstCase* 变量此时已是 *false*，在输出的叠筐前额外输出一个空行，达到了题面对输出格式的要求。
- ② 边界数据处理。根据上文，在缓存数组中完成字符阵列排版后，需要将这个阵列 4 个角的字符修改为空格，但这一修改并不是一定需要的。当输入的 n 为 1 时，这一修改会变得多余，它会使输出仅变为一个空格，这与题面要求不符。因此，在进行这一修改之前，需要对 n 的数值进行判断，若其不为 1 则进行修改，否则跳过修改部分。由此，不难看出，机试考题要求考生在作答时，不仅要能够把握算法，而且要细致地考虑边界数据会给程序造成什么样的影响。只有充分考虑了所有情况，并且在题面明确将会出现的所有条件下，保证程序依旧能够正常地工作，才能使程序真正符合题目要求。

本例介绍了另一种求解排版题的思路。当输出图形具有的规律不能或很难直接应用到输出时，就应考虑采用该例所用的方法，即首先用一个缓存数组来保存将要输出的字符阵列，然后在该数组上完成排版。按照自己的需要或图形的规律完成排版后，依次输出图形，进而完成题目的要求。

习题 2.4 Repeater（北京大学复试上机题）

【题目大意】给你一个仅包含一种字符和空格的模板，模板显示如何创建无尽的图片，将字符用作基本元素并将它们放在正确的位置以形成更大的模板，然后不断进行该操作。

提交网址：

<http://t.cn/EqjcaVb>

习题 2.5 Hello World for U (浙江大学复试上机题)

【题目大意】给定任意 $N \geq 5$ 个字符的字符串，要求将该字符串组成 U 形，而且字符必须按照给定字符串原来的顺序进行打印。

提交网址：

<http://t.cn/Eqjizni>

2. 日期问题

日期类运算的各种问题同样被频繁地选入机试考题中，但这类问题通常都有规律可循。只要能够把握这类问题题面中的核心规律，求解这类问题就不会有太大的难度。

例题 2.6 今年的第几天？(清华大学复试上机题)

题目描述：

输入年、月、日，计算该天是本年的第几天。

输入：

包括 3 个整数：年 ($1 \leq Y \leq 3000$)、月 ($1 \leq M \leq 12$)、日 ($1 \leq D \leq 31$)。

输出：

输入可能有多组测试数据，对于每组测试数据，
输出一个整数，代表 Input 中的年、月、日对应本年的第几天。

样例输入：

1990 9 20
2000 5 1

样例输出：

263
122

提交网址：

<http://t.cn/EqjXK5A>

【分析】

日期类题目的一种常见求解方法是预处理，即在程序真正开始处理输入数据前，预处理出所有月份的天数并保存。真正需要处理数据时，只需用 $O(1)$ 的时间复杂度便可读出保存的数据，稍加处理后便能得到答案。值得一提的是，预处理是以空间换时间的常见手段（保存预处理所得数据所需的内存来换取实时处理所需的时间消耗）。

日期类问题通常有一个需要特别注意的要点——闰年。每逢闰年，2 月就会有 29 天，这对天数的计算势必产生重大影响。这里必须明确闰年的判断规则：当年数不能被 100 整除时，若

其能被 4 整除或能被 400 整除，则为闰年。用逻辑语言表示即 $(year \% 4 == 0 \ \&\& \ year \% 100 != 0) \parallel (year \% 400 == 0)$ ，当这个逻辑表达式为 *true* 时，其为闰年，否则不是闰年。从中还可看出，闰年并不严格按照每 4 年一次的规律出现，在某种情况下也可能出现两个相邻闰年相隔 8 年的情况（如 1896 年与 1904 年）。因此，应严格按照上述表达式来判断某年是否是闰年，而不能采用某个闰年后的第四年又是闰年的规则。

代码 2.6

```
#include <iostream>
#include <cstdio>

using namespace std;

int daytab[2][13] = {                                     //预处理
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

bool IsLeapYear(int year) {                               //判断是否为闰年
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

int main() {
    int year, month, day;
    while (scanf("%d%d%d", &year, &month, &day) != EOF) {
        int number = 0;                                    //记录天数
        int row = IsLeapYear(year);                       //判断用 daytab 中的哪一行
        for (int j = 0; j < month; ++j) {                 //逐月添加天数
            number += daytab[row][j];
        }
        number += day;
        printf("%d\n", number);
    }
    return 0;
}
```

- ① 预处理每个月的天数，用二维数组提前保存平年和闰年的每月天数，这样在数据处理过程中就不必逐月地判断，有利于提高代码的效率，是一种空间换时间的常用技巧，而且也会使得代码更加有逻辑，减少出错的概率。
- ② 为了判断某年是否是闰年，定义了一个函数（尽量用函数而不要用宏定义），这个函数

利用上文中提到的逻辑表达式来判断该年是否是闰年，根据这个表达式的逻辑值使表达式为 1 或 0，并且根据逻辑值来选择保存每月天数的数组的不同行，取得该月应有的天数，进而保证闰年时存在 2 月 29 日。

例题 2.7 打印日期（华中科技大学复试上机题）

题目描述：

给出年份 m 和一年中的第 n 天，算出第 n 天是几月几号。

输入：

输入包括两个整数： y ($1 \leq y \leq 3000$) 和 n ($1 \leq n \leq 366$)。

输出：

可能有多组测试数据，对于每组数据，

按格式 `yyyy-mm-dd` 将输入中对应的日期打印出来。

样例输入：

```
2000 3
2000 31
2000 40
2000 60
2000 61
2001 60
```

样例输出：

```
2000-01-03
2000-01-31
2000-02-09
2000-02-29
2000-03-01
2001-03-01
```

提交网址：

<http://t.cn/E9YP2a8>

【分析】

这道题是上道题的逆问题。上道题是知道年、月、日而求天数；本题是知道天数和年，反过来求月和日。因此，上题代码的主体可以不变，只需对求天数部分的代码稍做修改：从逐月不断累加天数，变成天数不断减少，直到求出月份，剩下的天数便是日期，从而完成解答。

然而，本题的输出要求年必须是 4 位，月和日必须是 2 位，对于位数不足的数据，应该前置补零。例如，如果是 985 年 2 月 1 日，那么应该输出 `0985-02-01`。如果考生不对输出做任何限制，那么就会输出格式 `985-2-1`。这样的结果会被直接判错。要解决这个问题，需要学会 `printf` 函数的输出格式，`printf` 函数可以在“%”和字母之间插入数字表示最大场宽。下面给出常用的输出格式样例，更多格式请读者自行查阅相关资料：

- `%2d` 表示输出场宽为 2 的整数，超过 2 位按实际数据输出，不够 2 位右对齐输出。

- `%02d` 表示输出场宽为 2 的整数，超过 2 位按实际数据输出，不够 2 位前置补 0。
- `%5.2f` 表示输出场宽为 5 的浮点数，其中小数点后有 2 位，不够 5 位右对齐输出。

代码 2.7

```
#include <iostream>
#include <cstdio>

using namespace std;

int daytab[2][13] = {
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

bool IsLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

int main() {
    int year, month, day;
    int number; //记录天数
    while (scanf("%d%d", &year, &number) != EOF) {
        month = 0; //初始化月
        int row = IsLeapYear(year);
        while (number > daytab[row][month]) { //确定月
            number -= daytab[row][month];
            month++;
        }
        day = number; //确定日
        printf("%04d-%02d-%02d\n", year, month, day);
    }
    return 0;
}
```

例題 2.8 日期累加（北京理工大学复试上机题）

題目描述：

设计一个程序，它能够计算一个日期加上若干天后是什么日期。

输入：

输入第一行表示样例个数 m ，接下来的 m 行中，每行 4 个整数，分别表示年、月、日和累加的天数。

输出：

输出 m 行，每行按 `yyyy-mm-dd` 的个数输出。

样例输入：

```
1
2008 2 3 100
```

样例输出：

```
2008-05-13
```

提交网址：

<http://t.cn/E9YwOCr>

【分析】

求解完上面两道题后，再来看这道题时，相信大部分读者会发现这道题其实就是上面两道题的结合体。本题可以首先通过给定的年、月、日计算出这一天是该年的第几天（即天数），然后将该天数加上给定的数值，最后对加上数值后的天数反向求解，得出月和日。

这样的想法固然是对的，也充分说明读者完全理解了前面的两道题。但在这样做时，需要额外注意一点，即加上给定数值后的天数有可能超出了该年的天数（365 或 366），这时需要将年数更新为正确的年数后，才能继续按照之前的方式计算月和日。

代码 2.8

```
#include <iostream>
#include <cstdio>

using namespace std;

int daytab[2][13] = {
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

bool IsLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

int NumberOfYear(int year) { // 返回该年天数
    if (IsLeapYear(year)) {
```

```

        return 366;
    } else {
        return 365;
    }
}

int main() {
    int year, month, day;
    int number; //记录天数
    int caseNumber; //组数
    scanf("%d", &caseNumber);
    while (caseNumber--) {
        scanf("%d%d%d", &year, &month, &day, &number);
        int row = IsLeapYear(year);
        for (int j = 0; j < month; ++j) {
            number += daytab[row][j];
        }
        number += day;
        while (number > NumberOfYear(year)) { //确定年
            number -= NumberOfYear(year);
            year++;
        }
        month = 0;
        row = IsLeapYear(year);
        while (number > daytab[row][month]) { //确定月
            number -= daytab[row][month];
            month++;
        }
        day = number; //确定日
        printf("%04d-%02d-%02d\n", year, month, day);
    }
    return 0;
}

```

这道题的解法其实是：融合前面两道题的解法，并在中间加一段确定年的代码。相信通过前面两道题的学习，读者应能轻松地读懂或独立地写出本题的代码。

然而，需要注意的是，本题不是只有一次输入，不是未告知具体有多少组输入，而是明确告知有 m 组输入，因此常用的方法是用一个变量 `caseNumber` 来记录组数，并用循环 `while (caseNumber--)` 进行判断。还有数据要输入时，`caseNumber` 大于 0，此时循环成立，程序

进入循环体继续执行程序；不再有数据输入时，*caseNumber* 等于 0，循环条件不成立，程序跳过循环体，执行 *return 0*，使程序正常结束。这个循环判断条件既保证了可以对多组测试数据进行处理，又使程序在输入结束后能够正常退出。

习题 2.6 日期差值（上海交通大学复试上机题）

有两个日期，求两个日期之间的天数，如果两个日期是连续的，则规定它们之间的天数为两天。

提交网址：

<http://t.cn/E9YzOLE>

习题 2.7 Day of Week（上海交通大学复试上机题）

【题目大意】编写一个程序，计算给定的日期是周几。

提交网址：

<http://t.cn/E9YZLbi>

习题 2.8 日期类（北京理工大学复试上机题）

编写一个日期类，要求按 `xxxx-xx-xx` 的格式输出日期，实现加一天的操作。

提交网址：

<http://t.cn/E9RJUp4>

3. 其他模拟

除上面两类常考的模拟题外，其他类型的模拟题通常多变，没有确定的出题方式。然而，在面对这类题目时，大可不必担心，这类题目是怎么描述的，就按照它说的做，用代码模拟出题面的要求即可。

例题 2.9 剩下的树（清华大学复试上机题）

题目描述：

有一条长度为整数 L ($1 \leq L \leq 10000$) 的马路，可以把它想象为数轴上长度为 L 的一条线段，起点是坐标原点，在每个整数坐标点处有一棵树，即在 $0, 1, 2, \dots, L$ 共 $L+1$ 个位置上有 $L+1$ 棵树。现在要移走一些树，移走的树的区间用一对数字表示，如“100 200”表示移走从 100 到 200 之间（包括端点）的所有树。可能有 M ($1 \leq M \leq 100$) 个区间，区间之间可能有重叠。现在要求移走所有区间的树之后剩下的树的棵数。

输入：

两个整数 L ($1 \leq L \leq 10000$) 和 M ($1 \leq M \leq 100$)。接下来有 M 组整数，每组有一对数字。

输出：

可能有多组输入数据，对于每组输入数据，输出一个数，表示移走所有区间的树之后剩下的树的棵数。

样例输入：

500 3

100 200

150 300

470 471

样例输出：

298

提交网址：

<http://t.cn/E9ufYo5>

【分析】

题目要求模拟一条包含 $L + 1$ 棵树的线段，并在这条线段上施加 M 次操作，每次操作将一个区间内的树全部移除。因此，可以用一个 *bool* 型数组模拟这条线段，有树就设为 *true*，没有树就设为 *false*。最初，树的棵数为 $L + 1$ ，每次操作时将区间内为 *true* 的元素设为 *false*，同时将树的棵数减 1。当全部操作完成后，便会得到答案。

代码 2.9

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 10001;           //数据量定义为常数

bool arr[MAXN];

int main() {
    int l, m;
    while (scanf("%d%d", &l, &m) != EOF) {
        for (int i = 0; i <= l; ++i) {
            arr[i] = true;
        }
        int number = l + 1;         //初始化树的数量
        while (m--) {
            int left, right;
            scanf("%d%d", &left, &right);
            for (int i = left; i <= right; ++i) {
                if (arr[i]) {        //将该树移除
                    arr[i] = false;
                    number--;
                }
            }
        }
        printf("%d\n", number);
    }
    return 0;
}
```

例题 2.10 手机键盘（清华大学复试上机题）

题目描述：

按照手机键盘输入字母的方式，计算所花费的时间。例如，a, b, c 都在“1”键上，输入 a 只需按 1 次键，输入 c 需要连续按 3 次键。如果连续两个字符不在同一个键上，那么可以直接按；例如，ad 需要按 2 下，kz 需要按 6 下。如果连续两个字符在同一个键上，那么两次按键之间需要等一段时间；例如 ac，按 a 之后，需要等一会儿才能按 c。现在假设每按一次需要花费一个时间段，等待时间需要花费两个时间段。现在给出一串字符，计算输入它所花费的时间。

输入：

一个长度不大于 100 的字符串，其中只有手机按键上有的小写字母。

输出：

输入可能包括多组数据，对于每组数据，输出按下 *Input* 所给字符串需要的时间。

样例输入：

```
500 3
100 200
150 300
470 471
```

样例输出：

```
bob
www
```

提交网址：

<http://t.cn/E9ulclc>

【分析】

题目要求模拟按键所花费的时间，而花费时间是与按键时间和等待时间有关的，同一个按键上不同字母的按键次数是其在按键上的顺序，连续按下相同按键时就需要等待一段时间。值得注意的是，不要想当然地认为每个按键上都有 3 个字母，大家只要看一下自己的手机，就会发现大部分按键上有 3 个字母，但有的手机的少数几个按键上有 4 个字母。

- ① 由于每个按键上字母的数量不相同，因此每个字母需要的按键次数很难直接通过数学公式计算出来。这时，可以用之前学过的预处理策略，先将每个字母的按键次数记录在一个数组中，每当遇到一个字母，直接访问数组便可得到该字母的按键次数。
- ② 由于有的按键上有 3 个字母，有的按键上有 4 个字母，因此通过对 3 取模来判断两个字母是否属于某个按键的方法行不通。这时，可以换一个思路：如果两个字母属于同一个按键，那么字母本身之间的间距就等于它们的按键次数的差；反之，如果不等，那么这两个字母必定属于不同的按键。

代码 2.10

```

#include <iostream>
#include <cstdio>

using namespace std;

int keytab[26] = {1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1,
                 2, 3, 1, 2, 3, 4, 1, 2, 3, 1, 2, 3, 4};

int main() {
    string str;
    while (cin >> str) {
        int time = 0;
        for (int i = 0; i < str.size(); ++i) {
            time += keytab[str[i] - 'a'];           //按键时间
            if (i != 0 && str[i] - str[i - 1] ==
                keytab[str[i] - 'a'] - keytab[str[i - 1] - 'a']) {
                time += 2;                         //等待时间
            }
        }
        printf("%d\n", time);
    }
    return 0;
}

```

相信大部分读者能够轻松地理解上面的主体程序的含义，但有些读者可能不太熟悉字符串 `string` 这种基本数据类型。由于 C 语言不提供字符串这一基本类型，因此通常需要使用字符数组来代替字符串，而这样操作起来很不方便。好在 C++ 提供了字符串这种基本数据类型，可以很方便地对字符串进行各种操作。因此在输入和输出字符串时，就不能再用 C 语言的输入、输出函数 `scanf` 和 `printf`，而要使用 C++ 的输入、输出函数 `cin` 和 `cout`。读者实在读不懂上面的主体程序时，也没有关系，这时可以先跳过这段程序，学完后面关于字符串的一章后，回头再看这段程序就会非常清晰。

例题 2.11 $\times\times\times$ 定律（浙江大学复试上机题）

题目描述：

对于一个数 n ，如果是偶数，那么把 n 砍掉一半；如果是奇数，那么把 n 变成 $3n+1$ 后砍掉一半，直到该数变为 1 为止。计算需要经过几步才能将 n 变为 1，具体可见样例。

输入：

测试包含多个用例，每个用例包含一个整数 n ，当 n 为 0 时表示输入结束（ $1 \leq n \leq 10000$ ）。

输出：

对于每组测试用例请输出一个数，表示需要经过的步数，每组输出占一行。

样例输入:

```
3
1
0
```

样例输出:

```
5
0
```

提交网址:

<http://t.cn/E937wDs>

【分析】

题目要求模拟 n 经过多少步数到达 1，因此可用 `while` 循环不断地判断 n 是否为 1。

- ① 若 n 为 1，则跳出循环。
- ② 若 n 不为 1，则根据 n 的奇偶性进行相应的操作，并将步数加 1。
- ③ 跳出循环时，步数便是问题的答案。

代码 2.11

```
#include <iostream>
#include <cstdio>

using namespace std;

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        int step = 0;
        while (n != 1) {
            if (n % 2 == 0) {
                n = n / 2;
            } else {
                n = (3 * n + 1) / 2;
            }
            step++;
        }
        printf("%d\n", step);
    }
}
```


相对来说，本题比较简单，但仍然有一个需要关注的问题，那就是本题题面中规定了程序的退出条件，即当 n 等于 0 时不做任何输出并且程序退出。因此，在编写程序时，就需要特别为此加入判断语句。读者应仔细阅读题，确定题目是否约定了一种退出条件，从而保证程序不会出现莫名其妙的超时错误。

学完上面的题目后，读者可能会觉得这些模拟题目千奇百怪。然而，只要能够按照题面的要求进行编程，便能够轻松地应对这些模拟题目。

习题 2.9 Grading (浙江大学复试上机题)

【题目大意】对成千上万的研究生入学考试试题进行评分是一项艰苦的工作，而且评分要做到尽可能公平就更加困难。一种策略是将每个问题分配给 3 名独立的评分专家。如果他们彼此给出不同的分，那么就请审判长做出最终的决定。现在要求你编写程序来帮助完成此过程。对于每个问题，给出满分 P 和公差 T ($T < P$)。评分规则如下：

- 首先将问题分配给 2 位专家，获得评分 G_1 和 G_2 。如果这两个分数的差在公差范围内，即如果 $|G_1 - G_2| \leq T$ ，那么这个问题的分数就是 G_1 和 G_2 的平均值。
- 如果两个分数的差超过 T ，那么第 3 位专家将给出评分 G_3 。
- 如果 G_3 与 G_1 或 G_2 的差在公差范围内，但和两者的差并不是都在公差范围内，那么这个问题的评分是 G_3 与 G_1 或 G_2 中最接近的那一个求平均值。
- 如果 G_3 与 G_1 和 G_2 的差都在公差范围内，那么这个问题的评分是三个评分中的最大值。
- 如果 G_3 与 G_1 和 G_2 的差都在公差范围外，那么审判长将给出最终成绩 G_J 。

提交网址：

<http://t.cn/E9rDPSq>

习题 2.10 路径打印 (上海交通大学复试上机题)

给你一串路径，譬如“a\b\c a\d\e b\cst d\”，请你画出这些路径中蕴含的目录结构，子目录直接列在父目录下，并且比父目录向右缩进一格，如下所示：

```
a
  b
    c
    d
      e
  b
    cst
  d
```

同一级的需要按字母顺序排列，不能乱。

提交网址：

<http://t.cn/E9dvHs4>

习题 2.11 坠落的蚂蚁 (北京大学复试上机题)

一根长度为 1m 的木棒上有若干蚂蚁在爬动。它们的速度是 1cm/s 或静止不动；方向只有两个，即向左或向右。如果两只蚂蚁碰头，那么它们立即交换速度并继续爬动。如果有三只蚂蚁碰头，

那么两边的蚂蚁交换速度，中间的蚂蚁仍然静止。如果它们爬到了木棒的边缘（0 或 100cm 处），那么会从木棒上坠落下去。在某一时刻，蚂蚁的位置各不相同且均在整数厘米处（即 1cm, 2cm, 3cm, ..., 99cm），有且只有一只蚂蚁 A 的速度为 0，其他蚂蚁均在向左或向右爬动。给出该时刻木棒上的所有蚂蚁的位置和初速度，找出蚂蚁 A 从此刻到坠落所需要的时间。

提交网址：

<http://t.cn/E9dhoRA>

小结

本章介绍了枚举与模拟两大类在机试中频繁出现的题型的基本算法。学完本章之后，读者不仅要能较好地掌握这些基本算法，而且要能够了解机试题的考查方式和注意事项。做到这些后，机试中的大部分简单题就难不倒你了。

第 3 章 排序与查找

排序与查找可以说是计算机领域最经典的问题，相信很多读者在本科阶段都学过各式各样的排序和查找方法。排序和查找问题在考研机试真题中经常出现，本章旨在讲解一些经典问题，详细说明解题方法，使读者日后在面对此类问题时能够做到游刃有余。

3.1 排序

排序考点在历年机试考点中分布广泛。排序既是考生必须掌握的基本算法，又是考生学习其他大部分算法的前提和基础。

首先学习对基本类型的排序。对基本类型排序，是指对诸如整数、浮点数等计算机编程语言内置的基本类型进行排序的过程。

例题 3.1 排序（华中科技大学复试上机题）

题目描述：

对输入的 n 个数进行排序并输出。

输入：

输入的第一行包括一个整数 n ($1 \leq n \leq 100$)。接下来的一行包括 n 个整数。

输出：

可能有多组测试数据，对于每组数据，输出排序后的 n 个整数，每个数后面都有一个空格。每组测试数据的结果占一行。

样例输入：

```
4
1 4 3 2
```

样例输出：

```
1 2 3 4
```

提交网址：

<http://t.cn/E9dLx5K>

【分析】

这是在机试中曾经出现的关于排序考点的一个真题。面对这样的考题，读者可能很快就会联想到教科书上各种特点不同的排序算法，如快速排序、归并排序等。然而，要在机试的短时间内快速并正确地写出高效的排序算法，相信大部分考生都是力不从心的。然而，并不需要为此担心，因为 C++ 内部已为大家编写了基于快速排序的函数 `sort`，只需调用这个函数，便能轻易地完成排序。下面简单介绍 `sort` 函数。`sort(first, last, comp)` 函数有三个参数：`first` 和 `last` 为待排序序列的起始地址和结束地址；`comp` 为排序方式，可以不填写，不填写时默认为升序方式。

代码 3.1

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 100;           //数据量定义为常数

int arr[MAXN];

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            scanf("%d", &arr[i]);
        }
        sort(arr, arr + n);      //默认升序排序
        for (int i = 0; i < n; ++i) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
    return 0;
}
```

`sort` 库函数包含在 `algorithm` 头文件中，因此要在程序中引入 `algorithm` 头文件。本题中，`sort` 函数的两个参数分别代表待排序序列的起始地址 `arr` 和结束地址 `arr+n`。该函数调用完成后，`arr` 数组中的数字就已经通过快速排序变成了升序序列，之后只需对其输出即可。

如果要对给定的数组进行降序排列或以自定义的方式排列，那么应该如何操作呢？此时，

`sort` 函数的第三个参数会起作用。可以通过编写比较函数的方式来实现想要的自定义排序方式。

例题 3.2 成绩排序（清华大学复试上机题）

题目描述：

用一维数组存储学号和成绩，然后按成绩排序输出。

输入：

输入的第一行中包括一个整数 N ($1 \leq N \leq 100$)，它代表学生的个数。

接下来的 N 行中，每行包括两个整数 p 和 q ，分别代表每个学生的学号和成绩。

输出：

按照学生的成绩从小到大进行排序，并将排序后的学生信息打印出来。

如果学生的成绩相同，那么按照学号的大小从小到大排序。

样例输入：

```
3
1 90
2 87
3 92
```

样例输出：

```
2 87
1 90
3 92
```

提交网址：

<http://t.cn/E9d3ysv>

【分析】

本题和上题类似，都是以升序方式排序，但本题的排序规则多了一些限制：

- ① 本题不再是对基本类型进行排序，而是对一组基本类型进行排序。因此，需要首先将这组基本类型构建为一个结构体或类，然后对结构体或类进行排序。
- ② 排序的要求在题面中已经明确给出，即先按照成绩升序排序，成绩相同时按照学号升序排序。此时，`sort` 函数的默认排序方式无法满足本题的要求，需要编写一个自定义的比较函数才能完成上述要求。

代码 3.2

```
#include <iostream>
#include <cstdio>
```



```

#include <algorithm>

using namespace std;

struct Student {
    int number;                //学号
    int score;                 //成绩
};

const int MAXN = 100;

Student arr[MAXN];

bool Compare(Student x, Student y) {    //比较函数
    if (x.score == y.score) {           //成绩相等比较学号
        return x.number < y.number;
    } else {                            //成绩不等比较成绩
        return x.score < y.score;
    }
}

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        scanf("%d%d", &arr[i].number, &arr[i].score);
    }
    sort(arr, arr + n, Compare);        //按照比较函数排序
    for (int i = 0; i < n; ++i) {
        printf("%d %d\n", arr[i].number, arr[i].score);
    }
    return 0;
}

```

- ① 上述代码定义了一个学生结构体 *Student*，这个结构体包括两个整型变量，即学号和成绩。读者以后遇到一组基本类型数据时，要学会将其封装为一个结构体或类进行处理。
- ② 上述代码还定义了一个比较函数 *Compare*，这个比较函数实现了题目中的排序规则。考生在遇到新的排序规则时，只需记住如下这条黄金法则：当比较函数的返回值为 *true* 时，表示的是比较函数的第一个参数将会排在第二个参数前面。记住这条规则后，就可

在日后面对不同的排序规则时，设计出相应的比较函数。

例题 3.3 成绩排序（清华大学复试上机题）

题目描述：

输入任意(用户,成绩)序列，可以获得成绩从高到低或从低到高的排列，相同成绩都按先录入者排列在前的规则处理。

示例：

jack	70
peter	96
Tom	70
smith	67

从高到低 成绩

peter	96
jack	70
Tom	70
smith	67

从低到高 成绩

smith	67
jack	70
Tom	70
peter	96

输入：

输入多行，首先输入要排序的人的个数，然后输入排序方法 0（降序）或 1（升序），再分别输入他们的名字和成绩，以一个空格隔开。

输出：

按照指定方式输出名字和成绩，名字和成绩之间以一个空格隔开。

样例输入：

```
3
0
fang 90
yang 50
ning 70
```

样例输出：

```
fang 90
```

ning 70

yang 50

提交网址:

<http://t.cn/E9gyHM1>

【分析】

和上一道题相比，本题最大的特点是排序方式并不唯一，需要根据输入来确定是采用升序排序还是采用降序排序。对于本题而言，最简单的方法是，首先根据题意设计两个不同的比较函数（一个用于升序排序，一个用于降序排序），然后根据不同的输入选择不同的比较函数。

代码 3.3

```

#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

struct Student {
    string name;           //姓名
    int score;             //成绩
    int order;             //次序
};

bool CompareDescending(Student x, Student y) { //降序比较函数
    if (x.score == y.score) { //成绩相等比较次序
        return x.order < y.order;
    } else { //成绩不等比较成绩
        return x.score > y.score;
    }
}

bool CompareAscending(Student x, Student y) { //升序比较函数
    if (x.score == y.score) { //成绩相等比较次序
        return x.order < y.order;
    } else { //成绩不等比较成绩
        return x.score < y.score;
    }
}

int main() {
    int n;
    int type;
    while (scanf("%d%d", &n, &type) != EOF) {
        Student arr[n]; //数据范围未知
        for (int i = 0; i < n; ++i) {
            cin >> arr[i].name >> arr[i].score;
            arr[i].order = i;
        }
    }
}

```

```

        if (type == 0) {                                //选择比较函数
            sort(arr, arr + n, CompareDescending);
        } else {
            sort(arr, arr + n, CompareAscending);
        }

        for (int i = 0; i < n; ++i) {
            cout << arr[i].name << " " << arr[i].score << endl;
        }
    }
    return 0;
}

```

- ① 上述代码定义了一个学生结构体 *Student*，这个结构体包括一个字符串变量 *name*（姓名）及两个整型变量 *score*（成绩）和 *order*（次序）。
- ② 上述代码还定义了两个比较函数，一个用于成绩升序，一个用于成绩降序，当成绩相同时，两个比较函数都按照输入次序进行升序排序。程序会根据输入的不同选择不同的比较函数。

对于结构体或类的自定义排序，除通过设计比较函数外，还有一种常用的方法，即在结构体或类的内部重载小于号。对此方法有兴趣的读者，可以自行阅读相关资料。

相信通过本节的学习与练习，读者不会再对排序问题感到恐惧：只需用 *sort* 函数按照题目要求定义不同的排序规则。如果读者对 *sort* 函数还有疑问，或者想要深入学习，那么可以访问网址 <http://www.cplusplus.com/reference/algorithm/sort/>。

习题 3.1 特殊排序（华中科技大学复试上机题）

输入一系列整数，将其中最大的数挑出（有多个最大数时，挑出一个即可），并对剩下的数排序，如果无剩下的数，那么输出-1。

提交网址：

<http://t.cn/E9gio39>

习题 3.2 整数奇偶排序（北京大学复试上机题）

输入 10 个整数，彼此以空格分隔。重新排序后输出（也按空格分隔），要求：

- 1) 首先输出其中的奇数，并且按照从大到小的顺序排列；
- 2) 然后输出其中的偶数，并且按照从小到大的顺序排列。

提交网址：

<http://t.cn/E9glPvp>

习题 3.3 小白鼠排队（北京大学复试上机题）

有 N 只小白鼠 ($1 \leq N \leq 100$)，每只小白鼠头上戴一顶有颜色的帽子。现在称出每只白鼠的重量，要求按照白鼠重量从大到小的顺序，输出它们头上帽子的颜色。帽子的颜色用“red”“blue”等字符串表示。不同的小白鼠可以戴相同颜色的帽子。白鼠的重量用整数表示。

提交网址：

<http://t.cn/E9gXh9Z>

习题 3.4 奥运排序问题（浙江大学复试上机题）

给定国家或地区的奥运金牌数、奖牌数和人口数（百万）。
排序有 4 种方式：金牌总数、奖牌总数、金牌人口比例、奖牌人口比例。
对每个国家给出最佳的排名方式和最终排名。

提交网址：

<http://t.cn/E9gYpyl>

3.2 查找

查找是另一类必须掌握的基础算法，它不仅会在机试中直接考查，而且是其他某些算法的基础。之所以将查找和排序放在一起讲，是因为二者有较强的联系。排序的重要意义之一便是帮助人们更加方便地进行查找。如果不对数据进行排序，那么在查找某个特定的元素时，需要依次检查所有的元素，这样的方式对于单次或少量的查找来说运行效率是很高的，但查找次数较多时，如果所有元素都是有序的，那么就能更快地进行检索，而不必逐个元素地进行比较。

例题 3.4 找 x （哈尔滨工业大学复试上机题）

题目描述：

输入一个数 n ，然后输入 n 个不同的数值，再输入一个值 x ，输出这个值在数组中的下标（从 0 开始，若不在数组中则输出 -1）。

输入：

测试数据有多组。输入 n ($1 \leq n \leq 200$)，接着输入 n 个数，然后输入 x 。

输出：

对于每组输入,请输出结果。

样例输入：

```
2
1 3
0
```

样例输出：

```
-1
```

提交网址：

<http://t.cn/E9gHFnS>

【分析】

由此例可以看出，即使是在数组中查找特定数字这样的最基本的问题，也会在考研机试真题中出现，由此可以看出查找在整个算法体系中的重要性。通过此例可以了解查找涉及的几个基本要素。

- ① 查找空间。也称解空间。所谓查找，就是在查找空间中找寻符合要求的解的过程。在此例中，整个数组包含的整数集就是查找空间。
- ② 查找目标。需要一个目标来判断查找空间中的各个元素是否符合要求，以便判断查找活动是否成功。在此例中，即判断数组中的数字与目标数字是否相同。
- ③ 查找方法。利用某种特定的策略在查找空间中查找各个元素。不同的策略对查找的效率和结果有不同的影响。因此，对于某个特定的问题，要选择切实可行的策略来查找解空间，以期事半功倍。在此题中，查找方法为线性地遍历数组。

代码 3.4

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 200;

int arr[MAXN];

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            scanf("%d", &arr[i]);
        }
        int x;
        scanf("%d", &x);
        int answer = -1;
        for (int i = 0; i < n; ++i) {
            if (arr[i] == x) {
                answer = i;
                break;
            }
        }
        printf("%d\n", answer);
    }
}
```

```
return 0;
}
```

下面介绍一种不同于线性查找的查找策略，而是一种策略性、跳跃性的方法来遍历查找空间，它就是二分查找。二分查找建立在待查找元素排列有序的前提下。假设元素是按升序排列的，将序列中间位置记录的关键字与查找关键字比较，若二者相等，则查找成功；否则利用中间位置记录的关键字将序列分成前、后两个子序列，若中间位置记录的关键字大于查找关键字，则进一步查找前面的子序列，否则进一步查找后面的子序列。重复以上过程，直到找到满足条件的记录（查找成功），或到子序列不存在为止（此时查找不成功）。

由于每次查找都会舍弃一半的元素，因此二分查找的时间复杂度可由线性查找的 $O(n)$ 降至 $O(\log n)$ 。有的读者可能会有疑问，虽然二分查找的时间复杂度要低于线性查找，但排序的时间复杂度至少是 $O(n \log n)$ ，如此说来线性查找岂不是要优于二分查找？如果只要求对序列进行单次查找，那么线性查找不失为一个好策略，但如果要进行多次查找，那么线性查找将不再具有优势。假设查找次数为 m ，那么线性查找的时间复杂度为 $O(nm)$ ，二分查找的时间复杂度为 $O(n \log n + m \log n)$ ，可见当 m 特别小时线性查找很有优势，但当 m 大到一定程度时，二分查找的性能便会远远优于线性查找。

例题 3.5 查找（北京邮电大学复试上机题）

题目描述：

输入数组长度 n ，输入数组 $a[1 \dots n]$ ，输入查找次数 m ，输入查找数字 $b[1 \dots m]$ ，输出 YES 或 NO，找到则输出 YES，否则输出 NO。

输入：

输入有多组数据。

每组输入 n ，然后输入 n 个整数，再输入 m ，最后输入 m 个整数（ $1 \leq m, n \leq 100$ ）。

输出：

若在 n 个数组中，则输出 YES，否则输出 NO。

样例输入：

```
5
1 5 2 4 3
3
2 5 6
```

样例输出：

```
YES
YES
NO
```

提交网址：

<http://t.cn/E9g8aaR>

【分析】

读题并分析后，读者肯定会发现本题的数据量和查询次数都比较小，因此仍然可以使用线性查找的方法求解。然而，当读者在考场上遇到题目的数据量和查询量都比较大时，如果仍然采用线性查找的方法求解，那么肯定会导致程序超时。这时，必须使用二分查找方法乃至更高性能的查找方法。本题的目的就是向读者介绍二分查找方法。

代码 3.5

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 100;

int arr[MAXN];

bool BinarySearch(int n, int target) { //二分查找
    int left = 0;
    int right = n - 1;
    while (left <= right) {
        int middle = (left + right) / 2; //中间位置
        if (arr[middle] < target) { //小于关键字，舍弃左边
            left = middle + 1;
        } else if (target < arr[middle]) { //大于关键字，舍弃右边
            right = middle - 1;
        } else {
            return true; //查找成功
        }
    }
    return false;
}

int main() {
    int n, m;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            scanf("%d", &arr[i]);
        }
    }
}
```

```

sort(arr, arr + n);           //先排序再查找
scanf("%d", &m);
for (int i = 0; i < m; ++i) {
    int target;
    scanf("%d", &target);
    if (BinarySearch(n, target)) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }
}
return 0;
}

```

结合上面的代码和之前的讲解，相信读者可以很快地掌握二分查找的特点，并能够独立地写出相关的代码。

注意中间位置的那行代码 $\text{int middle} = (\text{left} + \text{right}) / 2$ 。通常情况下，这样写既没有错误，又方便大家理解。不过，有时会出现 left 和 right 非常接近整型最大值的情况，这时如果再这样求中间位置就会出现溢出问题，因此要用代码 $\text{int middle} = \text{left} + (\text{right} - \text{left}) / 2$ 代替。二者的功能等价，并且后者不会出现溢出问题。

习题 3.5 找最小数（北京邮电大学复试上机题）

第一行输入一个数 n ， $1 \leq n \leq 1000$ ；接着输入 n 行数据，每一行有两个数，分别是 x 和 y ，输出格式是 “ $x y$ ”。输出一组 “ $x y$ ”，这组数据在所有数据中， x 是最小的，且 x 相等时 y 是最小的。

提交网址：

<http://t.cn/E9gekWa>

习题 3.6 打印极值点下标（北京大学复试上机题）

在一个整数数组上，对于下标为 i 的整数，如果它大于所有与它相邻的整数，或者小于所有与它相邻的整数，那么称该整数为一个极值点，极值点的下标就是 i 。

提交网址：

<http://t.cn/E9ehDw4>

习题 3.7 找位置（华中科技大学复试上机题）

对给定的一个字符串，找出有重复的字符，并给出其位置。例如，对于字符串 $abcaaAB12ab12$ ，

输出如下: a,1; a,4; a,5; a,10,b,2; b,11,1,8; 1,12,2,9; 2,13。

提交网址:

<http://t.cn/E9eh4jY>

小结

本章介绍了排序和查找的基本概念，其中排序是机试中出现概率很大的一类题型，希望读者能把握其中的规律。查找部分介绍了两种简单的查找方法，即线性查找和二分查找，其中二分查找需要读者重点把握，因此它在机试中出现的概率更大。更为高效的查找方式将在关于数据结构章节的散列表部分介绍。

第 4 章 字 符 串

本章介绍一种基础数据类型——字符串，并且介绍一些字符串处理的方法及字符串匹配的方法。虽然字符串的内容非常基础，但是十分重要。希望读者能够好好学习本章的内容，为此后的学习打下良好的基础。

4.1 字符串

由于 C 语言不提供字符串这一基本类型，因此通常需要使用字符数组来代替字符串，因此操作起来会很很不方便。好在 C++ 提供了字符串（*string*）这种基本数据类型，它可以很方便地对字符串进行各种操作。

对大部分读者而言，并不需要关心 *string* 内部的实现细节，学会使用即可。

(1) *string* 的定义

为了使用 *string* 标准模板，应在代码中添加头文件 `#include <string>`。定义一个字符串的方式和定义其他基本数据类型的方式相同，如 *string str*，其中 *str* 是定义的字符串的名字：

```
string str;
```

(2) *string* 的初始化

可以直接给字符串变量赋值，如 *string str* = "hello"：

```
string str = "hello world";  
cout << str << endl;
```

示例结果

```
hello world
```

(3) *string* 的长度

返回当前字符串长度的函数有 *size()* 和 *length()*，二者功能基本相同。

```
string str = "hello world";  
int n = str.size();  
printf("%d\n", n);
```

示例结果

11

(4) *string* 元素的访问

- ① 可以像数组那样通过元素下标进行访问，下标从 0 到 *size()*-1。

```
string str = "hello world";
for (int i = 0; i < str.size(); ++i) {
    printf("%c ", str[i]);
}
printf("\n");
printf("the 7th element of str is: %d\n", str[6]);
```

示例结果

```
h e l l o   w o r l d
w
```

- ② 可以通过迭代器进行访问，迭代器类似于指针。

```
string str = "hello world";
for (string::iterator it = str.begin(); it != str.end(); ++it) {
    printf("%c ", *it);
}
printf("\n");
```

示例结果

```
h e l l o   w o r l d
```

(5) *string* 中的元素操作

string 中常用的元素操作包括在任意位置插入元素的 *insert()*、在任意位置删除元素的 *erase()* 和将字符串清空的 *clear()*。

```
string str = "hello world";
str.insert(str.size(), " end world");
cout << str << endl;
str.erase(0, 12);
cout << str << endl;
str.insert(0, "how to ");
cout << str << endl;
str.erase(7);
cout << str << endl;
str.clear();
cout << str << endl;
```

示例结果

```
hello world end world
end world
how to end world
how to
```

(6) *string* 的运算符

string 可像数字一样进行运算，但不是进行加减乘除这样的数字逻辑运算，而是可以将两个或多个字符串拼接成为一个更长的字符串。连接字符串和字符串或字符的运算符有“+”和“+=”。

```
string str1 = "to be";
string str2 = "not to be";
string str3 = "that is a question";
string str = str1 + ", "      //"to be , "
str = str + str2 + ';';      //"to be , not to be; "
str += str3;
cout << str << endl;
```

示例结果

```
to be, or not to be; that is a question
```

也可以按照字典序进行大小比较，比较运算符有<、>、<=、>=，判断两个字符串是否相等的运算符有==和!=。

```
string str1 = "abc";
string str2 = "abcd";
string str3 = "bc";
string str4 = "abcd"
if (str1 <= str2) {
    printf("str1 <= str2\n");
}
if (str2 < str3) {
    printf("str2 < str3\n");
}
if (str3 > str1) {
    printf("str3 > str1\n");
}
if (str1 != str3) {
    printf("str1 != str3\n");
}
if (str2 == str4) {
```

```
printf("str2 == str4\n");
}
```

示例结果

```
str1 <= str2
str2 < str3
str3 > str1
str1 != str3
str2 == str4
```

(7) *string* 的常用函数

- ① 在字符串中寻找特定字符或字符串的函数是 *find()*。若函数找到相应的字符或字符串则返回对应的下标，若找不到返回 *string::npos*。

```
string str = "hello world";
int found = str.find("world"); //找字符串
if (found != string::npos) {
    printf("'world' found at: %d\n", found);
}
found = str.find('l');
if (found != string::npos) { //找字符
    printf("'l' found at: %d\n", found);
}
found = str.find('.');
if (found == string::npos) {
    printf("'.' not founded");
}
```

示例结果

```
'world' found at: 6
'l' found at 2
'.' not founded
```

- ② 返回字符串的子串的函数是 *substr()*。

```
string str1 = "We think in generalities, but we live in details.";
string str2 = str1.substr(3,5);
cout << str2 << endl;
```

示例结果

```
think
```

4.2 字符串处理

字符串处理是考研机试中非常常见的一类题型，这类题型可以很好地考查学生的代码编写能力。这类题型既能够很好地考查学生对程序的输入/输出格式的理解，又能很好地考查学生的思维逻辑。此外，由于这类问题并不涉及很深的算法和数据结构，因此常常作为机试中简单的题目出现。然而，字符串处理问题常常涉及边界等问题，需要小心对待。

例题 4.1 特殊乘法（清华大学复试上机题）

题目描述：

写个算法，对两个小于 1000000000 的输入，求特殊乘法的结果。

特殊乘法举例： $123 \times 45 = 1 \times 4 + 1 \times 5 + 2 \times 4 + 2 \times 5 + 3 \times 4 + 3 \times 5$ 。

输入：

两个小于 1000000000 的数。

输出：

输入可能有多组数据，对于每组数据，*Input* 中的两个数按照题目要求的方法进行运算后，输出得到的结果。

样例输入：

123 45

样例输出：

54

提交网址：

<http://t.cn/Aisby9vW>

【分析】

对于这道题，如果把输入当作数字进行处理，那么需要逐一地取余来得到每一位的数值，这种做法过于复杂并且不直观。如果把输入当作字符串来处理，那么就会比较方便并且直观，还不容易出错。

代码 4.1

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;
```

```
int main() {
    string str1, str2;
    while (cin >> str1 >> str2) {
        int answer = 0;
        for (int i = 0; i < str1.size(); ++i) {
            for (int j = 0; j < str2.size(); ++j) {
                answer += (str1[i] - '0') * (str2[j] - '0');
            }
        }
        printf("%d\n", answer);
    }
    return 0;
}
```

例题 4.2 密码翻译（北京大学复试上机题）

题目描述：

在情报传递过程中，为了防止情报被截获，往往需要用一定的方式对情报加密，简单的加密算法虽然不足以完全避免情报被破译，但仍然能防止情报被轻易地识别。我们给出一种最简单的加密方法：对给定的一个字符串，将其中从 $a \sim y$ 的字母和从 $A \sim Y$ 的字母用其后续字母替代，将 z 和 Z 用 a 和 A 替代，可得到一个简单的加密字符串。

输入：

读取一行字符串，每个字符串的长度小于 80 个字符。

输出：

对于每组数据，输出每行字符串的加密字符串。

样例输入：

Hello! How are you!

样例输出：

Ifmmp! lpx bsf zpv!

提交网址：

<http://t.cn/Ai8bGalx>

【分析】

这道题要求将字符串中的每个英文字母进行一定的变换。需要注意的是，本题的输入是一行字符串。如果直接按照 `string` 进行输入，那么遇到空格时就会停止输入，这样会和题目的要求不符。这时，需要用到函数 `getline()` 来获取一行字符串。

代码 4.2

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

int main() {
    string str;
    while (getline(cin, str)) {
        for (int i = 0; i < str.size(); ++i) {
```



```

        if (str[i] == 'z' || str[i] == 'Z') {
            str[i] -= 25;
        } else if (('A' <= str[i] && str[i] <= 'Y') ||
                   ('a' <= str[i] && str[i] <= 'y')) {
            str[i]++;
        }
    }
    cout << str << endl;
    return 0;
}

```

例题 4.3 简单密码（北京大学复试上机题）

题目描述：

Julius Caesar 曾经使用过一种很简单的密码。对于明文中的每个字符，用字母表中后 5 位所对应的字符代替，就得到了密文。例如，字符 A 用 F 代替。下面就是密文和明文中文字符的对应关系：

密文：A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

明文：V W X Y Z A B C D E F G H I J K L M N O P Q R S T U

你的任务是对给定的密文进行解密，得到明文。需要注意的是，密文中出现的字母都是大写字母。密文中也包括非字母的字符，对这些字符不用进行解码。

输入：

输入中的测试数据不超过 100 组。每组数据都有如下形式，而且各组测试数据之间没有空白的行。

一组测试数据包括三部分：

1. 起始行：一行，包括字符串"START"。
2. 密文：一行，给出密文，密文不为空，而且其中的字符数不超过 200。
3. 结束行：一行，包括字符串"END"。

在最后一组测试数据之后有一行，包括字符串"ENDOFINPUT"。

输出：

对每组数据，都有一行输出，给出密文对应的明文。

样例输入：

```

START
NS BFW, JAJSYX TK NRUTWYFSHJ FWJ YMJ WJXZQY TK YWNANFQ HFZXJX
END
START

```

```

N BTZQI WFYMJW GJ KNWXY NS F QNYYQJ NGJWNFS ANQQFLJ YMFS XJHTSI
NS WTRJ
END
START
IFSLJW PSTBX KZQQ BJQQ YMFY HFJXFW NX RTWJ IFSLJWZTX YMFS MJ
END
ENDOFINPUT

```

样例输出：

```

IN WAR, EVENTS OF IMPORTANCE ARE THE RESULT OF TRIVIAL CAUSES
I WOULD RATHER BE FIRST IN A LITTLE IBERIAN VILLAGE THAN SECOND IN
ROME
DANGER KNOWS FULL WELL THAT CAESAR IS MORE DANGEROUS THAN HE

```

提交网址：

<http://t.cn/Ai8bih2z>

【分析】

这道题和上一题非常相似，但可以用另外一种方式来求解这类循环平移的问题：对字母执行减法操作后，再对字母个数 26 取模，便可得到答案。

代码 4.3

```

#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

int main() {
    string str;
    while (getline(cin, str)) {          //起始行
        if (str == "ENDOFINPUT") {
            break;
        }
        getline(cin, str);              //密文
        for (int i = 0; i < str.size(); ++i) {
            if ('A' <= str[i] && str[i] <= 'Z') {
                str[i] = (str[i] - 'A' - 5 + 26) % 26 + 'A';
            }
        }
    }
}

```

```

        cout << str << endl;
        getline(cin, str);           //结束行
        return 0;
    }
}

```

例题 4.4 统计字符（浙江大学复试上机题）

题目描述：

统计一个给定字符串中指定的字符出现的次数。

输入：

测试输入包含若干测试用例，每个测试用例包含 2 行，第一行为一个长度不超过 5 的字符串，第二行为一个长度不超过 80 的字符串。注意这里的字符串包含空格，即空格也可能是要求被统计的字符之一。读到 '#' 时，输入结束，相应的结果不要输出。

输出：

对每个测试用例，统计第一行中字符串的每个字符在第二行字符串中出现的次数，并按如下格式输出：

```

c0 n0
c1 n1
c2 n2
...

```

其中 c_i 是第一行中的第 i 个字符， n_i 是 c_i 出现的次数。

样例输入：

```

I
THIS IS A TEST
i ng
this is a long test string
#

```

样例输出：

```

I 2
i 3
  5
n 2
g 2

```

提交网址：

<http://t.cn/Ai8fvq4l>

【分析】

求解这道题时，可以首先用一个 `number` 数组记录所有字符出现的频率，然后按照要求进行结果的输出即可。

代码 4.4

```
#include <iostream>
#include <cstdio>
#include <string>
#include <cstring>

using namespace std;

int number[128];

int main() {
    string str1, str2;
    while (getline(cin, str1)) {
        if (str1 == "#") {
            break;
        }
        getline(cin, str2);
        memset(number, 0, sizeof(number)); //初始化数组
        for (int i = 0; i < str2.size(); ++i) {
            number[str2[i]]++;
        }
        for (int i = 0; i < str1.size(); ++i) {
            printf("%c %d\n", str1[i], number[str1[i]]);
        }
    }
    return 0;
}
```

例题 4.5 字母统计（上海交通大学复试上机题）

题目描述：

输入一行字符串，计算其中大写字母 A~Z 各自出现的次数。

输入：

案例可能有多组，每个案例输入为一行字符串。

输出：

对每个案例，按 A~Z 的顺序输出其中大写字母出现的次数。

样例输入：

DFJEIWFNQLEF0395823048+_JDLSFJDLSJFKK

样例输出：

A:0
B:0
C:0
D:3
E:2
F:5
G:0
H:0
I:1
J:4
K:2
L:3
M:0
N:1
O:0
P:0
Q:1
R:0
S:2
T:0
U:0
V:0
W:1
X:0
Y:0
Z:0

提交网址：

<http://t.cn/Ai8VB72e>

【分析】

这道题只需用一个大小为 26 的 *number* 数组，依次统计大写字母的个数，便可以得到解答。

代码 4.5

```
#include <iostream>
#include <cstdio>
#include <string>
#include <cstring>

using namespace std;

int number[26];

int main() {
    string str;
    while (cin >> str) {
        memset(number, 0, sizeof(number));
        for (int i = 0; i < str.size(); ++i) {
            if ('A' <= str[i] && str[i] <= 'Z') {
                number[str[i] - 'A']++;
            }
        }
        for (int i = 0; i < 26; ++i) {
            printf("%c:%d\n", 'A' + i, number[i]);
        }
    }
    return 0;
}
```

从上面几个例题可以看出，字符串处理问题并不涉及特别复杂的算法和数据结构，其目的是考查考生的基本代码编写能力及对数据处理的细心程度。希望读者多加练习，以便在考场上遇到这类问题时能够很好地应对。

习题 4.1 skew 数（北京大学复试上机题）

在 skew binary 表示中，第 k 位的值 $x[k]$ 表示 $x[k] \times (2^{k+1} - 1)$ 。每个位上的可能数字是 0 或 1，最后面一个非零位可以是 2。

例如， $10120_{\text{skew}} = 1 \times (2^5 - 1) + 0 \times (2^4 - 1) + 1 \times (2^3 - 1) + 2 \times (2^2 - 1) + 0 \times (2^1 - 1) = 31 + 0 + 7 + 6 + 0 = 44$ 。

前 10 个 skew 数是 0, 1, 2, 10, 11, 12, 20, 100, 101, 102。

提交网址：

<http://t.cn/Ai8lALKI>

习题 4.2 单词替换（北京大学复试上机题）

输入一个字符串，以回车结束（字符串长度小于等于 100）。该字符串由若干单词组成，单词之间用一个空格隔开，所有单词区分大小写。现在需要将其中的某个单词替换为另一个单词，并输出替换之后的字符串。

提交网址：

<http://t.cn/Ai8lycp6>

习题 4.3 首字母大写（北京大学复试上机题）

对一个字符串中的所有单词，如果单词的首字母不是大写字母，那么把单词的首字母变成大写字母。在字符串中，单词之间通过空白符分隔，空白符包括：空格（' '）、制表符（'\t'）、回车符（'\r'）和换行符（'\n'）。

提交网址：

<http://t.cn/Ai8l2hco>

习题 4.4 浮点数加法（北京大学复试上机题）

求两个浮点数相加的和。输入/输出中出现的浮点数都有如下形式：

$P_1P_2\cdots P_i.Q_1Q_2\cdots Q_j$

对于整数部分， $P_1P_2\cdots P_i$ 是一个非负整数；对于小数部分， Q_j 不等于 0。

提交网址：

<http://t.cn/Ai8l4vOj>

习题 4.5 后缀子串排序（上海交通大学复试上机题）

有一个字符串，对其后缀子串排序。例如，字符串 *grain* 的子串有 *grain*, *rain*, *ain*, *in*, *n*。然后，对各子串按字典顺序排序，即 *ain*, *grain*, *in*, *n*, *rain*。

提交网址：

<http://1t.click/VGP>

4.3 字符串匹配

字符串匹配是字符串处理中的一个经典问题。该问题是：给出两个字符串，其中一个文本串 *test*，另一个是模式串 *pattern*，如何判断模式串是否为文本串的一个子串？

对于这个问题，最直接的解法是使用暴力算法：将模式串和文本串逐字符比较，如果发生

失配，那么就从字符串的下一个字符开始重新比较，直到完全匹配或遍历完整个文本串仍未成功。暴力算法的时间复杂度为 $O(nm)$ ，其中 n 和 m 分别是文本串和模式串的长度。

暴力算法的效率不高，下面为大家介绍 KMP 算法。KMP 算法是一种改进的字符串匹配算法，它由著名计算机科学家 Knuth、Morris 和 Pratt 提出，因此人们将它称为 KMP 算法。KMP 算法的核心是，模式串失配后，并不是从下一个字符开始重新匹配，而是利用已有的信息，跳过一些不可能成功匹配的位置，以便尽量减少模式串与主串的匹配次数，进而达到快速匹配的目的。

那么在失配之后，模式串具体需要跳过多少个位置呢？它其实可以通过模式串提前计算出来，并用 `next` 数组来保存结果。于是，在失配时，不必重新计算就可直接获得移动的位置。那么如何求这个 `next` 数组呢？`next` 数组的值代表的是字符串的前缀与后缀相同的最大长度。前缀指的是除最后一个字符以外，字符串的所有头部子串；后缀指的是除第一个字符外，字符串的所有尾部子串。下面以字符串“ABABAD”为例加以说明：

- “A”的前缀、后缀都为空集，没有共有元素，可以设为特殊值-1。
- “AB”的前缀为[A]，后缀为[B]，共有元素的长度为0。
- “ABA”的前缀为[A,AB]，后缀为[BA,A]，共有元素为“A”，长度为1。
- “ABAB”的前缀为[A,AB,ABA]，后缀为[BAB,AB,B]，共有元素为“AB”，长度为2。
- “ABABA”的前缀为[A,AB,ABA,ABAB]，后缀为[BABA,ABA,BA,A]，共有元素为“ABA”，长度为3。
- “ABABAD”的前缀为[A,AB,ABA,ABAB,ABABA]，后缀为[BABAD,ABAD,BAD,AD,D]，共有元素的长度为0。

因此，“ABABAD”的 `next` 表如下所示：

Pattern	*	A	B	A	B	A	D
Index	-1	0	1	2	3	4	5
Next	N/A	-1	0	1	2	3	0

之所以将 `next[0]` 设置为-1，是因为如果第一个字符就开始失配，那么需要将模式串右移一位后继续匹配。不妨多设置一位 `p[-1]`，并让 `p[-1]` 为通配符*，它可以和任何字符匹配，于是便能等效地实现首字符失配后模式串向右移动一位的操作。

有了 `next` 表后，只需依此进行字符串的匹配。模式串发生失配时，只需按照 `next` 表进行跳转，之后重新尝试匹配，直到匹配成功或遍历完整个文本串。由于只需要遍历一遍模式串便可获得 `next` 表，且遍历一遍文本串便可获得字符串的匹配情况，所以 KMP 算法的时间复杂度为 $O(m+n)$ 。

例题 4.6 Number Sequence

题目描述：

Given two sequences of numbers: $a[1], a[2], \dots, a[N]$, and $b[1], b[2], \dots, b[M]$ ($1 \leq M \leq 10000, 1 \leq N \leq 1000000$). Your task is to find a number K which make $a[K] = b[1], a[K+1] = b[2], \dots, a[K+M-1] = b[M]$. If there are more than one K exist, output the smallest one.

输入:

The first line of input is a number T which indicate the number of cases. Each case contains three lines. The first line is two numbers N and M ($1 \leq M \leq 10000, 1 \leq N \leq 1000000$). The second line contains N integers which indicate $a[1], a[2], \dots, a[N]$. The third line contains M integers which indicate $b[1], b[2], \dots, b[M]$. All integers are in the range of $[-1000000, 1000000]$.

输出:

For each test case, you should output one line which only contain K described above. If no such K exists, output -1 instead.

样例输入:

```
2
13 5
1 2 1 2 3 1 2 3 1 3 2 1 2
1 2 3 1 3
13 5
1 2 1 2 3 1 2 3 1 3 2 1 2
1 2 3 2 1
```

样例输出:

```
6
-1
```

【题目大意】

给你两个数字序列，你的任务是在序列 a 中找到和序列 b 完全匹配的子串，如果有多个匹配的位置，输出最小的那个。

【分析】

这道题目是求字符串匹配的典型问题，只需首先根据模式串建立 *next* 表，然后去匹配文本串，即可得到问题的解。

代码 4.6

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXM = 10000;
const int MAXN = 1000000;

int nextTable[MAXM];
int pattern[MAXM];
int text[MAXN];

void GetNextTable(int m) { //创建 next 表
    int j = 0;
    nextTable[j] = -1;
    int i = nextTable[j];
    while (j < m) {
        if (i == -1 || pattern[j] == pattern[i]) {
            i++;
            j++;
            nextTable[j] = i;
        } else {
            i = nextTable[i];
        }
    }
    return ;
}

int KMP(int n, int m) {
```

```

    GetNextTable(m);
    int i = 0;
    int j = 0;
    while (i < n && j < m) {
        if (j == -1 || text[i] == pattern[j]) {    //当前字符匹配成功
            i++;
            j++;
        } else {
            j = nextTable[j];                    //当前字符匹配失败
        }
    }
    if (j == m) {                                //模式串匹配成功
        return i - j + 1;
    } else {                                     //模式串匹配失败
        return -1;
    }
}

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    while (caseNumber--) {
        int n, m;
        scanf("%d%d", &n, &m);
        for (int i = 0; i < n; ++i) {
            scanf("%d", &text[i]);
        }
        for (int j = 0; j < m; ++j) {
            scanf("%d", &pattern[j]);
        }
        printf("%d\n", KMP(n, m));
    }
    return 0;
}

```

例题 4.7 Oulipo

题目描述:

The French author Georges Perec (1936–1982) once wrote a book, *La disparition*, without the letter ‘e’.

He was a member of the Oulipo group. A quote from the book:

Tout avait Pair normal, mais tout s'affirmait faux. Tout avait Fair normal, d'abord, puis surgissait l'inhumain, l'affolant. Il aurait voulu savoir où s'articulait l'association qui l'unissait au roman: stir son tapis, assaillant à tout instant son imagination, l'intuition d'un tabou, la vision d'un mal obscur, d'un quoi vacant, d'un non-dit: la vision, l'avisio d'un oubli commandant tout, où s'abolissait la raison: tout avait l'air normal mais...

Perec would probably have scored high (or rather, low) in the following contest. People are asked to write a perhaps even meaningful text on some subject with as few occurrences of a given "word" as possible. Our task is to provide the jury with a program that counts these occurrences, in order to obtain a ranking of the competitors. These competitors often write very long texts with nonsense meaning; a sequence of 500000 consecutive 'T's is not unusual. And they never use spaces.

So we want to quickly find out how often a word, i.e., a given string, occurs in a text. More formally: given the alphabet $\{'A', 'B', 'C', \dots, 'Z'\}$ and two finite strings over that alphabet, a word W and a text T , count the number of occurrences of W in T . All the consecutive characters of W must exactly match consecutive characters of T . Occurrences may overlap.

输入:

The first line of the input file contains a single number: the number of test cases to follow. Each test case has the following format:

One line with the word W , a string over $\{'A', 'B', 'C', \dots, 'Z'\}$, with $1 \leq |W| \leq 10000$ (here $|W|$ denotes the length of the string W).

One line with the text T , a string over $\{'A', 'B', 'C', \dots, 'Z'\}$, with $|W| \leq |T| \leq 1000000$.

输出:

For every test case in the input file, the output should contain a single number, on a single line: the number of occurrences of the word W in the text T .

样例输入:

```
3
BAPC
BAPC
AZA
AZAZAZA
VERDI
AVERDXIVYERDIAN
```

样例输出:

```
1
```


3
0

【题目大意】

法国作家乔治·佩雷克（1936—1982 年）曾写过一本书 *La disparition*，书中没有字母“e”。他是 Oulipo 集团的成员。书中的部分内容引述如下：

“一切都很正常，但一切都错了。一开始一切都有正常的公平，然后非人的出现，吓坏了他。他本来想知道将他与小说联系在一起的联系在哪里：在他的地毯上，随时攻击他的想象力，禁忌的直觉，晦涩邪恶的异象，空缺的东西，一个未说出口的：愿景，遗忘的意见指挥一切，原因被废除：一切看起来都很正常但……”

佩雷克在接下来的比赛中评为高分（或低分）。要求人们在某些主题上写一个可能的文本消息，尽可能少地出现一个给定的“单词”。我们的任务是为陪审团提供一个计算这些事件的程序，以获得竞争对手的排名。这些竞争对手经常写出非常长的文本，而且含义还是无意义的，一系列 500000 个连续“T”并不罕见。而且他们从不使用空格。

因此，我们希望快速找出文本中出现的单词（即给定字符串）的频率。更正式地，给出字母表 {A, 'B', 'C', ..., 'Z'} 和该字母表上的两个有限字符串，即单词 W 和文本 T ，计算 T 中 W 的出现次数， W 的所有连续字符必须与 T 的连续字符完全匹配。可能有重叠现象。

【分析】

这道题的题目很复杂，绕了一大圈，其实就是问：给你一个文本串，再给你一个模式串，文本串中有多少个子串与模式串完全匹配。这道题其实和上道题非常相似，只不过上道题在遇到第一个匹配的地方后，会直接停止向后搜索并输出位置。本题需要遍历完整个文本串，并记录匹配成功的次数。

代码 4.7

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

const int MAXM = 10000;

int nextTable[MAXM];

void GetNextTable(string pattern) { //创建 next 表
    int m = pattern.size();
    int j = 0;
    nextTable[j] = -1;
    int i = nextTable[j];
```

```

while (j < m) {
    if (i == -1 || pattern[j] == pattern[i]) {
        i++;
        j++;
        nextTable[j] = i;
    } else {
        i = nextTable[i];
    }
}
return ;
}

int KMP(string text, string pattern) {
    GetNextTable(pattern);
    int n = text.size();
    int m = pattern.size();
    int i = 0;
    int j = 0;
    int number = 0; //记录匹配次数
    while (i < n) {
        if (j == -1 || text[i] == pattern[j]) { //当前字符匹配成功
            i++;
            j++;
        } else {
            j = nextTable[j]; //当前字符匹配失败
        }
        if (j == m) { //模式串匹配成功
            number++;
            j = nextTable[j];
        }
    }
    return number;
}

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    while (caseNumber-- > 0) {
        string pattern, text;

```

```
cin >> pattern >> text;
printf("%d\n", KMP(text, pattern));
}
return 0;
}
```

习题 4.6 字符串匹配（北京航空航天大学复试上机题）

读入数据 `string[]`，然后读入一个短字符串。要求查找 `string[]` 中和短字符串的所有匹配，输出行号、匹配字符串。匹配时不区分大小写，并且可以有一个用中括号表示的模式匹配。例如，对“`aa[123]bb`”来说，`aa1bb`, `aa2bb`, `aa3bb` 都算匹配。

提交网址：

<http://1t.click/VQG>

习题 4.7 String Matching（上海交通大学复试上机题）

问模式串在文本串中有多少个完全匹配的子串。

提交网址：

<http://1t.click/VGH>

小结

本章介绍了字符串这种基本的数据类型，并通过各大高校的机试真题介绍了一些字符串处理和字符串匹配的方法。从各高校的题目中可以看出，机试比较喜欢考查考生对字符串处理的掌握情况。这类题目通常来说不会特别难，希望读者能够多多练习，争取能够在考场上从容应对。

第 5 章 数据结构一

本章介绍基本的线性数据结构在机试题中的常见考查形式，线性数据结构包括向量、队列和栈。这些基本的数据结构是后续很多高级内容的基础，因此希望读者能够充分理解本章的内容。

5.1 向量

一般的程序设计语言都将数组作为一种内置的数据类型，它是有限个类型相同的变量的线性集合，组成数组的各个变量称为数组的元素。给每个元素分配唯一的一个下标，就能用这个下标指代该元素。同时，还可通过下标直接访问数组中的任何一个元素，并且这些访问能在常数时间内完成。

然而，数组存在一个限制，即必须在创建数组时确定其大小。因此，对于一些数据量不确定的问题来说，就有可能出现数组开得太大而导致内存浪费的现象，还有可能出现数组开得太小而导致数组访问越界的现象。那么，有没有一种能够根据数据增加或减少而不断地将数组规模扩大或缩写的变长数组呢？

1. STL-vector

向量（vector）是可以改变其大小的线性序列容器。像数组一样，向量使用连续的空间存储元素，这表明向量也可以像数组一般通过下标来访问其元素。但与数组不同的是，向量的大小可以动态变化。

向量在内部使用动态分配数组的方式来存储元素，这表明在必要时会重新分配数组，以便在插入新元素时增大其容量。重新分配数组就需要将原来的所有元素移动到新数组中，而这是相当耗时的一项工作，因此向量并不会在每次添加新元素后就重新分配数组。相反，向量每次重新分配数组时，会多分配一些额外的存储空间，以便适应未来可能的增长，因此向量的容量可能会大于其元素需要的实际容量。重新分配数组时，数组的大小呈双倍增长，以便在平均意义上，保证能够在常数时间内将新元素插入到向量的末尾。因此，与数组相比，向量其实是通过消耗更多的内存来换取存储管理效率的。读者可无需关心向量内部的实现细节，而只需学会如何使用即可。

先来看看向量这个容器的具体用法。

（1）vector 的定义

要使用向量 `vector` 标准模板，就要在代码中添加头文件，格式为 `#include <vector>`。定义一个向量 `vector` 的写法是 `vector<typename> name`，其中 `typename` 是向量元素的类

型，它可以是任意的数据类型，*name* 是所定义向量的名字。

(2) *vector* 的状态

vector 中用做判断的状态通常有两个：一个是返回当前向量是否为空的 *empty()*，另一个是返回当前向量元素个数的 *size()*。

(3) *vector* 尾部元素的添加或删除

定义一个向量后，如果要向其尾部添加新的元素，或删除尾部的元素，那么就要使用 *push_back()* 和 *pop_back()*。

(4) *vector* 元素的访问

① 可以像数组一般通过元素下标进行访问，下标从 0 到 *size()-1*。

② 可以通过迭代器进行访问，迭代器类似于指针。

(5) *vector* 元素操作

vector 中的常用元素操作包括：在任意位置插入元素的 *insert()*，在任意位置删除元素的 *erase()*，将向量清空的 *clear()*。

(6) *vector* 迭代器操作

vector 中常用的迭代器操作包括：返回向量中的首元素的迭代器 *begin()*，返回向量中尾元素后一个位置的迭代器 *end()*。

示例代码

```
#include <iostream>
#include <cstdio>
#include <vector>

using namespace std;

vector<int> myVector;

int main() {
    for (int i = 0; i < 5; ++i) {
        myVector.push_back(i);           //从尾部逐一添加元素
    }
    myVector.insert(myVector.begin(), 3, 15); //头部插入 3 个 15
    myVector.pop_back();
    for (int i = 0; i < myVector.size(); ++i) {
        printf("%d ", myVector[i]);
    }
    printf("\n");
    printf("the 5th element of myVector:%d\n", myVector[4]);
}
```



```

printf("the size of myVector: %d\n", myVector.size());
myVector.erase(myVector.begin() + 5, myVector.end()); //删除第 5 后续的元素
vector<int>::iterator it; //定义迭代器
for (it = myVector.begin(); it != myVector.end(); it++) {
    printf("%d ", *it); //遍历向量
}
printf("\n");
myVector.clear();
return 0;
}

```

示例结果

```

15 15 15 0 1 2 3
the 5th element of myVector: 1
the size of myVector: 12
15 15 15 0 1

```

2. 向量的应用

例题 5.1 完数与盈数（清华大学复试上机题）

题目描述：

一个数如果恰好等于它的各个因子（该数本身除外）之和，如 $6 = 3 + 2 + 1$ ，那么称该数为“完数”；若因子之和大于该数，则称其为“盈数”。求出 2 到 60 之间的所有“完数”和“盈数”。

输入：

题目没有任何输入。

输出：

输出 2 到 60 之间的所有“完数”和“盈数”，并以如下形式输出：

E: $e_1 e_2 e_3 \dots$ (e_i 为完数)

G: $g_1 g_2 g_3 \dots$ (g_i 为盈数)

其中两个数之间要有空格，行尾不加空格。

样例输入：

样例输出：

提交网址：

<http://t.cn/AiKEyQWW>

【分析】

由于完数和盈数的数量未知，因此不知道应该开多大的数组。如果采用 *vector* 这样的变长数组，那么就可以轻松解决问题。

代码 5.1

```
#include <iostream>
#include <cstdio>
#include <vector>

using namespace std;

vector<int> numberE;
vector<int> numberG;

int Sum(int x) {
    int sum = 0;
    for (int i = 1; i < x; ++i) {
        if (x % i == 0) {
            sum += i;
        }
    }
    return sum;
}

int main() {
    for (int i = 2; i <= 60; ++i) {
        if (i == Sum(i)) {
            numberE.push_back(i);
        } else if (i < Sum(i)) {
            numberG.push_back(i);
        }
    }
    printf("E:");
    for (int i = 0; i < numberE.size(); ++i) {
        printf(" %d", numberE[i]);
    }
    printf("\n");
    printf("G:");
    for (int i = 0; i < numberG.size(); ++i) {
        printf(" %d", numberG[i]);
    }
    printf("\n");
}
```

```
return 0;  
}
```

从上例可以看出，向量 **vector** 可在数据量尚不确定的情况下，轻松地帮助完成数据的处理。无论是对输入时的数据量不确定，还是对输出时的数据量不确定，向量都能起到很好的作用。除能够解决输入/输出数据量不确定的问题外，向量 **vector** 还能实现图论中邻接表。由于后面在介绍图论时会详细讲解 **vector** 的用法，因此本小节不设置练习题。

5.2 队列

队列（Queue）是一种线性的序列结构，其存放的元素按照线性的逻辑次序排列，但与一般的线性序列结构如数组或向量相比，队列的操作只限于逻辑上的两端，即新元素只能从队列的一端插入，并且只能从另一端删除已有的元素。允许队列插入的一端称为队列的尾部，允许队列删除的一端称为队列的头部。因此，对于元素来说，插入与删除就分别称为入队和出队。由于队列对插入和删除的位置做了上述限制，因此可以轻松地看出，队列中各个元素的操作次序必定遵守所谓的先进先出（First-In First-Out, FIFO）规则，即越早入队的元素将会越早出队，而越晚入队的元素将会越晚出队。

1. STL-queue

在正式介绍队列的应用之前，先介绍标准库中的队列模板。对于队列模板，读者不必过多地关注其实现细节，掌握其在程序中的用法即可。

(1) queue 的定义

要使用 `queue` 标准模板，就应在代码中添加头文件，其格式为 `#include <queue>`。定义一个队列 `queue` 的写法是 `queue<typename> name`，其中 `typename` 是队列元素的类型，它可以是任意数据类型，`name` 是所定义队列的名字。

(2) queue 的状态

`queue` 中常用作判断的状态有两个：一个是返回当前队列是否为空的 `empty()`，另一个是返回当前队列元素个数的 `size()`。

(3) queue 元素的添加或删除

定义一个队列后，如果要向其中添加新的元素，或删除已有的元素，那么就要使用 `push()` 和 `pop()`。

(4) queue 元素的访问

前面说过，只能对队列的头尾两端进行操作，获得头尾两端元素的函数分别是 `front()` 和 `back()`。

示例代码

```
#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

queue<int> myQueue;

int main() {
```

```

printf("the size of myQueue: %d\n", myQueue.size());
for (int i = 0; i < 10; ++i) {
    myQueue.push(i);
}
printf("the front of myQueue: %d\n", myQueue.front());
printf("the back of myQueue: %d\n", myQueue.back());
printf("the size of myQueue: %d\n", myQueue.size());
int sum = 0;
while (!myQueue.empty()) {
    sum += myQueue.front();
    myQueue.pop();
}
printf("sum: %d\n", sum);
if (myQueue.empty()) {
    printf("myQueue is empty\n");
}
printf("the size of myQueue: %d\n", myQueue.size());
return 0;
}

```

示例结果

```

the size of myQueue: 0
the front of myQueue: 0
the back of myQueue: 9
the size of myQueue: 10
sum: 45
myQueue is empty
the size of myQueue: 0

```

2. 队列的应用

例题 5.2 约瑟夫问题 No. 2

题目描述:

n 个小孩围坐成一圈，并按顺时针编号为 $1, 2, \dots, n$ ，从编号为 p 的小孩顺时针依次报数，由 1 报到 m ，报到 m 时，这名小孩从圈中出去；然后下一名小孩再从 1 报数，报到 m 时再出去。以此类推，直到所有小孩都从圈中出去。请按出去的先后顺序输出小孩的编号。

输入:

第一个是 n ，第二个是 p ，第三个是 m ($0 < m, n < 300$)。
最后一行是: 0 0 0。

输出:

按出圈的顺序输出编号, 编号之间以逗号间隔。

样例输入:

8 3 4
0 0 0

样例输出:

6,2,7,4,3,5,1,8

【分析】

这道题是著名约瑟夫问题的变体。当约瑟夫问题的数据规模不大时, 可以考虑直接利用循环队列进行模拟进行求解。然而, *queue* 只是普通的队列, 是不是说就没办法利用 *queue*, 而必须重新写一个循环队列呢? 其实可以在把 *queue* 的队首元素弹出队列后, 再次将其压入队列尾部, 用 *queue* 来模拟循环队列的效果。

代码 5.2

```
#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

int main() {
    int n, p, m;
    while (scanf("%d%d%d", &n, &p, &m)) {
        if (n == 0 && p == 0 && m == 0) {
            break;
        }
        queue<int> children;
        for (int i = 1; i <= n; ++i) {           //依次加入队列
            children.push(i);
        }
        for (int i = 1; i < p; ++i) {           //使编号为 p 的小孩在队首
            children.push(children.front());
            children.pop();
        }
        while (!children.empty()) {
            for (int i = 1; i < m; ++i) {       //m-1 个小孩依次重新入队
```

```

        children.push(children.front());
        children.pop();
    }
    if (children.size() == 1) {           //最后一个小孩的输出不同
        printf("%d\n", children.front());
    } else {
        printf("%d,", children.front());
    }
    children.pop();
}
}
return 0;
}

```

例题 5.3 猫狗收容所

题目描述：

有家动物收容所只收留猫和狗，但有特殊的收养规则。收养人有两种收养方式：

第一种为直接收养所有动物中最早进入收容所的。

第二种为选择收养的动物类型（猫或狗），并收养该种动物中最早进入收容所的。

给定一个操作序列代表所有事件。

若第一个元素为 1，则代表有动物进入收容所。第二个元素为动物的编号，正数代表狗，负数代表猫。

若第一个元素为 2，则代表有人收养动物。第二个元素若为 0，则采取第一种收养方式；若为 1，则指定收养狗；若为 -1，则指定收养猫。

请按顺序输出收养动物的序列。

若出现不合法的操作，即没有可以符合领养要求的动物，则将这次领养操作忽略。

输入：

第一个是 n ，它代表操作序列的次数。接下来是 n 行，每行有两个值 m 和 t ，分别代表题目中操作的两个元素。

输出：

按顺序输出收养动物的序列，编号之间以空格间隔。

样例输入：

```

6
1 1
1 -1
2 0
1 2

```

```
2 -1
```

```
2 1
```

样例输出:

```
1 -1 2
```

【分析】

根据题目描述的先进先出方法, 首先应该想到采用队列来实现。然而, 如果只维护一个队列, 那么对于直接收养所有动物中最早进入收容所的第一种收养方式, 实现起来就会非常简单: 只需每次取出队列头部的动物。但是, 这样做的话, 对于选择性地收养动物的第二种方式就会比较复杂, 此时需要访问整个队列, 以便找出第一个被访问的猫或第一个被访问的狗。

因此, 可以维护两个队列: 一个队列用于存放狗, 另一个队列用于存放猫。于是, 对于选择性地收养动物的第二种方式, 实现起来就会非常简单: 根据要收养的是猫还是狗, 从相应的队列中取出队列头部的动物。然而, 对于第一种收养方式, 需要判断两个队列头部的猫和狗哪个更先进入收容所, 这时需要有一个标志进行判断。因此, 每次在把动物放入它们对应的队列时, 都要给它加一个次序标志。于是, 只需比较两个队列头部动物的次序标志, 就可实现第一种收养方式。

代码 5.3

```
#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

struct animal{
    int number;                //动物编号
    int order;                 //次序标志
    animal(int n, int o): number(n), order(o) {} //构造函数
};

int main() {
    queue<animal> cats;
    queue<animal> dogs;
    int n;
    int order = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        int method, type;
        scanf("%d%d", &method, &type);
```

```

if (method == 1) {
    if (type > 0) {
        dogs.push(animal(type, order++));
    } else {
        cats.push(animal(type, order++));
    }
} else {
    if (type == 0 && !dogs.empty() && !cats.empty()) {
        if (dogs.front().order < cats.front().order) {
            printf("%d ", dogs.front().number);
            dogs.pop();
        } else {
            printf("%d ", cats.front().number);
            cats.pop();
        }
    } else if (type == 0 && dogs.empty() && !cats.empty()) {
        printf("%d ", cats.front().number);
        cats.pop();
    } else if (type == 0 && !dogs.empty() && cats.empty()) {
        printf("%d ", dogs.front().number);
        dogs.pop();
    } else if (type == 1 && !dogs.empty()) {
        printf("%d ", dogs.front().number);
        dogs.pop();
    } else if (type == -1 && !cats.empty()) {
        printf("%d ", cats.front().number);
        cats.pop();
    }
}
}
printf("\n");
return 0;
}

```

结构体 `animal` 中定义了一个与结构体同名的构造函数，因此可以按照构造函数的方式来定义结构 `animal`。例如，`animal(2,4)`代表创建了一个动物，并且该动物的编号为 **2**，次序为 **4**。

首先，上面的两个问题向读者展示了队列这一数据结构的使用。队列这一数据结构的作用，主要是求解符合先入先出访问规则的一些特殊问题。机试中考查考生掌握队列的程度时，题目中一般会提示先入先出的访问规则。

其次，与重新模拟一个队列相比，使用标准模板库中的队列模板不仅更加方便，不容易出错，而且可使考生的注意力集中在题目要求的其他操作上，而不是实现队列的细节。因此，建议读者使用标准模板库中的队列模板。

最后，队列常用于宽度优先的搜索。后面在介绍宽度优先的搜索时，会提供关于队列的练习，因此本节不设置练习题。

5.3 栈

和队列一样，栈（Stack）也是一种线性序列结构，其存放的元素也是按照线性逻辑次序排列的。然而，与一般的线性结构相比，栈的操作仅限于逻辑上特定的一端，即新元素只能从栈的一端插入，也只能从这一端删除已有的元素。禁止操作的一端称为盲端。栈中允许元素插入和删除的一端称为栈顶，禁止操作的盲端就称为栈底。于是，插入元素和删除元素就分别称为入栈和出栈。由于栈对插入和删除的位置做了上述限制，因此可以看出，栈中各个元素的操作次序必定遵守所谓的后进先出（Last-In First-Out, LIFO）规则，即越后入栈的元素将会越早出栈，越先入栈的元素将会越晚出栈。

1. STL-stack

在正式介绍栈的用法之前，下面先介绍标准库中的栈模板。使用栈模板时，读者不必过多地关注栈的实现细节，而只需关注栈在程序中的用法。

(1) *stack* 的定义

要使用 *stack* 的标准模板，需要在代码中添加头文件，格式为 `#include <stack>`。定义一个栈 *stack* 的写法是 `stack<typename> name`，其中 *typename* 是栈元素的类型，它可以是任意数据类型，*name* 是所定义栈的名字。

(2) *stack* 的状态

stack 中常用作判断的状态有两个：一个是返回当前栈是否为空的 `empty()`，另一个是返回当前栈元素个数的 `size()`。

(3) *stack* 元素的添加或删除

定义一个栈后，要向栈中添加新元素或删除已有的元素，可使用函数 `push()` 和 `pop()`。

(4) *stack* 元素的访问

前面说过，只能对栈的栈顶一端进行操作，所以只能用 `top()` 来访问栈顶元素，而不像队列那样可以访问队头和队尾，这就是 *stack* 和 *queue* 的区别，务请读者注意。

示例代码

```
#include <iostream>
#include <cstdio>
#include <stack>

using namespace std;
```



```

stack<int> myStack;

int main() {
    printf("the size of myStack: %d\n", myStack.size());
    for (int i = 0; i < 10; ++i) {
        myStack.push(i);
    }
    printf("the top of myStack: %d\n", myStack.top());
    printf("the size of myStack: %d\n", myStack.size());
    int sum = 0;
    while (!myStack.empty()) {
        sum += myStack.top();
        myStack.pop();
    }
    printf("sum: %d\n", sum);
    if (myStack.empty()) {
        printf("myStack is empty\n");
    }
    return 0;
}

```

示例结果

```

the size of myStack: 0
the top of myStack: 9
the size of myStack: 10
sum: 45
myStack is empty

```

2. 栈的应用

(1) 逆序输出

栈的用途很多，最经典的用途是求解逆序输出问题。逆序输出问题有一个明显的特点，即问题的解需要以线性序列的方式给出。然而，线性序列是逆序计算并输出的，并且这类问题的规模有时不确定，难以事先知道存放数据的容器的大小。由于这类问题既有“后进先出”的特点，又在容量方面具有自适应性，而栈完美地符合了这两个特点，因此栈非常适合于求解这类问题。

例题 5.4 Zero-complexity Transposition（上海交通大学复试上机题）

题目描述：

You are given a sequence of integer numbers. Zero-complexity transposition of the sequence is the reverse of this sequence. Your task is to write a program that prints zero-complexity transposition of the given sequence.

输入:

For each case, the first line of the input file contains one integer n -length of the sequence ($0 < n \leq 10000$). The second line contains n integers numbers $a_1, a_2, \dots, a_n$ ($-1000000000000000 \leq a_i \leq 1000000000000000$).

输出:

For each case, on the first line of the output file print the sequence in the reverse order.

样例输入:

```
5
-3 4 6 -8 9
```

样例输出:

```
9 -8 6 4 -3
```

提交网址:

<http://t.cn/AiKa2Obt>

【题目大意】

给你一个整数序列，序列的零-复杂性换位是将序列反转。你的任务是写一个程序，对这个序列进行零-复杂性换位。

【分析】

这道题是典型的逆序输出问题，求解方法是将序列中的整数按照逆序输出。由于本题的序列中的整数个数是已知的，因此可以用数组的方式进行处理。但希望利用这道题向读者介绍栈的“后进先出”特点。

不过，本题还要注意的一点是 $-1000000000000000 \leq a_i \leq 1000000000000000$ 。由题目可以看出，序列中的整数非常大，远超 *int* 的最大值 $2^{31}-1$ (214748364)，所以不能再用 32 位整型 *int* 来表示，而要用 64 位整型 *long long* 来表示，*long long* 的最大值为 $2^{64}-1$ (9223372036854775807)，能够满足题目的要求。但要注意 *long long* 的输入格式，其在 Windows 系统下为 *scanf("%I64d")*，在 Linux 系统下为 *scanf("%lld")*，具体用哪种格式，需要取决于考试评测系统。如果怕出现问题，那么可以用 C++ 的 *cin* 进行输入，它无视系统的差异。

代码 5.4

```
#include <iostream>
#include <cstdint>
```

```

#include <stack>

using namespace std;

stack<long long> sequence;

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        while (n-->0) {
            long long number;
            scanf("%lld", &number);
            sequence.push(number);
        }
        while (!sequence.empty()) {
            printf("%lld ", sequence.top());
            sequence.pop();
        }
        printf("\n");
    }
    return 0;
}

```

(2) 括号匹配

除了逆序输出问题之外，括号匹配问题也是栈的一个典型应用，即检查嵌套意义下表达式中的括号是否完全匹配。例如，对于表达式 $a/((b+c)*d)$ 和 $a/(((b+c)*d))$ ，前者的括号不匹配，而后的括号匹配。

例题 5.5 括号匹配问题

题目描述：

在某个字符串（长度不超过 100）中有左括号、右括号和大小写字母；规定（与常见的算数式子一样）任何一个左括号都从内到外与在它右边且距离最近的右括号匹配。写一个程序，找到无法匹配的左括号和右括号，输出原来的字符串，并在下一行标出不能匹配的括号。不能匹配的左括号用“\$”标注，不能匹配的右括号用“?”标注。

输入：

输入包括多组数据，每组数据一行，包含一个字符串，只包含左右括号和大小写字母，字符串长度不超过 100。

输出：

对每组输出数据，输出两行，第一行包含原始输入字符，第二行由"\$""?"和空格组成，"\$"和"?"表示与之对应的左括号和右括号不能匹配。

样例输入：

```
)(rttyy())sss)(
```

样例输出：

```
)(rttyy())sss)(
?           ?$
```

【分析】

每个右括号必定要与之前未被匹配的左括号中最靠右的一个进行匹配。因此，可以从左至右的顺序遍历整个字符串，遍历过程中遇到左括号，就将其放入栈中以等待后续右括号的匹配；遇到右括号，若此时栈非空，则栈顶左括号必定和当前右括号匹配；相反，若此时栈为空，则表示当前右括号不存在与之匹配的左括号，右括号匹配失败；当字符串全部遍历完后，若栈非空，则表明栈中的左括号不存在与之匹配的右括号，左括号匹配失败。

代码 5.5

```
#include <iostream>
#include <cstdio>
#include <string>
#include <stack>

using namespace std;

int main() {
    string str;
    while (cin >> str) {
        stack<int> brackets;
        string answer(str.size(), ' '); // 设为输入长度个空格
        for (int i = 0; i < str.size(); ++i) {
            if (str[i] == '(') {
                brackets.push(i); // 压入左括号的下标
            } else if (str[i] == ')') {
                if (!brackets.empty()) {
                    brackets.pop();
                } else {
                    answer[i] = '?'; // 右括号不匹配
                }
            }
        }
    }
}
```

```

    }
    while (!brackets.empty()) {
        answer[brackets.top()] = '$'; //左括号不匹配
        brackets.pop();
    }
    cout << str << endl;
    cout << answer << endl;
}
return 0;
}

```

(3) 表达式求值

例如求表达式 $1+2*3/4*(5-6)$ 的值。这个表达式并不能简单地按照“从左到右”的次序依次计算，因为不同运算符的优先级不同，如乘除的优先级高于加法和减法。也就是说，按照线性方式扫描表达式的时候，如果扫描到运算符时并不能确定是否执行这一运算，而是应该用栈存储起来，等来必要的信息后，才能确定运算符是否可以执行。

例题 5.6 简单计算器（浙江大学复试上机题）

题目描述：

读入一个只包含+、-、*、/的非负整数计算表达式，计算该表达式的值。

输入：

测试输入包含若干测试用例，每个测试用例占一行，每行不超过 200 个字符，整数和运算符之间用一个空格分隔。没有非法表达式。当一行中只有 0 时输入结束，相应的结果不要输出。

输出：

对每个测试用例输出一行，即该表达式的值，精确到小数点后 2 位。

样例输入：

```

1 + 2
4 + 2 * 5 - 7 / 11
0

```

样例输出：

```

3.00
13.36

```

提交网址：

<http://t.cn/AiKoGS94>

【分析】

- ① 设立运算符和运算数两个栈，一个用来存储运算符，另一个用来存储运算数。
- ② 在运算符栈中放置一个特殊运算符 $\#$ ，其优先级最低。
- ③ 将表达式尾部添加一个特殊运算符 $\$$ ，其优先级次低。
- ④ 从左至右依次遍历字符串，若遍历到运算符，则将其与运算符栈的栈顶元素进行比较，若运算符栈的栈顶的优先级小于该运算符，则将该运算符压入运算符栈；若运算符栈的栈顶的优先级大于该运算符，则弹出该栈顶运算符，从运算数栈中依次弹出运算数，完成弹出运算符对应的运算后，再将该结果压入运算数栈。
- ⑤ 若遍历到表达式中的运算数，则直接压入运算数栈。
- ⑥ 若运算符栈中仅剩两个特殊运算符 $\#$ 和 $\$$ ，则表达式运算结束，此时运算数栈中唯一的数字就是表达式的值。

代码 5.6

```
#include <iostream>
#include <cstdio>
#include <cctype>
#include <string>
#include <stack>

using namespace std;

int Priority(char c) {                                //优先级顺序#,$,+,-,*/
    if (c == '#') {
        return 0;
    } else if (c == '$') {
        return 1;
    } else if (c == '+' || c == '-') {
        return 2;
    } else {
        return 3;
    }
}

double GetNumber(string str, int& index) {           //获得下个数字
    double number = 0;
    while (isdigit(str[index])) {
        number = number * 10 + str[index] - '0';
        index++;
    }
}
```

```

        return number;
    }

    double Calculate(double x, double y, char op) {
        double result = 0;
        if (op == '+') {
            result = x + y;
        } else if (op == '-') {
            result = x - y;
        } else if (op == '*') {
            result = x * y;
        } else if (op == '/') {
            result = x / y;
        }
        return result;
    }

    int main() {
        string str;
        while (getline(cin, str)) {
            if (str == "0") {
                break;
            }
            int index = 0; //字符串下标
            stack<char> oper; //运算符栈
            stack<double> data; //运算数栈
            str += '$'; //字符串尾部添加$
            oper.push('#'); //运算符栈底添加#
            while (index < str.size()) {
                if (str[index] == ' ') {
                    index++;
                } else if (isdigit(str[index])) {
                    data.push(GetNumber(str, index));
                } else {
                    if (Priority(oper.top()) < Priority(str[index])) {
                        oper.push(str[index]);
                        index++;
                    } else {
                        double y = data.top();

```



```

        data.pop();
        double x = data.top();
        data.pop();
        data.push(Calculate(x, y, oper.top()));
        oper.pop();
    }
}
}
printf("%.2f\n", data.top());
}
return 0;
}

```

- ① 将运算符栈栈底的特殊运算符#的优先级设为最低，于是后续遇到的任何运算符的优先级都会比它高，能够顺利入栈。
- ② 将表达式尾部的特殊运算符\$的优先级设为次低，于是当遍历到表达式尾部时，若运算符栈中尚有可以运算的符号，则可继续进行运算；若运算符栈中只剩下特殊符号#，则也可保证\$能够顺利入栈，并且让程序正确地退出。
- ③ `isdigit()`函数在头文件中 `cctype` 中，该函数能够用来检查字符是否为十进制数字字符。

习题 5.1 堆栈的使用（吉林大学复试上机题）

堆栈是一种基本的数据结构。堆栈具有两种基本操作方式：`push` 和 `pop`。`push` 将一个值压入栈顶，`pop` 将栈顶的值弹出。现在我们来验证一下堆栈的使用。

提交网址：

<http://t.cn/AiKKM6F6>

习题 5.2 计算表达式（上海交通大学复试上机题）

对于一个不存在括号的表达式进行计算。

提交网址：

<http://t.cn/AiKKJjJ5>

小结

本章介绍了机试中常见的线性数据结构，包括向量、队列和栈。如果只是单纯地考查数据结构，那么机试题一般不会太难。机试题通常会结合数据结构的特点来考查考生对其的掌握程度。本章是后续章节的基础，只有奠定了数据结构这个基础，才能很好地应对后面的高深知识。

第6章 数学问题

在机试中，考生经常会面对这样一类问题，它们并不涉及很深奥的算法和数据结构，而只与数理逻辑相关，将这类题目称为数学问题。这类问题通常不需要用到特别高深的数学知识，而只需要掌握简单的数理逻辑知识。本章重点讨论机试中常常涉及的一系列数学问题，以便帮助读者很好地掌握计算机考研机试中涉及的数学问题的求解。

6.1 进制转换

进制转换是机试中最爱考查考生的一类数学题。相信读者对于二进制并不陌生，但二进制与日常生活中使用的十进制有很大的区别，因此常常需要进行进制之间的转换。要讨论进制转换，首先就要明确什么是进制。进制是由“基数”和“位权”组成的，基数是指进制采用的数码，基数为 n ，就称为 n 进制。位权是进制中每个固定位置对应的值。

下面以十进制为例加以说明。十进制数字 $d_n \cdots d_2 d_1 d_0$ 用下式表示一个特定的整数：

$$x = d_0 \times 10^0 + d_1 \times 10^1 + d_2 \times 10^2 + \cdots + d_n \times 10^n$$

在十进制中，10 是基数，10 的各次幂称为各数位的位权。用各数位的值乘以其对应的位权，然后求和，便得到了该数的值。由于十进制与二进制最常用，因此下面先讨论如何将十进制数转换为二进制数。

假设有十进制数 x ，现在要将其转换为对应的二进制数。下面先写出其二进制表示：

$$x = b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + \cdots + b_n \times 2^n$$

可见，进制转换其实就是求各个数位的值 b_i 。将上式中的 x 对 2 取模：

$$x \% 2 = (b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + \cdots + b_n \times 2^n) \% 2$$

$$x \% 2 = b_0$$

便得到 x 的二进制表示的最低数位 b_0 的值。此时，将 x 对 2 整除，便可让 b_i 从高数位向低数位移动：

$$x/2 = (b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + \cdots + b_n \times 2^n)/2$$

$$x/2 = b_1 \times 2^0 + b_2 \times 2^1 + b_3 \times 2^2 + \cdots + b_n \times 2^{n-1}$$

然后，不断地进行求模和整除运算，便可依次求得被移动到最低数位的值；以此类推，便可得到所有数位上的值。

例题 6.1 二进制数（北京邮电大学复试上机题）

题目描述：

大家都知道，数据在计算机中是以二进制形式存储的。有一天，小明在学习 C 语言时，想知道一个类型为 *unsigned int* 的数字存储在计算机中的二进制串是什么样子的。你能帮帮小明吗？注意，小明不想要二进制串中前面没有意义的 0 串，即要去掉前导 0。

输入：

每行有一个数字 n ($0 \leq n \leq 10^8$)，表示要求的二进制串。

输出：

输出共 T 行。每行输出求得的二进制串。

样例输入：

23

样例输出：

10111

提交网址：

<http://t.cn/AiCuKTOv>

【分析】

本题是一道十进制数转换成二进制数的经典题目，只需将数字 n 不断进行对 2 取模和对 2 整除运算，便可得到各个数位的值。但要注意的是，这种方法求出的数位顺序是从 b_0 到 b_n ，而数字输出需要的数位顺序是从 b_n 到 b_0 ，因此在求出各个数位的值后，要将结果逆序输出。

代码 6.1

```
#include <iostream>
#include <cstdio>
#include <vector>

using namespace std;

int main() {
    unsigned int n;
    while (scanf("%d", &n) != EOF) {
        vector<int> binary;
        while (n != 0) {
```

```

        binary.push_back(n % 2);
        n /= 2;
    }
    for (int i = binary.size() - 1; i >= 0; --i) { //逆序输出
        printf("%d", binary[i]);
    }
    printf("\n");
}
return 0;
}

```

例题 6.2 进制转换（清华大学复试上机题）

题目描述：

将一个长度最多为 30 位数字的十进制非负整数转换为二进制数。

输入：

多组数据，每行为一个长度不超过 30 位的十进制非负整数。
（注意是十进制数字的个数可能有 30 个，而不是 30bits 的整数）

输出：

每行输出对应的二进制数。

样例输入：

```

0
1
3
8

```

样例输出：

```

0
1
11
1000

```

提交网址：

<http://t.cn/AiCuoPRO>

【分析】

初看本题，读者会觉得这道题和上一道题是一样的，即都是将十进制数转换成二进制数，

但本题明确指出输入的数可能多达 30 位，因此无法再用整型数来保存该题的输入，而要用字符串来模拟数字。和此前的想法一样，只需要对用字符串模拟的数字不断进行对 2 取模和对 2 整除运算，即可求出结果。

对于取模运算，可以将其转换为对字符串中的最低数位进行取模运算，二者在功能上是等价的。

对于整除运算，需要重新写一个函数来完成字符串的除法功能。实现字符串除法函数的做法和日常在纸上实现除法的做法是一样的，即把字符串从高到低逐位除以除数。如果某位不能整除，那么就保留它除以除数的余数，余数乘以 10 后和低一位一起进行处理。这样，就能模拟出除法的效果，但这种做法可能会前置多余的 0，这时只需取首个非 0 位之后的字符串，便可得到想要的结果。

代码 6.2

```
#include <iostream>
#include <cstdio>
#include <string>
#include <vector>

using namespace std;

string Divide(string str, int x) {           //字符串除法
    int remainder = 0;                       //保存余数
    for (int i = 0; i < str.size(); ++i) {
        int current = remainder * 10 + str[i] - '0';
        str[i] = current / x + '0';
        remainder = current % x;
    }
    int pos = 0;
    while (str[pos] == '0') {                 //寻找首个非 0 下标
        pos++;
    }
    return str.substr(pos);                  //删除前置多余的 0
}

int main() {
    string str;
    while (cin >> str) {
        vector<int> binary;
        while (str.size() != 0) {
```



```

        int last = str[str.size() - 1] - '0';    //最低位的值
        binary.push_back(last % 2);             //取模运算
        str = Divide(str, 2);                   //整除运算
    }
    for (int i = binary.size() - 1; i >= 0; --i) {
        printf("%d", binary[i]);
    }
    printf("\n");
}
return 0;
}

```

例题 6.3 十进制与二进制（清华大学复试上机题）

题目描述：

对于一个十进制数 A ，将 A 转换为二进制数，然后按位逆序排列，再转换为十进制数 B ， B 即为 A 的二进制逆序数。例如，对于十进制数 173，其二进制形式为 10101101，逆序排列得到 10110101，其十进制数为 181，181 即为 173 的二进制逆序数。

输入：

一个 1000 位（即 10^{99} ）以内的十进制数。

输出：

输入的十进制数的二进制逆序数。

样例输入：

173

样例输出：

181

提交网址：

<http://t.cn/AiCuoHKg>

【分析】

本题不仅要求从十进制数转换成二进制数，还要将二进制数再转换成十进制数。

代码 6.3

```

#include <iostream>
#include <cstdio>

```

```

#include <string>
#include <vector>

using namespace std;

string Divide(string str, int x) { //字符串除法
    int remainder = 0; //保存余数
    for (int i = 0; i < str.size(); ++i) {
        int current = remainder * 10 + str[i] - '0';
        str[i] = current / x + '0';
        remainder = current % x;
    }
    int pos = 0;
    while (str[pos] == '0') { //寻找首个非0下标
        pos++;
    }
    return str.substr(pos); //删除前置多余的0
}

string Multiple(string str, int x) { //字符串乘法
    int carry = 0; //保存进位
    for (int i = str.size() - 1; i >= 0; --i) {
        int current = x * (str[i] - '0') + carry;
        str[i] = current % 10 + '0';
        carry = current / 10;
    }
    if (carry != 0) { //仍有进位
        str = "1" + str;
    }
    return str;
}

string Add(string str, int x) { //字符串加法
    int carry = x;
    for (int i = str.size() - 1; i >= 0; --i) {
        int current = (str[i] - '0') + carry;
        str[i] = current % 10 + '0';
        carry = current / 10;
    }
}

```

```

    if (carry != 0) {
        str = "1" + str;
    }
    return str;
}

int main() {
    string str;
    while (cin >> str) {
        vector<int> binary;
        while (str.size() != 0) {
            int last = str[str.size() - 1] - '0'; //最低位的值
            binary.push_back(last % 2);           //取模运算
            str = Divide(str, 2);                 //整除运算
        }
        string answer = "0";
        for (int i = 0; i < binary.size(); ++i) {
            answer = Multiple(answer, 2);         //乘法运算
            answer = Add(answer, binary[i]);      //加法运算
        }
        cout << answer << endl;
    }
    return 0;
}

```

与上题相比，本题多了一个从二进制数转换到十进制数的步骤。从二进制数转换为十进制数是从十进制数转换成二进制数的逆向操作，只需从低位到高位依次加上各数位的值，并不断乘以 2 便能够实现转换。由于是对字符串进行操作，因此需要重新编写两个函数分别实现字符串的乘法功能和字符串的加法功能。

对于乘法运算，函数的做法是将字符串从低到高逐位乘以乘数。如果某位乘以乘数后超过了 10，那么就让它对 10 取模，并保留进位。之后，将进位和高一位一起进行处理。这样，就能够模拟字符串乘法的效果，但有可能出现最高位进位的情况，这时只需在字符串首部加一个“1”便能得到想要的结果。

对于加法运算，函数的实现和乘法运算的类似，不再赘述。

例题 6.4 进制转换 2（清华大学复试上机题）

题目描述：

将 M 进制的数 X 转换为 N 进制的数并输出。

输入：

输入的第一行包括两个整数： M 和 N ($2 \leq M, N \leq 36$)。

下面的一行输入一个数 X ， X 是 M 进制的数，现在要求你将 M 进制的数 X 转换成 N 进制的数并输出。

输出：

输出 X 的 N 进制表示的数。

样例输入：

10 2

11

样例输出：

1011

提交网址：

<http://t.cn/AiCuKQ7E>

【分析】

本题和前几题都不一样，它没有明确的进制说明，而是要求将 M 进制数转换为 N 进制数。可以先将数从 M 进制转换为十进制，再从十进制转换为 N 进制。对于本题，需要注意的是，进制大于 10 时，就要用字符来表示，而这需要实现字符与数字之间的转换。

代码 6.4

```
#include <iostream>
#include <cstdio>
#include <string>
#include <vector>

using namespace std;

char IntToChar(int x) {                                //数字转字符
    if (x < 10) {
        return x + '0';
    } else {
        return x - 10 + 'a';
    }
}

int CharToInt(char c) {                                //字符转数字
    if (c >= '0' && c <= '9') {
        return c - '0';
    }
}
```

```

    } else {
        return c - 'A' + 10;
    }
}

int main() {
    int m, n;
    scanf("%d%d", &m, &n);
    string str;
    cin >> str;
    long long number = 0;
    for (int i = 0; i < str.size(); ++i) {           //m 进制转十进制
        number *= m;
        number += CharToInt(str[i]);
    }
    vector<char> answer;
    while (number != 0) {                           //十进制转 n 进制
        answer.push_back(IntToChar(number % n));
        number /= n;
    }
    for (int i = answer.size() - 1; i >= 0; --i) {   //逆序输出
        printf("%c", answer[i]);
    }
    printf("\n");
    return 0;
}

```

总之，要求十进制数 x 的 k 进制表示时，只需不断地对 x 求余（对 k ）、求商（除以 k ），即可由低到高依次得到各个数位上的数。反过来，要求由 k 进制表示的数字的十进制值时，需要依次计算各个数位上的数字与该位权重的积（第 n 位的权重为 k^{n-1} ），然后将它们依次累加，即可得到该十进制值。

习题 6.1 八进制（华中科技大学复试上机题）

输入一个整数，将其转换成八进制数并输出。

提交网址：

<http://t.cn/AiCuOlHe>

习题 6.2 又一版 $A+B$ （浙江大学复试上机题）

输入两个不超过整型定义的非负十进制整数 A 和 B ($\leq 231-1$)，输出 $A+B$ 的 m ($1 < m < 10$) 进制数。

提交网址:

<http://t.cn/AiCuOSWv>

习题 6.3 进制转换 (北京大学复试上机题)

写出一个程序, 接受一个十六进制的数值字符串, 输出该数值的十进制字符串 (注意可能存在的一个测试用例里的多组数据)。

提交网址:

<http://t.cn/AiCuig9B>

习题 6.4 数制转换 (北京大学复试上机题)

求任意两个不同进制非负整数的转换 (二进制~十六进制), 所给整数在 *long* 所能表达的范围之内。不同进制的表示符号为 (0, 1, ..., 9, a, b, ..., f) 或 (0, 1, ..., 9, A, B, ..., F)。

提交网址:

<http://t.cn/AiCu6ne4>

6.2 最大公约数与最小公倍数

1. 最大公约数

最大公约数 (Greatest Common Divisor, GCD) 是指两个或多个整数共有约数中, 最大的一个约数。求最大公约数最常用的方法是欧几里得算法, 又称辗转相除法。

若整数 g 为 a, b (不同时为 0) 的公约数, 则 g 满足

$$\begin{aligned}a &= g \times l \\b &= g \times m\end{aligned}$$

其中, l, m 为整数。同时, a 又可由 b 表示为

$$a = b \times k + r$$

其中, k 为整数, r 为 a 除以 b 后的余数。对以上三个公式做如下变形:

$$\begin{aligned}g \times l &= g \times m \times k + r \\r &= g \times (l - m \times k), (g \neq 0)\end{aligned}$$

由上式可知, a, b 的公约数 g 可以整除 a 除以 b 后的余数 r (记为 $a \bmod b$)。即 (a, b) 和 $(b, a \bmod b)$ 的公约数是一样的, 其最大公约数也必然相等。

这样, 把求 a, b 的最大公约数转换成了求 $b, a \bmod b$ 的最大公约数, 问题不变而数据规模明显变小。通过不断重复该过程, 直到问题缩小成求某个非零数与零的最大公约数。这样, 该非零数即是所求。

例题 6.5 最大公约数（哈尔滨工业大学复试上机题）

题目描述：

输入两个正整数，求其最大公约数。

输入：

测试数据有多组，每组输入两个正整数。

输出：

对于每组输入，请输出其最大公约数。

样例输入：

49 14

样例输出：

7

提交网址：

<http://t.cn/AiCuWLTS>

代码 6.5

```
#include <iostream>
#include <cstdio>

using namespace std;

int GCD(int a, int b) {
    if (b == 0) {
        return a;
    } else {
        return GCD(b, a % b);
    }
}

int main() {
    int a, b;
    while (scanf("%d%d", &a, &b) != EOF) {
        printf("%d\n", GCD(a, b));
    }
}
```

```
return 0;
}
```

2. 最小公倍数

最小公倍数（Least Common Multiple, LCM）是指要两个或多个整数公有的倍数中，除 0 之外最小的那一个公倍数。 a, b 两个数的最小公倍数为两数的乘积除以它们的最大公约数。

例题 6.6 最小公倍数

题目描述：

给定两个正整数，计算这两个数的最小公倍数。

输入：

输入包含多组测试数据，每组只有一行，包括两个不大于 1000 的正整数。

输出：

对于每个测试用例，给出这两个数的最小公倍数，每个实例输出一行。

样例输入：

10 14

样例输出：

70

代码 6.6

```
#include <iostream>
#include <cstdio>

using namespace std;

int GCD(int a, int b) {
    if (b == 0) {
        return a;
    } else {
        return GCD(b, a % b);
    }
}

int main() {
    int a, b;
    while (scanf("%d%d", &a, &b) != EOF) {
```

```

        printf("%d\n", a * b / GCD(a, b));
    }
    return 0;
}

```

习题 6.5 最简真分数（北京大学复试上机题）

给出 n 个正整数，任取两个数分别作为分子和分母组成最简真分数，编程求共有几个这样的组合。

提交网址：

<http://t.cn/AiCua2g8>

6.3 质数

本节重点介绍质数。质数也称素数，是指只能被其自身和 1 整除的正整数。如何判断一个数是否为素数呢？可以用所有小于该数的正整数去试着整除该数，若存在某个数能够整除该数，则该数不是素数；若这些数都不能整除它，则该数为素数。这一朴素的算法思想的时间复杂度为 $O(n)$ 。然而，实际上并不用测试到 $n-1$ ，而只需测试到 $\text{sqrt}(n)$ （即 $\sqrt{n}$ ）的整数即可，若到这个整数为止，所有正整数均不能整除 n ，则可以断定 n 为素数。若 n 不存在大于 $\text{sqrt}(n)$ 的因数，则该做法显然正确；假设 n 存在大于 $\text{sqrt}(n)$ 的因数 y ，则 $z = n/y$ 必同时为 n 的因数，且其值小于 $\text{sqrt}(n)$ （否则 $zy > n$ ）因此，若 n 存在一个大于 $\text{sqrt}(n)$ 的因数，则必存在一个小于 $\text{sqrt}(n)$ 的因数，所以只需测试到 $\text{sqrt}(n)$ 便可判断该数是否为质数。这样，测试一个数是否是素数的复杂度就从 $O(n)$ 降到了 $O(\text{sqrt}(n))$ 。

例题 6.7 素数判定（哈尔滨工业大学复试上机题）

题目描述：

给定一个数 n ，要求判断其是否为素数（0, 1 和负数都是非素数）。

输入：

测试数据有多组，每组输入一个数 n 。

输出：

对于每组输入，若是素数则输出 yes，否则输出 no。

样例输入：

13

样例输出：

yes

提交网址:

<http://t.cn/AiCuWEOQ>

【分析】

本题是一道判断给定数字 n 是否为质数的简单题目, 只需按照上述分析的做法, 从 2 到 $\sqrt{n}$ 依次判断能否被 n 整除, 若存在整数可以被 n 整除, 那么 n 就不是质数; 若不存在这样的整数, 那么 n 就是质数。

代码 6.7

```
#include <iostream>
#include <cstdio>
#include <cmath>

using namespace std;

bool Judge(int x) {
    //判断是否为质数
    if (x < 2) {
        //小于 2 必定不是
        return false;
    }
    int bound = sqrt(x);
    //确定判断上界
    for (int i = 2; i <= bound; ++i) {
        if (x % i == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (Judge(n)) {
            printf("yes\n");
        } else {
            printf("no\n");
        }
    }
}
```

```
return 0;
}
```

上述代码比较简单，但其中利用了一个小技巧。先计算出枚举上界，并将它赋给 *bound*，再在 *for* 循环的循环条件中与 *bound* 进行比较，而不在 *for* 循环的循环条件中直接与 *sqrt(n)* 进行比较。这是有原因的。将 *sqrt(n)* 的值赋给变量 *bound*，然后令 *i* 与 *bound* 进行比较，这样做保证了 *sqrt* 运算只进行一次。而若直接在 *for* 循环的循环条件中与 *sqrt(n)* 进行比较，则比较多少次，*sqrt(n)* 也会运算那么多次。*Sqrt()* 函数是众所周知的几个比较耗时的函数之一，采用这样的编码技巧可为程序节省不少时间。

在学会了如何判断一个数是否为素数之后，那么该如何找出 0 到 1000000 之间的所有素数呢？依次枚举每个数，然后按照上文中判断某个数是否是素数的方法来确定其是否为素数，直到得出该区间中的所有素数？当然，这样做是可行的，但这种方法的时间复杂度过高，并且整个过程显得过于暴力。下面提出一种求解该问题更优雅的方法。

首先考虑这样一个命题：若一个数不是素数，则必存在一个小于它的素数为其因数。这个命题的正确性是显而易见的。那么，若已经获得了小于一个数的所有素数，则只需确定该数不能被这些素数整除，这个数即为素数。但是，这样的做法似乎依然需要大量的枚举测试工作。正因为如此，可以换一个角度来思考：找到一个素数，就将它的所有倍数均标记成非素数；这样，当遍历到一个数时，若它未被任何小于它的素数标记为非素数，则确定其为素数。工作步骤如下：

从 2 开始遍历 2 到 1000000 的所有整数，若当前整数没有因为它是某个小于其的素数的倍数而被标记为非素数，则判定其为素数，并标记它所有的倍数为非素数。然后继续遍历下一个数，直到遍历完 2 到 1000000 内的所有整数。此时，所有未被标记成非素数的数即为要求的素数。这种算法被称为素数筛法。

例题 6.8 素数（北京航空航天大学复试上机题）

题目描述：

输入一个整数 n ($2 \leq n \leq 10000$)，要求输出所有从 1 到这个整数之间（不包括 1 和这个整数）个位为 1 的素数，若没有则输出 -1。

输入：

输入有多组数据。
每组一行，输入 n 。

输出：

输出所有从 1 到这个整数之间（不包括 1 和这个整数）个位为 1 的素数（素数之间用空格隔开，最后一个素数后面没有空格），若没有则输出 -1。

样例输入：

100

样例输出:

```
11 31 41 61 71
```

提交网址:

```
http://t.cn/AiCulqtW
```

【分析】

本题便是一道需要运用到素数筛法的典型题目。在处理输入的数据之前，可以先利用素数筛法求出 2 到 10000 内的所有素数。当得到输入 n 之后，依次判断在 $1 \sim n$ 这个区间内的素数是否符合题目条件，若符合题目条件则输出该素数，否则继续判断下一个素数，直到将该区间内所有的素数都判断完为止。

代码 6.8

```
#include <iostream>
#include <cstdio>
#include <vector>

using namespace std;

const int MAXN = 10001;

vector<int> prime;           //保存质数
bool isPrime[MAXN];         //标记数组

void Initial() {
    for (int i = 0; i < MAXN; ++i) {           //初始化
        isPrime[i] = true;
    }
    isPrime[0] = false;
    isPrime[1] = false;
    for (int i = 2; i < MAXN; ++i) {
        if (!isPrime[i]) {                     //非质数则跳过
            continue;
        }
        prime.push_back(i);
        for (int j = i * i; j < MAXN; j += i) {
            isPrime[j] = false;                 //质数的倍数为非质数
        }
    }
}
```



```

    return ;
}

int main() {
    Initial();
    int n;
    while (scanf("%d", &n) != EOF) {
        bool isOutput = false;           //判断是否有输出
        for (int i = 0; i < prime.size() && prime[i] < n; ++i) {
            if (prime[i] % 10 == 1) {
                isOutput = true;
                printf("%d ", prime[i]);
            }
        }
        if (!isOutput) {
            printf("-1");
        }
        printf("\n");
    }
    return 0;
}

```

上述素数筛法代码中使用了一个小技巧。当判定 i 为素数，要标记其倍数为非素数时，并未从 $2*i$ 开始标记，而是直接从 $i*i$ 开始标记的。原因很明显： $i*k$ ($k < i$) 必定已经在求得 k 的某个素因数（必定小于 i ）时被标记过，即 $i*k$ 同时也是 k 的素因数的倍数。所以在这里，可以直接从 i 的平方开始标记，尽可能地避免重复工作从而提高运行效率。

习题 6.6 Prime Number（上海交通大学复试上机题）

【题目大意】输出第 k 个质数。

提交网址：

<http://t.cn/AiCulrSh>

6.4 分解质因数

上一节讨论了素数的问题，而素数常用于分解质因数。每个数都可以写成一个或几个质数相乘的形式，其中每个质数都是这个数的质因数。把一个数用质因数相乘的形式表示出来，就称为分解质因数。例如，对一个数 x 分解素因数就是确定素数 $p_1, p_2, \dots, p_n$ ，使其满足

$$x = p_1^{e_1} \cdot p_2^{e_2} \cdots p_n^{e_n}$$

要学会如何求解 $p_1, p_2, \dots, p_n$ 这些质因数的方法。对于某些特定的题目，还必须要求考生掌握如何求解 $e_1, e_2, \dots, e_n$ 等幂指数。

例题 6.9 质因数的个数（清华大学复试上机题）

题目描述：

求正整数 N ($N > 1$) 的质因数的个数。相同的质因数需要重复计算。例如， $120 = 2 \times 2 \times 2 \times 3 \times 5$ ，共有 5 个质因数。

输入:

可能有多组测试数据, 每组测试数据的输入是一个正整数 N ($1 < N < 10^9$)。

输出:

对于每组数据, 输出 N 的质因数的个数。

样例输入:

120

样例输出:

5

提交网址:

<http://t.cn/Aip7J00o>

【分析】

本题的题意是将输入的整数分解素因数, 并计算每个素因数对应的幂指数之和。首先利用上节所讲的素数筛法, 预先筛选出所有可能在题目给定数据范围内是素因数的素数, 然后在程序输入待处理数字 n 时, 依次遍历所有小于 n 的素数, 判断其是否为 n 的因数。若确定某素数为 n 的因数, 则通过试除确定其对应的幂指数。

代码 6.9

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <cmath>

using namespace std;

const int MAXN = sqrt(1e9) + 1;

vector<int> prime;           //保存质数
bool isPrime[MAXN];         //标记数组

void Initial() {
    for (int i = 0; i < MAXN; ++i) {           //初始化
        isPrime[i] = true;
    }
    isPrime[0] = false;
```

```

isPrime[1] = false;
for (int i = 2; i < MAXN; ++i) {
    if (!isPrime[i]) { //非质数则跳过
        continue;
    }
    prime.push_back(i);
    for (int j = i * i; j < MAXN; j += i) {
        isPrime[j] = false; //质数的倍数为非质数
    }
}
return ;
}

int main() {
    Initial();
    int n;
    while (scanf("%d", &n) != EOF) {
        int answer = 0;
        for (int i = 0; i < prime.size() && prime[i] < n; ++i) {
            int factor = prime[i];
            while (n % factor == 0) { //不断试除
                n /= factor;
                answer++;
            }
        }
        if (n > 1) { //还存在一个质因数
            answer++;
        }
        printf("%d\n", answer);
    }
    return 0;
}

```

- ① 利用素数筛法筛出 0 到 MAXN 内的所有素数。
- ② 输入 n。
- ③ 依次测试素数能否整除 n。若能，则表明该素数为它的一个素因数。
- ④ 不断将 n 除以该素数，直到不能再被整除为止，同时统计其幂指数。
- ⑤ 遍历完预处理出来的素数后，若 n 为 1，则表明 n 的所有素因数全部被分解出来。
- ⑥ 遍历完预处理出来的素数后，若 n 不为 1，则表明 n 存在一个大于 MAXN 的因子，该

因子必为其素因子，且其幂指数必然为 1。

首先说明为什么素数筛法只需筛到 $MAXN$ ，而不需要筛到与输入数据同规模的 $1e9$ 。这样处理的理论依据是： n 至多只存在一个大于 $\sqrt{n}$ 的素因数（否则两个大于 $\sqrt{n}$ 的数相乘会大于 n ）。这样，只需将所有小于 $\sqrt{n}$ 的素数从 n 中去除，剩余的部分必为大于 $\sqrt{n}$ 的素因数。正是由于这样的原因，不必依次测试 $\sqrt{n}$ 到 n 的素数，而是在处理完小于 $\sqrt{n}$ 的素因数时，就能确定是否存在大于 $\sqrt{n}$ 的素因数，若存在其幂指数也必为 1。

习题 6.7 约数的个数（清华大学复试上机题）

输入 n 个整数，依次输出每个数的约数的个数。

提交网址：

<http://t.cn/Aip7dTUp>

习题 6.8 整除问题（上海交通大学复试上机题）

给定 n 和 a ，求最大的 k ，使 $n!$ 可被 a^k 整除但不能被 a^{k+1} 整除。

提交网址：

<http://t.cn/Aip7eHBD>

6.5 快速幂

快速幂是指快速求得 a 的 b 次方的方法。在介绍快速幂之前，对于求 a 的 b 次方这个问题，最朴素的想法便是不断地进行 b 次累乘 a ，但真的需要进行 b 次乘法运算吗？

例如，假设要计算 2^{32} 。当求得 2^{16} 之后，可以直接对 2^{16} 进行平方即可得到 2^{32} ，并不需要在 2^{16} 的基础上再进行 16 次累乘。

相应地，也不需要进行 16 次累乘来获得 2^{16} ，只需对 2^8 求平方即可。同理，要求 2^8 ，只需对 2^4 平方……以此类推，最后可得到如下计算过程：

$$\begin{aligned} 2^1 &= 2 \\ 2^2 &= 2^1 \times 2^1 = 4 \\ 2^4 &= 2^2 \times 2^2 = 16 \\ 2^8 &= 2^4 \times 2^4 = 256 \\ 2^{16} &= 2^8 \times 2^8 = 65536 \\ 2^{32} &= 2^{16} \times 2^{16} = 4294967296 \end{aligned}$$

这样，原本需要使用 32 次乘法才能完成的工作，现在就只需要 $\log(32) = 5$ 次乘法。这种高效地求某个数指定次幂的方法便是本节要介绍的快速幂。

读者可能会注意到， 2^{32} 有一种特殊性，即 32 刚好为 2 的 5 次方，所以它可以被一直二分到 1 次方为止。那么，假设要求的次数不再具有这种特殊性，快速幂还能适用吗？答案是肯定

的。下面以求 3 的 29 次方为例，继续了解快速幂的工作特点：

$$3^{29} = 3^1 \times 3^{28}$$

$$3^{29} = 3^1 \times 3^4 \times 3^{24}$$

$$3^{29} = 3^1 \times 3^4 \times 3^8 \times 3^{16}$$

其中，3 的各次幂可由之前讨论的方法求得，即对 3 的 1 次幂平方，求得 3 的 2 次幂，再对 3 的 2 次幂求平方求得 3 的 4 次幂，以此类推。求得这些 3 的各次幂后，只需将需要的次幂进行累乘即可得到答案。那么如何确定哪些 3 的次幂是需要进行累乘的呢？

这里就需要额外地介绍一个知识点，即任何一个数字 n 都可以分解为若干 2^k 之和，如 $9 = 1 + 8$, $23 = 1 + 2 + 4 + 16$ 。因此，可以先将指数 b 分解为若干 2^k 的和，如上面的例子中要求 3^{29} ，其指数 b 为 29，而 $29 = 1 + 4 + 8 + 16$ 。于是，分解的数字便是需要的 3 的次幂，即 $3^1, 3^4, 3^8$ 和 3^{16} 。再将它们累乘即可得到结果。而要将 b 分解为若干 2^k ，便是在求 b 的二进制数，其二进制位为 1 的数位代表的权重即是分解的结果。例如，29 的二进制数 11101，它可以表示为 $11101 = 2^0 + 2^2 + 2^3 + 2^4$ ，即上文中的 1, 4, 8 和 16。

例题 6.10 人见人爱 A^B

题目描述：

求 A^B 的最后三位数表示的整数。说明： A^B 的含义是“A 的 B 次方”。

输入：

输入数据包含多个测试实例，每个实例占一行，由两个正整数 A 和 B 组成（ $1 \leq A, B \leq 10000$ ）。如果 $A=0$, $B=0$ ，则表示输入数据的结束，不做处理。

输出：

对于每个测试实例，请输出 A^B 的最后三位表示的整数，每个输出占一行。

样例输入：

```
2 3
12 6
6789 10000
0 0
```

样例输出：

```
8
984
1
```

【分析】

相信读者很轻松地就会想到求一个数的后三位数的方法：只需将该数对 1000 取模即可。

在本例中，是否可以先求得 A^B 的具体数字后再求其后三位数呢？毫无疑问，这是不可行的。按照题面给出的输入规模， A^B 至多可以达到 10000 的 10000 次方，这么庞大的数字是不容易存储的，但 A^B 的后三位数其实只与 A 的后三位数和 B 有关。由于题目要求的只是结果的后三位数，因此在计算该结果的过程中产生的中间值也只需保存其后三位数即可。即在利用快速幂求 A^B 的计算过程中，只需不断地将中间结果对 1000 取模。这样，就可以既不用担心数字不能被保存，又能够获得正确的最终结果。

代码 6.10

```
#include <iostream>
#include <cstdio>

using namespace std;

int FastExponentiation(int a, int b, int mod) {
    int answer = 1;           //初始化为 1
    while (b != 0) {           //不断将 b 转换为二进制数
        if (b % 2 == 1) {       //若当前位为 1，累乘 a 的  $2^k$  次幂
            answer *= a;
            answer %= mod;      //求后三位
        }
        b /= 2;
        a *= a;                 //a 不断平方
        a %= mod;
    }
    return answer;
}

int main() {
    int a, b;
    while (scanf("%d%d", &a, &b) != EOF) {
        if (a == 0 && b == 0) {
            break;
        }
        printf("%d\n", FastExponentiation(a, b, 1000));
    }
    return 0;
}
```

习题 6.9 求 $\text{root}(N, k)$ (清华大学复试上机题)

$N < k$ 时, $\text{root}(N, k) = N$, 否则 $\text{root}(N, k) = \text{root}(N', k)$ 。 N' 为 N 的 k 进制表示的各位数字之和。输入 x, y, k , 输出 $\text{root}(x^y, k)$ 的值, $2 \leq k \leq 16$, $0 < x, y < 2000000000$, 有一半的测试点里 x^y 会溢出 int 的范围 (≥ 2000000000)。

提交网址:

<http://t.cn/AipAw4B1>

6.6 矩阵与矩阵快速幂

1. 矩阵

数学问题中另外一个常考的知识点是矩阵。考查的内容不难, 一般只考查矩阵的基本运算, 如矩阵加法、矩阵乘法、矩阵转置等。通常, 可用一个二维数组来模拟矩阵。

例题 6.11 计算两个矩阵的乘积 (哈尔滨工业大学复试上机题)

题目描述:

计算两个矩阵的乘积, 第一个是 2×3 , 第二个是 3×2 。

输入:

输入为两个矩阵, 其中一个为 2×3 的矩阵, 另一个为 3×2 的矩阵。

输出:

一个 2×2 的矩阵 (每个数字后都跟一个空格)。

样例输入:

```
1 2 3
3 4 5
6 7
8 9
10 11
```

样例输出:

```
52 58
100 112
```

提交网址:

<http://t.cn/Aip450PJ>

【分析】

本题是求两个矩阵的乘法, 只需按照矩阵乘法的规则进行模拟, 便可以求解这道题。

代码 6.11

```

#include <iostream>
#include <cstdlib>

using namespace std;

struct Matrix {
    int matrix[3][3];
    int row, col;                                     //行与列
    Matrix(int r, int c) : row(r), col(c) {}         //构造函数
};

Matrix Multiply(Matrix x, Matrix y) {                //矩阵乘法
    Matrix answer(x.row, y.col);
    for (int i = 0; i < answer.row; ++i) {
        for (int j = 0; j < answer.col; ++j) {
            answer.matrix[i][j] = 0;
            for (int k = 0; k < x.col; ++k) {
                answer.matrix[i][j] += x.matrix[i][k] * y.matrix[k][j];
            }
        }
    }
    return answer;
}

void PrintMatrix(Matrix x) {                          //输出矩阵
    for (int i = 0; i < x.row; ++i) {
        for (int j = 0; j < x.col; ++j) {
            printf("%d ", x.matrix[i][j]);
        }
        printf("\n");
    }
    return ;
}

int main() {
    Matrix x(2, 3);
    Matrix y(3, 2);

```

```

for (int i = 0; i < x.row; ++i) {
    for (int j = 0; j < x.col; ++j) {
        scanf("%d", &x.matrix[i][j]);
    }
}
for (int i = 0; i < y.row; ++i) {
    for (int j = 0; j < y.col; ++j) {
        scanf("%d", &y.matrix[i][j]);
    }
}
Matrix answer = Multiply(x, y);
PrintMatrix(answer);
return 0;
}

```

2. 矩阵快速幂

类似于普通快速幂，矩阵的快速幂就是高效求矩阵多幂次的方法，其思想和普通快速幂的思想相同。唯一不同的地方在于，对数字的快速幂而言，其初始值是 1；而对于矩阵快速幂而言，其初始值是单位矩阵。

例题 6.12 矩阵幂（北京邮电大学复试上机题）

题目描述：

给定一个 $n \times n$ 的矩阵 P ，求该矩阵的 k 次幂，即 P^k 。

输入：

第一行：两个整数 n ($2 \leq n \leq 10$) 和 k ($1 \leq k \leq 5$)，两个数字之间用一个空格隔开，含义如上所示。接下来有 n 行，每行 n 个正整数，其中，第 i 行的第 j 个整数表示矩阵中第 i 行第 j 列的矩阵元素 P_{ij} 且 $0 \leq P_{ij} \leq 10$ 。另外，数据保证最后结果不会超过 10^8 。

输出：

对于每组测试数据，输出其结果。格式为：
 n 行 n 列整数，每行数之间用空格隔开。注意，每行最后一个数后面不应该有多余的空格。
 根据指令输出结果。

样例输入：

```

2 2
9 8
9 3

```

样例输出：

153 96

108 81

提交网址:

<http://t.cn/Aip4T3HX>

【分析】

本题是一道考查矩阵求幂的题目。通过该题向读者展示矩阵快速幂的实现方法。

代码 6.12

```
#include <iostream>
#include <cstdint>

using namespace std;

struct Matrix {
    int matrix[10][10];
    int row, col;
    Matrix(int r, int c) : row(r), col(c) {}
};

Matrix Multiply(Matrix x, Matrix y) {           //矩阵乘法
    Matrix answer(x.row, y.col);
    for (int i = 0; i < answer.row; ++i) {
        for (int j = 0; j < answer.col; ++j) {
            answer.matrix[i][j] = 0;
            for (int k = 0; k < x.col; ++k) {
                answer.matrix[i][j] += x.matrix[i][k] * y.matrix[k][j];
            }
        }
    }
    return answer;
}

void PrintMatrix(Matrix x) {                    //输出矩阵
    for (int i = 0; i < x.row; ++i) {
        for (int j = 0; j < x.col; ++j) {
            if (j != 0) {
                printf(" ");
            }
        }
    }
}
```

```

        printf("%d", x.matrix[i][j]);
    }
    printf("\n");
}
return ;
}

Matrix FastExponentiation(Matrix x, int k) {
    Matrix answer(x.row, x.col);
    for (int i = 0; i < answer.row; ++i) {                //初始化为单位矩阵
        for (int j = 0; j < answer.col; ++j) {
            if (i == j) {
                answer.matrix[i][j] = 1;
            } else {
                answer.matrix[i][j] = 0;
            }
        }
    }
    while (k != 0) {                                       //不断将 k 转换为二进制
        if (k % 2 == 1) {                                  //累乘 x 的 2^k 次幂
            answer = Multiply(answer, x);
        }
        k /= 2;
        x = Multiply(x, x);                               //x 不断平方
    }
    return answer;
}

int main() {
    int n, k;
    while (scanf("%d%d", &n, &k) != EOF) {
        Matrix x(n, n);
        for (int i = 0; i < x.row; ++i) {
            for (int j = 0; j < x.col; ++j) {
                scanf("%d", &x.matrix[i][j]);
            }
        }
        Matrix answer = FastExponentiation(x, k);
        PrintMatrix(answer);
    }
}

```



```

    }
    return 0;
}

```

习题 6.10 $A + B$ for Matrices (浙江大学复试上机题)

【题目大意】计算 $A + B$ ，其中 A 和 B 是两个矩阵，然后统计矩阵中全为零的行和列的数目。

提交网址：

<http://t.cn/Aipb7GBG>

习题 6.11 递推数列 (清华大学复试上机题)

给定 a_0, a_1 ，以及 $a_n = pa_{n-1} + qa_{n-2}$ 中的 p, q ，其中 $n \geq 2$ 。

求第 k 个数对 10000 的模。

提交网址：

<http://t.cn/AipbZ2sS>

6.7 高精度整数

本书前面曾多次出现这样的问题：有些整数的数值非常巨大，以至于不能使用任何内置整数类型来保存。在前面的例子中，一直利用各种技巧回避了直接保存这种整数。但在有些问题中，不得不保存并处理数值巨大的整数，那么该如何处理呢？这就是本节要讨论的内容——高精度整数。

对于使用 Java 的考生，若机试系统允许使用 Java，那么本节的内容对你毫无用处，因为 Java 类库中内置了 *BigInteger* 类，只需查阅相关资料了解其用法并完成本节练习即可。

对于使用 C/C++ 的读者，在这里为读者提供高精度整数的模板，因此机试时按照模板“敲”就好了，需要注意的是，模板只适用于正整数的运算，小数减大数的运算会出错。模板包括加、减、乘、除、取模、输入、输出等一系列操作的实现，机试时需要用到什么运算，只需要“敲”对应的操作即可，无须写完整个模板。

```
const int MAXN = 10000;
```

```

struct BigInteger {
    int digit[MAXN];
    int length;
    BigInteger();
    BigInteger(int x);
    BigInteger(string str);
    BigInteger(const BigInteger& b);
}

```

```

BigInteger operator=(int x);
BigInteger operator=(string str);
BigInteger operator=(const BigInteger& b);
bool operator<=(const BigInteger& b);
bool operator==(const BigInteger& b);
BigInteger operator+(const BigInteger& b);
BigInteger operator-(const BigInteger& b);
BigInteger operator*(const BigInteger& b);
BigInteger operator/(const BigInteger& b);
BigInteger operator%(const BigInteger& b);
friend istream& operator>>(istream& in, BigInteger& x);
friend ostream& operator<<(ostream& out, const BigInteger& x);
};

istream& operator>>(istream& in, BigInteger& x) {
    string str;
    in >> str;
    x = str;
    return in;
}

ostream& operator<<(ostream& out, const BigInteger& x) {
    for (int i = x.length - 1; i >= 0; --i) {
        out << x.digit[i];
    }
    return out;
}

BigInteger::BigInteger() {
    memset(digit, 0, sizeof(digit));
    length = 0;
}

BigInteger::BigInteger(int x) {
    memset(digit, 0, sizeof(digit));
    length = 0;
    if (x == 0) {
        digit[length++] = x;
    }
}

```

```
while (x != 0) {
    digit[length++] = x % 10;
    x /= 10;
}

BigInteger::BigInteger(string str) {
    memset(digit, 0, sizeof(digit));
    length = str.size();
    for (int i = 0; i < length; ++i) {
        digit[i] = str[length - i - 1] - '0';
    }
}

BigInteger::BigInteger(const BigInteger& b) {
    memset(digit, 0, sizeof(digit));
    length = b.length;
    for (int i = 0; i < length; ++i) {
        digit[i] = b.digit[i];
    }
}

BigInteger BigInteger::operator=(int x) {
    memset(digit, 0, sizeof(digit));
    length = 0;
    if (x == 0) {
        digit[length++] = x;
    }
    while (x != 0) {
        digit[length++] = x % 10;
        x /= 10;
    }
    return *this;
}

BigInteger BigInteger::operator=(string str) {
    memset(digit, 0, sizeof(digit));
    length = str.size();
    for (int i = 0; i < length; ++i) {
```

```

        digit[i] = str[length - i - 1] - '0';
    }
    return *this;
}

BigInteger BigInteger::operator=(const BigInteger& b) {
    memset(digit, 0, sizeof(digit));
    length = b.length;
    for (int i = 0; i < length; ++i) {
        digit[i] = b.digit[i];
    }
    return *this;
}

bool BigInteger::operator<=(const BigInteger& b) {
    if (length < b.length) {
        return true;
    } else if (b.length < length) {
        return false;
    } else {
        for (int i = length - 1; i >= 0; --i) {
            if (digit[i] == b.digit[i]) {
                continue;
            } else {
                return digit[i] < b.digit[i];
            }
        }
    }
    return true;
}

bool BigInteger::operator==(const BigInteger& b) {
    if (length != b.length) {
        return false;
    } else {
        for (int i = length - 1; i >= 0; --i) {
            if (digit[i] != b.digit[i]) {
                return false;
            }
        }
    }
}

```

```
    }  
}  
return true;  
}  
  
BigInteger BigInteger::operator+(const BigInteger& b) {  
    BigInteger answer;  
    int carry = 0;  
    for (int i = 0; i < length || i < b.length; ++i) {  
        int current = carry + digit[i] + b.digit[i];  
        carry = current / 10;  
        answer.digit[answer.length++] = current % 10;  
    }  
    if (carry != 0) {  
        answer.digit[answer.length++] = carry;  
    }  
    return answer;  
}  
  
BigInteger BigInteger::operator-(const BigInteger& b) {  
    BigInteger answer;  
    int carry = 0;  
    for (int i = 0; i < length; ++i) {  
        int current = digit[i] - b.digit[i] - carry;  
        if (current < 0) {  
            current += 10;  
            carry = 1;  
        } else {  
            carry = 0;  
        }  
        answer.digit[answer.length++] = current;  
    }  
    while (answer.digit[answer.length] == 0 && answer.length > 1) {  
        answer.length--;  
    }  
    return answer;  
}  
  
BigInteger BigInteger::operator*(const BigInteger& b) {
```

```

BigInteger answer;
answer.length = length + b.length;
for (int i = 0; i < length; ++i) {
    for (int j = 0; j < b.length; ++j) {
        answer.digit[i + j] += digit[i] * b.digit[j];
    }
}
for (int i = 0; i < answer.length; ++i) {
    answer.digit[i + 1] += answer.digit[i] / 10;
    answer.digit[i] %= 10;
}
while (answer.digit[answer.length] == 0 && answer.length > 1) {
    answer.length--;
}
return answer;
}

```

```

BigInteger BigInteger::operator/(const BigInteger& b) {
    BigInteger answer;
    answer.length = length;
    BigInteger remainder = 0;
    BigInteger temp = b;
    for (int i = length - 1; i >= 0; --i) {
        if (!(remainder.length == 1 && remainder.digit[0] == 0)) {
            for (int j = remainder.length - 1; j >= 0; --j) {
                remainder.digit[j + 1] = remainder.digit[j];
            }
            remainder.length++;
        }
        remainder.digit[0] = digit[i];
        while (temp <= remainder) {
            remainder = remainder - temp;
            answer.digit[i]++;
        }
    }
    while (answer.digit[answer.length] == 0 && answer.length > 1) {
        answer.length--;
    }
    return answer;
}

```


例题 6.13 $a + b$ (华中科技大学复试上机题)

实现一个加法器，使其能够输出 $a + b$ 的值。

输入包括两个数 a 和 b ，其中 a 和 b 的位数不超过 1000 位。

可能有多组测试数据，对于每组数据，输出 $a + b$ 的值。

[illegible]

8
100000000000010000000000000000000000

• CXXXIII •

<http://t.cn/AipWiSG>

【分析】

这是一例考查高精度整数的典型考研机试真题，但本题只要求两个高精度整数的加法，因此只需用到上面模板中关于构造、赋值、输入、输出和加法的部分即可。

代码 6.13

```
#include <iostream>
#include <cstdio>
#include <string>
#include <cstring>

using namespace std;

const int MAXN = 10000;

struct BigInteger {
    int digit[MAXN];
    int length;
    BigInteger();
    BigInteger operator=(string str);
    BigInteger operator+(const BigInteger& b);
    friend istream& operator>>(istream& in, BigInteger& x);
    friend ostream& operator<<(ostream& out, const BigInteger& x);
};

istream& operator>>(istream& in, BigInteger& x) {
    string str;
    in >> str;
    x = str;
    return in;
}

ostream& operator<<(ostream& out, const BigInteger& x) {
    for (int i = x.length - 1; i >= 0; --i) {
        out << x.digit[i];
    }
    return out;
}
```

```

BigInteger::BigInteger() {
    memset(digit, 0, sizeof(digit));
    length = 0;
}

BigInteger BigInteger::operator=(string str) {
    memset(digit, 0, sizeof(digit));
    length = str.size();
    for (int i = 0; i < length; ++i) {
        digit[i] = str[length - i - 1] - '0';
    }
    return *this;
}

BigInteger BigInteger::operator+(const BigInteger& b) {
    BigInteger answer;
    int carry = 0;
    for (int i = 0; i < length || i < b.length; ++i) {
        int current = carry + digit[i] + b.digit[i];
        carry = current / 10;
        answer.digit[answer.length++] = current % 10;
    }
    if (carry != 0) {
        answer.digit[answer.length++] = carry;
    }
    return answer;
}

int main() {
    BigInteger a;
    BigInteger b;
    while (cin >> a >> b) {
        cout << a + b << endl;
    }
    return 0;
}

```

例题 6.14 N 的阶乘（清华大学复试上机题）

题目描述：

输入一个正整数 N ，输出 N 的阶乘。

输入：

正整数 N ($0 \leq N \leq 1000$)。

输出：

输入可能包括多组数据，对于每组输入数据，输出 N 的阶乘。

样例输入：

4
5
15

样例输出：

24
120
1307674368000

提交网址：

<http://t.cn/AipaBKQJ>

【分析】

本题考查的高精度整数的乘法问题，需要用到模板中构造、赋值、输入、输出和乘法的部分。

代码 6.14

```
#include <iostream>
#include <cstdio>
#include <string>
#include <cstring>

using namespace std;

const int MAXN = 10000;

struct BigInteger {
    int digit[MAXN];
    int length;
    BigInteger();
    BigInteger(int x);
```

```
BigInteger operator=(string str);
BigInteger operator*(const BigInteger& b);
friend istream& operator>>(istream& in, BigInteger& x);
friend ostream& operator<<(ostream& out, const BigInteger& x);
};

istream& operator>>(istream& in, BigInteger& x) {
    string str;
    in >> str;
    x = str;
    return in;
}

ostream& operator<<(ostream& out, const BigInteger& x) {
    for (int i = x.length - 1; i >= 0; --i) {
        out << x.digit[i];
    }
    return out;
}

BigInteger::BigInteger() {
    memset(digit, 0, sizeof(digit));
    length = 0;
}

BigInteger::BigInteger(int x) {
    memset(digit, 0, sizeof(digit));
    length = 0;
    if (x == 0) {
        digit[length++] = x;
    }
    while (x != 0) {
        digit[length++] = x % 10;
        x /= 10;
    }
}

BigInteger BigInteger::operator=(string str) {
    memset(digit, 0, sizeof(digit));
```

```

length = str.size();
for (int i = 0; i < length; ++i) {
    digit[i] = str[length - i - 1] - '0';
}
return *this;
}

BigInteger BigInteger::operator*(const BigInteger& b) {
    BigInteger answer;
    answer.length = length + b.length;
    for (int i = 0; i < length; ++i) {
        for (int j = 0; j < b.length; ++j) {
            answer.digit[i + j] += digit[i] * b.digit[j];
        }
    }
    for (int i = 0; i < answer.length; ++i) {
        answer.digit[i + 1] += answer.digit[i] / 10;
        answer.digit[i] %= 10;
    }
    while (answer.digit[answer.length] == 0 && answer.length > 1) {
        answer.length--;
    }
    return answer;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        BigInteger answer(1);
        for (int i = 1; i <= n; ++i) {
            answer = answer * BigInteger(i);
        }
        cout << answer << endl;
    }
    return 0;
}

```

习题 6.12 数字阶梯求和（哈尔滨工业大学复试上机题）

给定 a 和 n ，计算 $a + aa + aaa + \cdots + \underbrace{a \cdots a}_{n\text{个}}$ 的和。

提交网址：

<http://t.cn/Aipak8BQ>

习题 6.13 大整数的因子（北京大学复试上机题）

已知正整数 k 满足 $2 \leq k \leq 9$ ，现给出长度最大为 30 位的十进制非负整数 c ，求所有能整除 c 的 k 。

提交网址：

<http://t.cn/AipaFCJE>

小结

本章介绍了机试中可能涉及的数学问题的求解方法，具体包括进制转换、最大公约数、最小公倍数、素数、素因数、快速求幂等问题的求解方法。最后，讨论了高精度整数的保存及各种运算的实现。学习这些问题的求解方法后，相信读者能很好地把握机试中的数学问题。

第 7 章 贪心策略

本章介绍程序设计中非常重要的一种思想——贪心策略。贪心策略常用于求解最优化问题，其核心思想是，总是选择当前状态下最优的策略。也就是说，它并不以整体最优进行考虑，而只考虑当前这一步。因此，这种方法能够得到局部最优解，最后往往也能够获得全局较优解，但并不保证收敛到全局最优解。

既然贪心策略并不保证收敛到全局最优解，那么为什么在最优化问题还要使用它呢？原因是对于特定的最优化问题，贪心策略保证一定能够保证收敛到全局最优解。这类最优化问题具备无后效性，即某个状态以前的过程不会影响以后的状态，而只与当前状态有关。只要最优化问题具有这样的性质，便能保证使用贪心策略一定能够获得最优解。

7.1 简单贪心

例题 7.1 鸡兔同笼（北京大学复试上机题）

题目描述：

一个笼子里面关了鸡和兔子（鸡有 2 只脚，兔子有 4 只脚，没有例外）。已知笼子里面脚的总数是 a ，问笼子里面至少有多少只动物，至多有多少只动物。

输入：

每组测试数据占 1 行，每行一个正整数 a ($a < 32768$)。

输出：

输出包含 n 行，每行对应一个输入，包含两个正整数，第一个是最少的动物数，第二个是最多的动物数，两个正整数用一个空格分开。若没有满足要求的答案，则输出两个 0。

样例输入：

3

样例输出：

0 0

提交网址：

<http://t.cn/E9ewERU>

【分析】

大部分读者马上会想到：计算最多动物数的时候，优先考虑脚少的鸡；计算最少动物数的时候，优先考虑脚多的兔子，直到脚的数目无法构成兔子时再考虑脚少的鸡。这是一种典型的贪心策略。别看这种贪心的思想很简单，这种简单的贪心策略恰好能够获得题目的最优解。

在动手写代码之前，可以先深入分析一下：不论是兔子还是鸡，它们的脚数都是偶数；也就是说，当总脚数 a 是奇数时，问题无解。因此在总脚数是偶数的前提下，求最多动物数的时候，只需将它们全部考虑成鸡。然而，求最少动物数的时候，优先考虑脚多的兔子后，会出现两种情况：1. 有多余的脚但又不够构成兔子；2. 刚好没有多余的脚。对于第一种情况，剩下只可能是两只脚；第二种情况没有剩下脚。因此，可以用剩下的脚除以 2 来判断需不需要多添加一只鸡。

代码 7.1

```
#include <iostream>
#include <cstdio>

using namespace std;

int main() {
    int a;
    while (scanf("%d", &a) != EOF) {
        int minimum = 0;
        int maximum = 0;
        if (a % 2 == 0) { //偶数才有解
            minimum = a / 4 + (a % 4) / 2;
            maximum = a / 2;
        }
        printf("%d %d\n", minimum, maximum);
    }
    return 0;
}
```

例题 7.2 FatMouse' Trade

题目描述：

FatMouse prepared M pounds of cat food, ready to trade with the cats guarding the warehouse containing his favorite food, JavaBean. The warehouse has N rooms. The i -th room contains $J[i]$ pounds of JavaBeans and requires $F[i]$ pounds of cat food. FatMouse does not have to trade for all the JavaBeans in the room, instead, he may get $J[i] * a\%$ pounds of JavaBeans if he pays $F[i] * a\%$ pounds of cat food. Here a is a real number. Now he is assigning this homework to you: tell him the maximum amount of JavaBeans

he can obtain.

输入:

The input consists of multiple test cases. Each test case begins with a line containing two non-negative integers M and N . Then N lines follow, each contains two non-negative integers $J[i]$ and $F[i]$ respectively. The last test case is followed by two -1's. All integers are not greater than 1000.

输出:

For each test case, print in a single line a real number accurate up to 3 decimal places, which is the maximum amount of JavaBeans that FatMouse can obtain.

样例输入:

```
5 3
7 2
4 3
5 2
20 3
25 18
24 15
15 10
-1 -1
```

样例输出:

```
13.333
31.500
```

【题目大意】

老鼠准备了 M 磅猫粮，并且准备拿这些猫粮和守卫仓库的猫交换自己最爱的咖啡豆 (JavaBean)。仓库共有 N 个房间，每个房间里面都有咖啡豆。在第 i 个房间内有 $J[i]$ 磅咖啡豆，但相应地需要付出 $F[i]$ 磅的猫粮才能进行兑换。但是，老鼠并不一定要兑换该房间内全部的咖啡豆；相反，如果老鼠支付 $a\% * F[i]$ 的猫粮，那么可以获得 $a\% * J[i]$ 的咖啡豆。现在请你告诉它， M 磅猫粮最多可以获得多少咖啡豆。

【分析】

看到这一题，精于计算的读者可能会马上反应过来，这和日常买物品是一样的，每次购买商品的时候，优先挑选剩余物品中性价比（即重量价格之比）最高的物品，直到该物品被买完或金钱耗尽。

- ① 若当前性价比最高的物品已被买完，则继续在剩余的物品中寻找性价比最高的物品，并不断重复这个过程。
- ② 若当前金钱耗尽，则代表交易结束；此时，已经买到了最多的商品，输出这个最优解即可。

本题中，每个房间的咖啡豆 (JavaBean) 对应于不同的商品，每个房间的 $J[i]$ 代表该商品的重量， $F[i]$ 代表该商品的价格，老鼠手中的猫粮对应于金钱。

代码 7.2

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 1000;

struct JavaBean {
    double weight;
    double cost;
};

JavaBean arr[MAXN];

bool Compare(JavaBean x, JavaBean y) {
    return (x.weight / x.cost) > (y.weight / y.cost);
}

int main() {
    int m, n;
    while (scanf("%d%d", &m, &n) != EOF) {
        if (m == -1 && n == -1) {
            break;
        }
        for (int i = 0; i < n; ++i) {
            scanf("%lf%lf", &arr[i].weight, &arr[i].cost);
        }
        sort(arr, arr + n, Compare);           //性价比降序排列
        double answer = 0;
        for (int i = 0; i < n; ++i) {
            if (m >= arr[i].cost) {              //全部买下
                m -= arr[i].cost;
                answer += arr[i].weight;
            } else {                             //只能买部分
                answer += arr[i].weight * (m / arr[i].cost);
                break;
            }
        }
    }
}
```



```

    }
    }
    printf("%.3f\n", answer);
}
return 0;
}

```

学完这两道题后，读者可能会认为贪心策略不过如此，即这一策略看起来理所当然。的确，贪心策略是一种十分简单而且不需要进行过多处理的方法。对于较为简单的贪心问题，贪心策略应如何选择很容易判断；但对于复杂的贪心问题，正确的贪心策略并不是那么显而易见的，因此如何选择合适的贪心策略就需要好好思考。

例题 7.3 Senior's Gun

题目描述：

Xuejiejie is a beautiful and charming sharpshooter. She often carries n guns, and every gun has an attack power $a[i]$.

One day, Xuejiejie goes outside and comes across m monsters, and every monster has a defensive power $b[j]$.

Xuejiejie can use the gun i to kill the monster j , which satisfies $b[j] \leq a[i]$, and then she will get $a[i] - b[j]$ bonus.

Remember that every gun can be used to kill at most one monster, and obviously every monster can be killed at most once. Xuejiejie wants to gain most of the bonus. It's no need for her to kill all monsters.

输入：

In the first line there is an integer T , indicates the number of test cases. In each case: The first line contains two integers n, m . The second line contains n integers, which means every gun's attack power. The third line contains m integers, which mean every monster's defensive power. $1 \leq n, m \leq 100000$, $-10^9 \leq a[i], b[j] \leq 10^9$.

输出：

For each test case, output one integer which means the maximum of the bonus Xuejiejie could gain.

样例输入：

```

1
2 2
2 3
2 2

```

样例输出：

```

1

```

【题目大意】

薛杰杰是一位美丽迷人的神枪手。

她经常携带 n 支枪，每支枪都具有攻击力 $a[i]$ 。

有一天，薛杰杰走到外面，遇到了怪物，每个怪物都有防御力 $b[j]$ 。

在满足 $b[j] \leq a[i]$ 的情况下，薛杰杰可用 i 枪杀死怪物 j ，她会得到 $a[i] - b[j]$ 的奖金。

请记住，每支枪只能用来杀死一个怪物，而且显然每个怪物只能被杀死一次。

薛杰杰想最大化她的奖金，并且她没有必要杀死所有的怪物。

【分析】

本题的贪心策略不是很明显，每支枪只能用一次，而且要使总奖金最大，就需要在 $a[i] > b[i]$ 的情况下， $a[i]$ 的值尽量大而 $b[i]$ 的值尽量小，于是可以想到一直用剩下的枪中最厉害的去干掉剩下怪物中最弱的，以便让每一枪都能获得当前情况下最大的奖金。

代码 7.3

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 100001;

long long gun[MAXN];
long long monster[MAXN];

bool Compare(long long x, long long y) {
    return x > y;
}

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    while (caseNumber--) {
        int n, m;
        scanf("%d%d", &n, &m);
        for (int i = 0; i < n; ++i) {
            scanf("%lld", &gun[i]);
        }
        for (int i = 0; i < m; ++i) {
            scanf("%lld", &monster[i]);
        }
        sort(gun, gun + n, Compare); //枪从大到小排序
        sort(monster, monster + m); //怪物从小到大排序
        long long answer = 0;
        for (int i = 0; i < n; ++i) {
```

```

        if (i >= m || gun[i] <= monster[i]) {
            break;
        }
        answer += (gun[i] - monster[i]);
    }
    printf("%lld\n", answer);
}
return 0;
}

```

习题 7.1 代理服务器（清华大学复试上机题）

使用代理服务器能够在一定程度上隐藏客户端信息，从而保护用户在互联网上的隐私。我们知道 n 个代理服务器的 IP 地址，现在要用它们去访问 m 个服务器。这 m 个服务器的 IP 地址和访问顺序也已经给出。系统在同一时刻只能使用一个代理服务器，并要求不能用代理服务器去访问和它的 IP 地址相同的服务器（不然客户端信息很有可能会被泄露）。在这样的条件下，找到一种使用代理服务器的方案，使得代理服务器切换的次数尽可能少。

提交网址：

<http://t.cn/E9emuS9>

7.2 区间贪心

区间贪心也是一种常见的贪心策略类的题型。它是指当有多个不同的区间存在，且这些区间有可能相互重叠的时候，如何选择才能从众多区间中，选取最多的两两互不相交的区间。

例题 7.4 今年暑假不 AC

题目描述：

“今年暑假不 AC？”

“是的。”

“那你干什么呢？”

“看世界杯呀，笨蛋！”

确实如此，世界杯来了，球迷的节日也来了，估计很多 ACMer 也会抛开计算机，奔向电视了。作为球迷，一定想看尽量多的完整的比赛。当然，作为新时代的好青年，你一定还会看一些其他的节目，如《新闻联播》（永远不要忘记关心国家大事）《非常 6+7》《超级女声》及王小丫主持的《开心辞典》等，假设你已经知道了所有你喜欢看的电视节目的转播时间表，你会合理安排吗？（目标是能看尽量多的完整节目。）

输入：

输入数据包含多个测试实例，每个测试实例的第一行只有一个整数 n ($n \leq 100$)，表示你喜欢看的节目的总数；然后是 n 行数据，每行包括两个数据 $T_{i,s}$ 和 $T_{i,e}$ ($1 \leq i \leq n$)，分别表示第 i 个节目的开始和结束时间，为简化问题，每个时间都用一个正整数表示。 $n=0$ 表示输入结束，不做处理。

输出：

对于每个测试实例，输出能完整看到的电视节目的个数，每个测试实例的输出占一行。

样例输入：

```
12
1 3
3 4
0 7
3 8
15 19
15 20
10 15
8 18
6 12
5 10
4 14
2 9
0
```

样例输出：

```
5
```

【分析】

读者应该已经发现，此题的贪心策略不再像之前的题目那样显而易见。在继续阅读之前，请读者先考虑一下这道题应该用何种贪心策略。

首先思考这样一个问题：第一个节目应该选什么？

- ① 选择开始时间最早的：如果有电视节目 $A[0,5], B[1,2], C[3,4]$ ，那么显然选择最先开始的节目并不一定能够得到最优解。
- ② 选择持续时间最短的：如果有电视节目 $A[0,10], B[11,20], C[9,12]$ ，那么显然选择持续时间最短的节目也并不一定能够得到最优解。
- ③ 选择结束时间最早的：在以上两组案例中优先选择结束时间最早的节目，是可以得到最优解的。那么它是否就真的是我们所需要的贪心策略呢？

在最优解中，观看的第一个的节目一定是所有节目中结束时间最早的那个节目。因为这样才能保证看完第一个节目后，剩余观看时间的最大化。那么在看完第一个节目后，如法炮制，在剩余观看时间内选择结束时间最早且仍能观看的那个节目。以此类推，直到剩余观看时间内没有任何节目可以看为止。通过不断地利用这种贪心策略，就能完成最优解的求解。

因此，具体的做法可以是首先对所有节目按照结束时间的早晚进行升序排序，然后按照这一顺序逐一对节目进行判断：

- ① 如果当前时间能够观看该节目（即当前时间小于等于该节目的开始时间），那么就选择观看该节目，并将当前时间更新为该节目的结束时间。
- ② 如果当前时间不能够观看该节目（当前时间已超过该节目的开始时间），那么就选择放弃观看该节目，进而去判断下一个节目是否符合要求。

代码 7.4

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

struct Program {
    int startTime;
    int endTime ;
};

const int MAXN = 100;

Program arr[MAXN];

bool Compare(Program x, Program y) {
    return x.endTime < y.endTime;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        for (int i = 0; i < n; ++i) {
            scanf("%d%d", &arr[i].startTime, &arr[i].endTime);
        }
        sort(arr, arr + n, Compare);           //按结束时间升序排序
        int currentTime = 0;                    //设置当前时间
        int answer = 0;
        for (int i = 0; i < n; ++i) {
            if (currentTime <= arr[i].startTime) {
                currentTime = arr[i].endTime;
                answer++;
            }
        }
        printf("%d\n", answer);
    }
}
```



```
return 0;
}
```

例题 7.5 Case of Fugitive

题目描述:

Andrewid the Android is a galaxy-famous detective. He is now chasing a criminal hiding on the planet Oxa-5, the planet almost fully covered with water.

The only dry land there is an archipelago of n narrow islands located in a row. For more comfort let's represent them as non-intersecting segments on a straight line: island i has coordinates $[l_i, r_i]$, besides, $r_i < l_{i+1}$ for $1 \leq i \leq n-1$.

To reach the goal, Andrewid needs to place a bridge between each pair of adjacent islands. A bridge of length a can be placed between the i -th and the $(i+1)$ -th islands, if there are such coordinates of x and y , that $l_i \leq x \leq r_i$, $l_{i+1} \leq y \leq r_{i+1}$ and $y - x = a$.

The detective was supplied with m bridges, each bridge can be used at most once. Help him determine whether the bridges he got are enough to connect each pair of adjacent islands.

输入:

The first line contains integers n ($2 \leq n \leq 2 \times 10^5$) and m ($1 \leq m \leq 2 \times 10^5$)

— the number of islands and bridges.

Next n lines each contain two integers l_i and r_i ($1 \leq l_i \leq r_i \leq 10^{18}$)

— the coordinates of the island endpoints.

The last line contains m integer numbers $a_1, a_2, \dots, a_m$ ($1 \leq a_i \leq 10^{18}$)

— the lengths of the bridges that Andrewid got.

输出:

If it is impossible to place a bridge between each pair of adjacent islands in the required manner, print on a single line "No" (without the quotes), otherwise print in the first line "Yes" (without the quotes), and in the second line print $n-1$ numbers $b_1, b_2, \dots, b_{n-1}$, which mean that between islands i and $i+1$ there must be used a bridge number b_i .

If there are multiple correct answers, print any of them. Note that in this problem it is necessary to print "Yes" and "No" in correct case.

样例输入:

```
4 4
1 4
7 8
9 10
12 14
4 5 3 8
```

样例输出:

```
Yes
2 3 1
```

提示:

In the sample test you can, for example, place the second bridge between points 3 and 8, place the third bridge between points 7 and 10 and place the first bridge between points 10 and 14.

【题目大意】

安卓的安德鲁德是一位著名的银河侦探。他现在正在追捕一名隐藏在 Oxa-5 行星上的罪犯，这颗行星几乎完全被水覆盖。

唯一的旱地是一个由 n 个狭窄岛屿组成的群岛。为了更加便于表述，将它们表示为直线上不相交的线段：岛屿的坐标为 $[l_i, r_i]$ ，对于所有 $1 \leq i \leq n-1$ 都有 $r_i < l_{i+1}$ 。

为了达到目标，安德鲁德需要在每对相邻岛屿之间架一座桥。长度为 a 的桥可以架在第 i 和第 $i+1$ 号岛屿之间，如果它们的坐标为 x 和 y ，则有 $l_i \leq x \leq r_i$ ， $l_{i+1} \leq y \leq r_{i+1}$ 和 $y - x = a$ 。

侦探可以用 m 座桥，每座桥最多可以使用一次。请帮助他确定他可用的桥梁是否能够连接每对相邻的岛屿。

【分析】

这道题要比上道题复杂，贪心策略也更加难以想到。首先，从题目得知，在岛屿与岛屿之间架设桥梁是有条件的，比如说第一个岛屿为“1 4”，第二个岛屿为“7 8”，那么桥梁最短必须为 $7 - 4 = 3$ ，最长不能超过 $8 - 1 = 7$ 。共有 n 个岛屿，所以可以得到 $n-1$ 个可以架桥的长度区间。

对于每个区间，设立最大值 *maximum* 与最小值 *minimum*，并对区间按照最小值 *minimum* 进行排序，再对桥的长度 *length* 进行排序。

之后，按照桥从小到大的方式来选择搭建的区间，在所有 $\text{minimum} \leq \text{length} \leq \text{maximum}$ 的区间中，选择 *maximum* 最小的那个作为该桥搭建的区间即可。

由于本题还会用到后面才介绍的优先队列，读者如果觉得学习起来比较困难，那么可以先行跳过。

代码 7.5

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <algorithm>
#include <queue>

using namespace std;

const int MAXN = 200001;

struct Island {
    long long left;           //岛屿左端点
    long long right;          //岛屿右端点
};

struct Bridge {
    long long length;          //桥的长度
    long long index;           //桥的编号
```

```

};

struct Interval {
    long long minimum;           //区间最小值
    long long maximum;          //区间最大值
    long long index;             //区间编号
    bool operator< (Interval x) const {
        return maximum > x.maximum;
    }
};

bool IntervalCompare(Interval x, Interval y) {
    if (x.minimum == y.minimum) {
        return x.maximum < y.maximum;
    } else {
        return x.minimum < y.minimum;
    }
}

bool BridgeCompare(Bridge x, Bridge y) {
    return x.length < y.length;
}

Island island[MAXN];
Bridge bridge[MAXN];
Interval interval[MAXN];
long long answer[MAXN];

bool Solve(int n, int m) {
    priority_queue<Interval> myQueue;
    int position = 0;           //当前区间下标
    int number = 0;             //搭建桥的数目
    for (int i = 0; i < m; ++i) {
        while (myQueue.top().maximum < bridge[i].length
            && !myQueue.empty()) {
            myQueue.pop();       //当前区间无法搭建
        }
        while (position < n - 1 &&
            interval[position].minimum <= bridge[i].length &&

```

```

        interval[position].maximum >= bridge[i].length) {
            myQueue.push(interval[position]);
            position++;
        }
        if (!myQueue.empty()) {
            Interval current = myQueue.top();
            myQueue.pop();
            answer[current.index] = bridge[i].index;
            number++;
        }
    }
    return number == n - 1;    //判断桥数与区间数是否相等
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        memset(island, 0, sizeof(island));
        memset(bridge, 0, sizeof(bridge));
        memset(interval, 0, sizeof(interval));
        memset(answer, 0, sizeof(answer));
        for (int i = 0; i < n; ++i) {
            scanf("%lld%lld", &island[i].left, &island[i].right);
        }
        for (int i = 0; i < m; ++i) {
            scanf("%lld", &bridge[i].length);
            bridge[i].index = i + 1;
        }
        for (int i = 0; i < n - 1; ++i) {
            interval[i].minimum = island[i + 1].left - island[i].right;
            interval[i].maximum = island[i + 1].right - island[i].left;
            interval[i].index = i;
        }
        sort(interval, interval + n - 1, IntervalCompare);
        sort(bridge, bridge + m, BridgeCompare);
        if (Solve(n, m)) {
            printf("Yes\n");
            for (int i = 0; i < n - 1; ++i) {
                printf("%lld\n", answer[i]);
            }
        }
    }
}

```

```
    }  
    } else {  
        printf("No\n");  
    }  
}  
return 0;  
}
```

通过本题的学习，希望读者能够了解贪心策略的思想是比较简单的，但在面对不同问题时，如何选择合适的贪心策略则需要一定的技巧。希望读者可以在之后的练习中慢慢总结经验，以便日后能轻松应对考场遇到的贪心策略问题。

习题 7.2 To Fill or Not to Fill（浙江大学复试上机题）

【题目大意】由于有高速公路，从杭州开车到任何其他城市都很方便。但由于汽车的油箱容量有限，不得不在途中找加油站加油。在不同的加油站，加油的价格可能不同。请设计最便宜的路线。

提交网址：

<http://t.cn/E9ewERU>

小结

本章介绍了常常用来求解最优化问题的贪心策略。读者在考场上遇到求最大、最小、最多等最值问题时，应优先考虑是否能够用贪心策略求解。若问题满足最优子结构性质，即该问题具备无后效性，那么全局的最优解便可由求子问题的最优解得到。此时就应该选择使用贪心策略。尽管贪心策略是一种高效实用的方法，但不适合于求解所有的最优化问题。无法通过贪心策略求解的最优化问题，将在动态规划一章中介绍。

第 8 章 递归与分治

本章介绍程序设计中的另一个非常重要的思想——递归策略。递归是指函数直接或间接调用自身的一种方法，它通常可把一个复杂的大型问题层层转化为与原问题相似但规模较小的问题来求解。递归策略只需少量的程序就可描述解题过程所需的多次重复计算，因此大大减少了程序的代码量。

8.1 递归策略

构成递归需要具备两个条件：

1. 子问题必须与原始问题相同，且规模更小。
2. 不能无限地调用本身，必须有一个递归出口。

下面通过一个简单的例子帮助大家更好地了解递归策略的思想。

例题 8.1 n 的阶乘（清华大学复试上机题）

题目描述：

输入一个整数 n ，输出 n 的阶乘（每组测试用例可能包含多组数据，请注意处理）。

输入：

一个整数 n ($1 \leq n \leq 20$)。

输出：

n 的阶乘。

样例输入：

3

样例输出：

6

提交网址：

<http://t.cn/AiOocOUY>

【分析】

求 n 的阶乘的问题能否转换为更小的子问题呢？ n 的阶乘其实等于 n 乘上 $n-1$ 的阶乘，于是在求 n 的阶乘时可以转换为求 $n-1$ 的阶乘，求 $n-1$ 的阶乘时也可以转换为求 $n-2$ 的阶乘，以此类推，将问题不断缩小，这就完成了递归的第一个条件。将求 n 的阶乘转换为求 $n-1$ 的阶乘，再转换成求 $n-2$ 的阶乘，直到问题转换成求 0 的阶乘时，按照阶乘的定义就不能再往下分割这个问题了。0 的阶乘是最底层的子问题，该子问题不可继续分解且易于求解。这便是递归的出口，这样就完成了递归的第二个条件。

代码 8.1

```
#include <iostream>
#include <cstdio>

using namespace std;

long long Factorial(int n) {           //递归函数
    if (n == 0) {                     //递归出口
        return 1;
    } else {                          //递归调用
        return n * Factorial(n - 1);
    }
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        printf("%lld\n", Factorial(n));
    }
    return 0;
}
```

例题 8.2 汉诺塔 III

题目描述：

约 19 世纪末，在欧洲的商店中出售一种智力玩具：在一块铜板上有三根杆，最左边的杆自上而下、由小到大顺序串着由 64 个圆盘构成的塔。目的是将最左边杆上的圆盘全部移到右边的杆上，条件是一次只能移动一个圆盘，并且不允许大圆盘放在小圆盘的上面。现在我们改变这个游戏的玩法：不允许直接从最左（右）边移动到最右（左）边（每次移动一定是移到中间杆或从中间杆移出），也不允许大圆盘放到小圆盘的上面。Daisy 已经做过原来的汉诺塔问题和汉诺塔 II 问题，但碰到这个问题时，她想了很久都无法解决。请你帮助她。现在有 N 个圆盘，她至少需要多少次移动才能把这些圆盘从最左边移到最右边？

输入:

包含多组数据, 每次输入一个 N 值 ($1 \leq N \leq 35$)。

输出:

对于每组数据, 输出移动最小的次数。

样例输入:

```
1
3
12
```

样例输出:

```
2
26
531440
```

【分析】

汉诺塔问题历来被认为是理解递归最好的例题, 凡是涉及递归的教科书, 其教学用例十有八九会采用汉诺塔。这里给出汉诺塔问题的一个变种, 旨在通过该问题让读者更好地理解递归策略。

与原始的汉诺塔问题不同, 该变种对圆盘的移动做了更多的限制, 即每次只允许将圆盘移到中间杆上或从中间杆上移出, 即不允许直接将圆盘从第一根杆上移到第三根杆上, 或直接将圆盘从第三根杆上移到第一根杆上。

在这种情况下, 考虑第一根杆上移动 N 个圆盘到第三根杆上。为了将初始时最底下、最大的圆盘移动到第三根杆上, 首先需要将其上的 $N-1$ 个圆盘移动到第三根杆上, 而这恰好等价于从第一根杆上移动 $N-1$ 个圆盘到第三根杆上。这一移动完成后, 第一根杆上仅剩下最大的圆盘, 第二根杆上为空, 第三根杆上按顺序摆放着 $N-1$ 个圆盘。将最大的圆盘移动到此时没有任何圆盘的第二根杆上, 并再次将 $N-1$ 个圆盘从第三根杆上移动到第一根杆上, 此时仍然需要 $N-1$ 个圆盘从第一根杆上移动到第三根杆上所需的移动次数 (第一根杆和第三根杆等价), 这一移动完成后, 将最大的圆盘移到第三根杆上, 最后将 $N-1$ 个圆盘从第一根杆上移动到第三根杆上。若从第一根杆上移动 N 个圆盘到第三根杆上需要 $F[N]$ 次移动, 那么综上所述 $F[N]$ 的组成方式如下: 先移动 $N-1$ 个圆盘到第三根杆上需要 $F[N-1]$ 次移动, 然后将最大的圆盘移动到中间杆上需要 1 次移动, 再将 $N-1$ 个圆盘移动回第一根杆上同样需要 $F[N-1]$ 次移动, 移动最大的盘子到第三根杆上需要 1 次移动, 最后将 $N-1$ 个圆盘移动到第三根杆上需要 $F[N-1]$ 次移动, 于是便有了 $F[N]=3F[N-1]+2$ 。也就是说, 从第一根杆上移动 N 个圆盘到第三根杆上, 需要三次从第一根杆上移动 $N-1$ 个圆盘到第三根杆上, 外加两次对最大圆盘的移动。这样就可以将移动 N 个圆盘的问题转换为问题相同但规模更小的移动 $N-1$ 个圆盘的问题。于是, 满足了递归的第一个条件。

同时, 要确定该问题的递归出口。当 N 为 1 时, 即从第一根杆上移动一个盘子到第三根杆上, 所需的移动次数显而易见为 2。移动一个盘子所需次数是最底层的子问题, 该子问题不可继续分解且易于求解。这便是递归的出口, 于是满足了递归的第二个条件。

代码 8.2

```
#include <iostream>
#include <cstdio>

using namespace std;

long long Function(int n) {                //递归函数
    if (n == 1) {                          //递归出口
        return 2;
    } else {                              //递归调用
        return 3 * Function(n - 1) + 2;
    }
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        printf("%lld\n", Function(n));
    }
    return 0;
}
```

读者可能会对如此精简的代码感到震惊，这正是递归策略的魅力所在。如本例所示，我们并不关注具体的移动方法，而只是将当前问题与规模更小的问题联系起来，并利用这一关系确定递归关系式。在确定递归关系式后，根据该问题的底层子问题来确定递归的出口。此外，不需要做任何额外的工作，递归程序就会按照预先设定的递归方式“自动”地算出答案。

习题 8.1 杨辉三角形（西北工业大学复试上机题）

输入 n 值，使用递归函数，求杨辉三角形中各个位置上的值。

提交网址：

<http://t.cn/AiOKcLRI>

习题 8.2 全排列（北京大学复试上机题）

给定一个由不同小写字母组成的字符串，输出这个字符串的所有全排列。假设对于小写字母有 'a' < 'b' < ... < 'y' < 'z'，而且给定的字符串中的字母已经按照从小到大的顺序排列。

提交网址：

<http://t.cn/AiOKOhXZ>

8.2 分治法

分治法 (Divide and Conquer) 是另一个非常重要的算法。分治法字面上的解释是“分而治之”，就是把一个复杂的问题分成两个或更多个子问题，子问题之间互相独立且与原问题相同或相似。之后再把子问题分成更小的子问题，以此类推，直到最后的子问题可以简单地直接求解，原问题的解即子问题的解的合并。

由于分治法产生的子问题往往与原问题相同且模式更小，这就为使用递归策略求解提供了条件。在这种情况下，通过反复利用分治手段，可以使子问题的规模不断缩小，最终缩小到可以直接求解的情况。在从过程中会导致了递归过程的发生。分治与递归就像一对孪生兄弟，经常同时应用在算法设计中，这也是将分治法和递归策略放在同一章中进行讲解的原因。

分治法的步骤如下：

1. 分：将问题分解为规模更小的子问题。
2. 治：将这些规模更小的子问题逐个击破。
3. 合：将已解决的子问题合并，最终得出“母”问题的解。

例题 8.3 Fibonacci (上海交通大学复试上机题)

题目描述：

The Fibonacci Numbers $\{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots\}$ are defined by the recurrence: $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, n \geq 2$. Write a program to calculate the Fibonacci Numbers.

输入：

Each case contains a number n and you are expected to calculate F_n ($0 \leq n \leq 30$).

输出：

For each case, print a number F_n on a separate line, which means the n th Fibonacci Number.

样例输入：

1

样例输出：

1

提交网址：

<http://t.cn/AiOK3tU5>

【题目大意】

斐波纳契数 $\{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots\}$ 的递归定义是： $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, n \geq 2$ 。编写程序计算斐波纳契数。

【分析】

相信读者对斐波纳契数应不陌生，可以将规模为 N 的问题分解为两个规模分别为 $N-1$ 和

$N-2$ 的子问题，通过不断分解直到问题规模小到 0 或 1 时，便可直接给出答案，之后将子问题的答案进行相加，便得到母问题的答案。由上面的分析可以看出，这个问题完美地符合分治法实现的三个步骤：首先将问题分解为规模更小的子问题；然后，当问题分解到可被直接解决的情况下，将这些子问题逐个击破；最后将已解决的子问题的答案进行相加合并，便可以最终得出原问题的解。虽然分治法并不是求解斐波那契数的最优解法，但希望通过这道题向读者展示分治法的思想与实现步骤。

代码 8.3

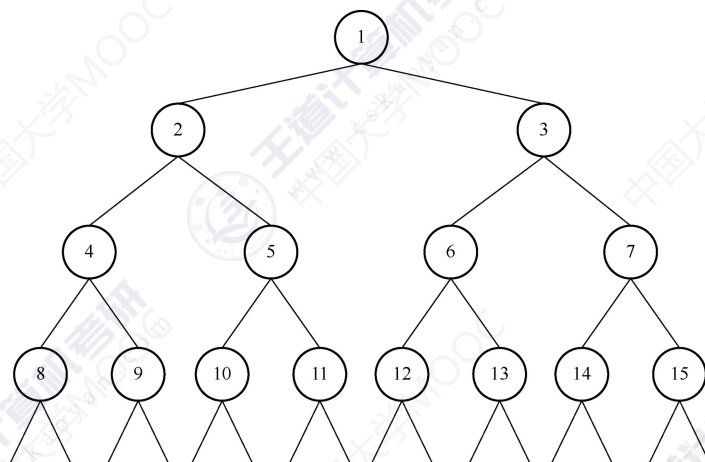
```
#include <iostream>
#include <cstdio>

using namespace std;

int Fibonacci(int n) {
    if (n == 1 || n == 0) {                //递归出口
        return n;
    } else {                               //递归调用
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        printf("%d\n", Fibonacci(n));
    }
    return 0;
}
```

例题 8.4 二叉树（北京大学复试上机题）



题目描述：

如上所示，由正整数 $1, 2, 3, \dots$ 组成了一棵特殊二叉树。我们已知这棵二叉树的最后一个结点是 n 。现在的问题是，结点 m 所在的子树中一共包括多少个结点。比如， $n = 12$ ， $m = 3$ ，那么上图中的结点 13, 14, 15 及后面的结点都是不存在的，结点 m 所在子树中包括的结点有 3, 6, 7, 12，因此结点 m 所在的子树中共有 4 个结点。

输入：

输入数据有多行，每行给出一组测试数据，包括两个整数 m 和 n ($1 \leq m \leq n \leq 1000000000$)。

输出：

对于每组测试数据，输出一行，该行包含一个整数，给出结点 m 所在子树中包括的结点的数目。

样例输入：

3 12

样例输出：

4

提交网址：

<http://t.cn/AiOKe6lO>

【分析】

对于求根结点为 m 的树中有多少结点数目这个问题，可以将问题分解为求它左子树和右子树的结点数目这两个相同的子问题。在得到其左、右子树的结点数目后，加上结点 m 本身便可得到问题的解。

当结点 m 大于 n 时，以 m 为根结点的树为空，那么该树的结点数目必定为 0，这个便是这个问题的可被轻松求解的底层子问题，也是递归出口。

代码 8.4


```

#include <iostream>
#include <cstdio>

using namespace std;

int CountNodes(int m, int n) {
    if (m > n) {                                //递归出口
        return 0;
    } else {                                    //递归调用
        return 1 + CountNodes(m * 2, n) + CountNodes(m * 2 + 1, n);
    }
}

int main() {
    int m, n;
    while (scanf("%d%d", &m, &n) != EOF) {
        printf("%d\n", CountNodes(m, n));
    }
    return 0;
}

```

习题 8.3 2 的幂次方（上海交通大学复试上机题）

【题目大意】每个正数都可以用指数形式表示，如 $137 = 2^7 + 2^3 + 2^0$ 。我们用 $a(b)$ 形式给出 a^b 。于是，137 表示为 $2(7) + 2(3) + 2(0)$ 。由于 $7 = 2^2 + 2 + 2^0$ 和 $3 = 2 + 2^0$ ，因此 137 最终表示为 $2(2(2) + 2 + 2(0)) + 2(2 + 2(0)) + 2(0)$ 。给定正数 n ，你的任务是以指数形式呈现 n ，该形式仅包含数字 0 和 2。

提交网址：

<http://t.cn/AiONL7t2>

小结

本章介绍了关于递归与分治的知识点和问题。递归思想在计算机学科中非常重要，但由于其调用自身的特殊性，导致许多初学者常常不得其要领。不过，只要记住，当问题能够转换成比其原来规模更小且问题相同的子问题时，便要考虑能否用递归的方式求解。希望读者在之后的学习中，不断掌握递归的使用技巧。

第 9 章 搜索

本章介绍程序设计中另一种非常重要的方法——搜索。搜索是一种有目的地枚举问题的解空间中部分或全部情况，进而找到解的方法。然而，与枚举策略相比，搜索通常是有目的的查找，发现解空间的某一子集内不存在解时，它便会放弃对该子集的搜索，而不像枚举那般逐个地检查子集内的解是否为问题的解。

9.1 宽度优先搜索

宽度优先搜索（Breadth First Search, BFS）策略从搜索的起点开始，不断地优先访问当前结点的邻居。也就是说，首先访问起点，然后依次访问起点尚未访问的邻居结点，再按照访问起点邻居结点的先后顺序依次访问它们的邻居，直到找到解或搜遍整个解空间。宽度优先搜索类似于向静止的湖中扔一个石块，波纹以石块为中心依次向外传播。由于宽度优先搜索的这种不断向外扩展的特性，因此常用于搜索最优值的问题。

例题 9.1 Catch That Cow

题目描述：

Farmer John has been informed of the location of a fugitive cow and wants to catch her immediately. He starts at a point N ($0 \leq N \leq 100000$) on a number line and the cow is at a point K ($0 \leq K \leq 100000$) on the same number line. Farmer John has two modes of transportation: walking and teleporting.

* Walking: Farmer John can move from any point X to the points $X - 1$ or $X + 1$ in a single minute.

* Teleporting: Farmer John can move from any point X to the point $2X$ in a single minute.

If the cow, unaware of its pursuit, does not move at all, how long does it take for Farmer John to retrieve it?

输入：

Line 1: Two space-separated integers: N and K .

输出：

Line 1: The least amount of time, in minutes, it takes for Farmer John to catch the fugitive cow.

样例输入:

5 17

样例输出:

4

提示:

The fastest way for Farmer John to reach the fugitive cow is to move along the following path: 5-10-9-18-17, which takes 4 minutes.

【题目大意】

农夫约翰被告知逃亡奶牛所在的位置，并希望能够立即抓住奶牛。他刚开始站在点 N ($0 \leq N \leq 100000$) 上，并且母牛站在同一条线上的点 K ($0 \leq K \leq 100000$) 上。农夫约翰有两种交通方式：步行和传送。

* 行走：农夫约翰可以在 1 分钟内从任何一点 X 移动到点 $X-1$ 或点 $X+1$ 。

* 传送：农夫约翰可以在 1 分钟内从任何一点 X 移动到 $2X$ 点。

如果母牛不知道有人要去追它，根本不动，那么农夫约翰需要多长时间才能找回它？

【分析】

很明显，这是需要寻找一条到达母牛所在位置的最优路径问题，非常适合运用宽度优先搜索进行求解。但在探讨宽度优先搜索之前，必须先指明问题中的状态。为了能够考虑搜索的问题，通常会在搜索问题中指明某种状态，以便使搜索问题转换成为对状态的搜索。在本题中，由于是求从起点 N 到终点 K 的最短耗时，因此可以设定状态为二元组 (n, t) ，其中 n 为农夫约翰当前所在位置的坐标点， t 为从起点 N 走到当前坐标点 n 所耗费的时间。那么在宽度优先搜索中进行查找的各个参数如下：

查找空间：所有可能出现的状态，即所有可能出现的二元组 (n, t) 。

查找目标：即在所有可能出现的状态中搜索这样一个二元组 (n, t) ，其中 n 为终点 K ， t 为达到这个状态所需要的最短时间。

由于任意一个状态 (n, t) 经过 1 秒后可以转入下一个状态，所以说任意一个状态 (n, t) 可以扩展为三种状态 $(n-1, t+1)$ 、 $(n+1, t+1)$ 、 $(2n, t+1)$ 中的合法状态。所谓合法，是指该点的坐标在区间 $[0, 100000]$ 内。于是便可从起始状态 $(N, 0)$ 不断地扩展到它能够到达的下一个状态，即起点的邻居结点。再按邻居结点访问的先后顺序依次地扩展到它们能够到达的下一个状态，以此类推，当扩展到目标状态 (K, t) 时，目标状态中的 t 必然为最短耗时。

此外，在宽度优先搜索中可以记录是否去过某个坐标点，以便下次经过这个点时不必再次搜索，避免重复的搜索过程，进而提高程序的运行效率。

代码 9.1

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <queue>
```

```

using namespace std;

const int MAXN = 100001;

struct Status {
    int n, t;
    Status(int n, int t): n(n), t(t) {}
};

bool visit[MAXN];

int BFS(int n, int k) {
    queue<Status> myQueue;
    myQueue.push(Status(n, 0));           //压入初始状态
    visit[n] = true;                      //起始点已被访问
    while (!myQueue.empty()) {
        Status current = myQueue.front();
        myQueue.pop();
        if (current.n == k) {             //查找成功
            return current.t;
        }
        for (int i = 0; i < 3; ++i) {     //转入不同状态
            Status next(current.n, current.t + 1);
            if (i == 0) {
                next.n += 1;
            } else if (i == 1) {
                next.n -= 1;
            } else {
                next.n *= 2;
            }
            if (next.n < 0 || next.n >= MAXN || visit[next.n]) {
                continue;                 //新状态不合法
            }
            myQueue.push(next);           //压入新的状态
            visit[next.n] = true;         //该点已被访问
        }
    }
}

```

```
int main() {  
    int n, k;  
    scanf("%d%d", &n, &k);  
    memset(visit, false, sizeof(visit));  
    printf("%d\n", BFS(n, k));  
    return 0;  
}
```

例题 9.2 Find The Multiple

题目描述:

Given a positive integer n , write a program to find out a nonzero multiple m of n whose decimal representation contains only the digits 0 and 1. You may assume that n is not greater than 200 and there is a corresponding m containing no more than 100 decimal digits.

输入:

The input file may contain multiple test cases. Each line contains a value of n ($1 \leq n \leq 200$). A line containing a zero terminates the input.

输出:

For each value of n in the input print a line containing the corresponding value of m . The decimal representation of m must not contain more than 100 digits. If there are multiple solutions for a given value of n , any one of them is acceptable.

样例输入:

```
2
6
19
0
```

样例输出:

```
10
100100100100100100
111111111111111111
```

【题目大意】

给定正整数 n , 编写程序找出非零值 n 的倍数 m , 并且 m 的十进制数表示仅包含数字 0 和 1。你可以假设 n 不大于 200, 且有不超过 100 位的相应 m 。

【分析】

本题乍看之下好像和宽度优先搜索无关, 只需逐个测试 n 的倍数是否全由 0 和 1 组成即可, 但这样做相当于枚举, 需要枚举 n 在整个搜索空间 (小于 100 位的整数) 中的所有倍数, 这个计算量非常大, 因为逐个判断会花费大量时间, 因此这种方法的效率非常低。

可以反过来想这个问题: 只由 0 和 1 组成的数相比于整个搜索空间而言是非常少的, 可以去搜索由 0 和 1 组成的数, 然后通过判断这些数能否整除 n 来求解这个问题。

于是, 可将由 0 和 1 组成的数字 num 作为一个搜索状态。能由 num 扩展得到的状态有两个, 一个是 $10 * num$, 另一个是 $10 * num + 1$ 。这两个状态必然都是由 0 和 1 组成的。若将 num 的初始状态设置为 1, 则可通过状态的扩展来覆盖整个查找空间中由 0 和 1 组成的数。

由于从初始状态 1 开始不断扩展, 扩展的数字必定比之前的数字大, 扩展出来的状态不可

能在此前出现过，因此不需要用数组来记录访问过的数字。

代码 9.2

```

#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

void BFS(int n) {
    queue<long long> myQueue;
    myQueue.push(1); //压入初始状态
    while (!myQueue.empty()) {
        long long current = myQueue.front();
        myQueue.pop();
        if (current % n == 0) { //查找成功
            printf("%lld\n", current);
            return ;
        }
        myQueue.push(current * 10);
        myQueue.push(current * 10 + 1);
    }
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        BFS(n);
    }
    return 0;
}

```

学习上面的例题后，宽度优先搜索总结如下。

- ① 状态。需要确定所求解问题中的状态。通过状态的扩展，遍历所有的状态，从中寻找需要的答案。
- ② 状态扩展方式。在宽度优先搜索中，需要尽可能地扩展状态，并对先扩展得到的状态先进行下一次状态扩展。
- ③ 有效状态。对有些状态，并不需要再次扩展它，而是直接舍弃它。因为根据问题的分析

可知，目标状态不可能由这些状态经过若干次扩展得到，所以直接舍弃。

- ④ 队列。为了使得先得出的状态能够先扩展，于是使用队列，将得到的状态依次放入队尾，每次取队头元素进行扩展。
- ⑤ 标记。为了判断哪些状态是有效的，哪些是无效的，往往使用标记。
- ⑥ 有效状态数。问题中的有效状态数与算法的时间复杂度同数量级，所以在进行搜索之前，必须估算其是否在能够接受的范围内。
- ⑦ 最优。宽度优先搜索常被用来求解最优值问题，因为其搜索到的状态总是按照某个关键字递增，这个特性非常适合求解最优值问题。因此，一旦问题中出现最少、最短、最优等关键字，就要考虑是否是宽度优先搜索问题。

习题 9.1 玛雅人的密码（清华大学复试上机题）

玛雅人有一种密码，字符串中出现连续的 2012 四个数字时就能解开密码。给定一个长度为 N 的字符串（ $2 \leq N \leq 13$ ），该字符串中只含有 0, 1, 2 三种数字，问这个字符串要移位几次才能解开密码，每次只能移动相邻的两个数字。例如，02120 经过一次移位，可以得到 20120, 01220, 02210, 02102，其中 20120 符合要求，因此输出为 1。如果无论移位多少次都解不开密码，输出 -1。

提交网址：

<http://t.cn/AiOlUhJj>

9.2 深度优先搜索

在介绍深度优先搜索（Depth First Search, DFS）之前，先回顾一下之前介绍过的宽度优先搜索。在宽度优先搜索过程中，获得一个状态后，立即扩展这个状态，并且保证早得到的状态优先得到扩展。因此，使用队列的先进先出特性来实现先得到的状态先扩展这一特性。

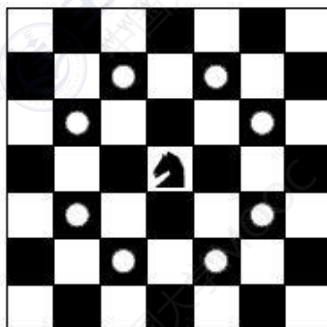
如果在搜索过程中，首先访问起点，之后访问起点的一个邻居，先不访问除该点之外的其他起点的邻居结点，而是访问该点的邻居结点，如此往复，直到找到解，或者当前访问结点已经没有尚未访问过的邻居结点为止，之后回溯到上一个结点并访问它的另一个邻居结点。这样的搜索策略便是深度优先搜索，它类似于人在迷宫中找出口：每遇到一个路口，先往一个既定的方向走到底，直到发现出口或遇到死胡同。发现死胡同后，就回到上一个路口，并选择另外一个方向继续寻找出口。

在深度优先搜索中，对搜索的状态而言，获得一个状态后，同样立即扩展这个状态，但需要保证早得到的状态较后得到扩展。这种先入后出的特点让人想到了栈这种数据结构，不过递归策略也能保证这一特性。考虑到递归的代码比较简洁明了，因此常常使用递归策略来求解深度优先搜索问题。

由于深度优先搜索并没有先入先出的特点，所以搜索到需要的状态时，该状态不再像是在宽度优先搜索中的状态一样，具有某种最优的特性。因此，使用深度优先搜索策略时，常常是为了知道问题是否有解，而一般不使用深度优先搜索求解最优解问题。

例题 9.3 A Knight's Journey

题目描述:



The eight possible moves of a knight

Background

The knight is getting bored of seeing the same black and white squares again and again and has decided to make a journey around the world. Whenever a knight moves, it is two squares in one direction and one square perpendicular to this. The world of a knight is the chessboard he is living on. Our knight lives on a chessboard that has a smaller area than a regular 8×8 board, but it is still rectangular. Can you help this adventurous knight to make travel plans?

Problem

Find a path such that the knight visits every square once. The knight can start and end on any square of the board.

输入:

The input begins with a positive integer n in the first line. The following lines contain n test cases. Each test case consists of a single line with two positive integers p and q , such that $1 \leq p \times q \leq 26$. This represents a $p \times q$ chessboard, where p describes how many different square numbers $1, \dots, p$ exist, q describes how many different square letters exist. These are the first q letters of the Latin alphabet: A,

输出:

The output for every scenario begins with a line containing "Scenario $\#i$:", where i is the number of the scenario starting at 1. Then print a single line containing the lexicographically first path that visits all squares of the chessboard with knight moves followed by an empty line. The path should be given on a single line by concatenating the names of the visited squares. Each square name consists of a capital letter followed by a number.

样例输入:

```
3
1 1
2 3
4 3
```

样例输出：

Scenario #1:

A1

Scenario #2:

impossible

Scenario #3:

A1B3C1A2B4C2A3B1C3A4B2C4

【题目大意】

背景

骑士每天看着相同的黑白方块感到越来越无聊并决定去世界各地旅行。骑士按照“日”字规则行走。骑士的世界就是他生活的棋盘。骑士生活在比普通 8×8 棋盘更小的棋盘上，但棋盘的形状仍然是长方形的。你能帮助这位冒险骑士制订旅行计划吗？

问题

找一条能够让骑士遍历棋盘上所有点的路径。骑士可以在任何一块方块上开始或结束他的旅行。

【分析】

在这个问题中，题面不再是求问题的最优解，而是要判定是否存在一条符合条件的路径，所以可以使用深度优先搜索来达到这个目的。

搜索状态三元组 $(x, y, step)$ ， (x, y) 是当前点的坐标， $step$ 是从起点走到该点经过的步数。需要搜索到的最终目标状态为 $(x, y, step)$ ，其中 $step = p * q$ ，即经过的步数等于棋盘的格数。题目要求在所有可行的路径中输出字母表排序最小的那个解，如果存在一条路径可以遍历所有方格的路径，那么该路径必定会过点 A1。于是只需最开始从点 A1 出发，这样的路径的字母表排序必定是最小的。因此，初始状态为 $(sx, sy, 1)$ ，其中 (sx, sy) 是 A1 所在点的坐标 $(0, 0)$ 。在本题中需要记录棋盘中的某点骑士是否曾经访问过，以便确保骑士的路径上没有重复地经过同一点，也只有这样，判断 $step = p * q$ 才有意义。

代码 9.3

```
#include <iostream>
#include <cstdio>
#include <string>
#include <cstring>
```

```

using namespace std;

const int MAXN = 30;
int p, q;                                     //棋盘参数
bool visit[MAXN][MAXN];                       //标记矩阵
int direction[8][2] = {
    {-1, -2}, {1, -2}, {-2, -1}, {2, -1}, {-2, 1}, {2, 1}, {-1, 2}, {1, 2}
};

bool DFS(int x, int y, int step, string ans) {
    if (step == p * q) {                       //搜索成功
        cout << ans << endl << endl;
        return true;
    } else {
        for (int i = 0; i < 8; ++i) {         //遍历邻居结点
            int nx = x + direction[i][0];      //扩展状态坐标
            int ny = y + direction[i][1];
            char col = ny + 'A';               //该点编号
            char row = nx + '1';
            if (nx < 0 || nx >= p || ny < 0 || ny >= q || visit[nx][ny]){
                continue;
            }
            visit[nx][ny] = true;              //标记该点
            if (DFS(nx, ny, step + 1, ans + col + row)) {
                return true;
            }
            visit[nx][ny] = false;             //取消标记
        }
    }
    return false;
}

int main() {
    int n;
    scanf("%d", &n);
    int caseNumber = 0;
    while (n--) {
        scanf("%d%d", &p, &q);
        memset(visit, false, sizeof(visit));
    }
}

```

```
cout << "Scenario #" << ++caseNumber << ":" << endl;
visit[0][0] = true; //标记 A1 点
if (!DFS(0, 0, 1, "A1")) {
    cout << "impossible" << endl << endl;
}
}
return 0;
}
```


例题 9.4 Square

题目描述:

Given a set of sticks of various lengths, is it possible to join them end-to-end to form a square?

输入:

The first line of input contains N , the number of test cases. Each test case begins with an integer $4 \leq M \leq 20$, the number of sticks. M integers follow; each gives the length of a stick - an integer between 1 and 10000.

输出:

For each case, output a line containing "yes" if it is possible to form a square; otherwise output "no".

样例输入:

```
3
4 1 1 1 1
5 10 20 30 40 50
8 1 7 2 6 4 4 3 5
```

样例输出:

```
yes
no
yes
```

【题目大意】

给出一堆长度各异的木棍，这些木棍能否头尾相连形成一个正方形？

【分析】

题面要求判断能否搜索出一个方案让这些木棍形成正方形。首先，将木棍长度进行累加，得到总长 $length$ ；然后将 $length$ 除以 4 得到正方形的边长 $side$ ；之后，将木棍拼凑成长度为 $side$ 的边，拼凑完一条边后，再拼凑下一条，直到拼凑出 4 条长度为 $side$ 的边，从而构成一个正方形。

搜索状态三元组 $(sum, number, position)$ ，其中 sum 是当前拼凑的木棍长度， $number$ 是已拼凑成边长的数量， $position$ 是当前木棍的编号。需要搜索到的目标状态为 $number=3$ 。为什么不是 4 呢？因为如果总长 $length$ 能够整除 4，且已经拼凑出 3 条边，那么剩下的木棍必定可以构成最后的一条边。由于从第一根木棍开始拼凑，故搜索初始状态为 $(0, 0, 0)$ 。

在搜索过程中，可以通过放弃对某些不可能产生结果的子集的搜索，达到提高效率的目的。这样的技术称为剪枝。本题中可以剪枝的途径如下：

- ① 如果总长 $length$ 不能整除 4，那么必定无法构成正方形。
- ② 如果某根木棍的长度大于边长 $side$ ，那么必定无法构成正方形。

③ 拼凑中发现某长度的木棍无法构成当前边时，再遇等长木棍时可跳过（需排序）。

代码 9.4

```
#include <iostream>
#include <algorithm>
#include <cstdio>
#include <cstring>

using namespace std;

const int MAXN = 25;
int side; //边长
int m; //树枝数目
int sticks[MAXN];
bool visit[MAXN];

bool DFS(int sum, int number, int position) {
    if (number == 3) {
        return true;
    }
    int sample = 0; //剪枝(3)
    for (int i = position; i < m; ++i) {
        if (visit[i] || sum + sticks[i] > side || sticks[i] == sample){
            continue;
        }
        visit[i] = true;
        if (sum + sticks[i] == side) { //凑成一条边
            if (DFS(0, number + 1, 0)) {
                return true;
            } else {
                sample = sticks[i]; //记录失败棍子长度
            }
        } else {
            if (DFS(sum + sticks[i], number, i + 1)) {
                return true;
            } else {
                sample = sticks[i]; //记录失败棍子长度
            }
        }
    }
}
```

```
        visit[i] = false;
    }
    return false;
}

bool Compare(int x, int y) {
    return x > y;
}

int main() {
    int n;
    scanf("%d", &n);
    while (n--) {
        int length = 0; //总长
        scanf("%d", &m); //木棍数目
        for (int i = 0; i < m; ++i) {
            scanf("%d", &sticks[i]);
            length += sticks[i];
        }
        memset(visit, false, sizeof(visit));
        if (length % 4 != 0) { //剪枝(1)
            printf("no\n");
            continue;
        }
        side = length / 4; //边长
        sort(sticks, sticks + m, Compare); //从大到小排序
        if (sticks[0] > side) { //剪枝(2)
            printf("no\n");
            continue;
        }
        if (DFS(0, 0, 0)) {
            printf("yes\n");
        } else {
            printf("no\n");
        }
    }
    return 0;
}
```

学习上面的例题后，深度优先搜索总结如下。

深度优先搜索的查找空间和查找目的与宽度优先搜索是一致的，与宽度优先搜索不同的是其查找方式。深度优先搜索对状态的查找采用的是立即扩展新得到的状态，而不是将当前状态全部扩展完后再扩展新的状态。因此，常使用递归函数来实现这一功能。由于采用了这样扩展方法，故搜索得到的解不再拥有最优解的特性，所以常用它来判断一个问题的解是否存在。

在结束对深度优先搜索的讨论之前，还需要特别强调的是，使用递归函数时一定要注意递归的层数。一个程序可以使用的栈空间是有限的，当递归层次过深或每层递归所需的栈空间太大时，会造成栈溢出，使评判系统返回程序运行时异常终止的结果。一旦递归程序出现了这种错误，就要考虑是否是由递归层次太深造成了“爆栈”。这是使用递归程序时一个重要的注意点。具体可以使用的栈大小，会因评判系统的不同而有所差异，需要读者自行测试后确定。至此，已经学习了两种既有联系又有区别的搜索方式，在考研机试中，究竟选择哪种搜索方式进行搜索，需要考生联系实际考题做出选择。

习题 9.2 神奇的口袋（北京大学复试上机题）

有一个神奇的口袋，其容积是 40，用这个口袋可以变出一些物品，这些物品的总体积必须是 40。John 现在有 n 个想要得到的物品，每个物品的体积分别是 $a_1, a_2, \dots, a_n$ 。John 可以从这些物品中选择一些，如果选出的物体的总体积是 40，那么利用这个神奇的口袋 John 就能得到这些物品。现在的问题是，John 有多少种选择物品的不同方式。

提交网址：

<http://t.cn/AiOuOQUz>

习题 9.3 八皇后（北京大学复试上机题）

会下国际象棋的人都很清楚：皇后可以在横线、竖线、斜线上不限步数地吃掉其他棋子。如何将 8 个皇后放在棋盘上（有 8×8 个方格），使它们谁也不能被吃掉？这就是著名的八皇后问题。对于某个满足要求的 8 皇后的摆放方法，定义一个皇后串 a 与之对应，即 $a = b_1 b_2 \dots b_8$ ，其中 b_i 为相应摆放中第 i 行皇后所处的列数。已经知道 8 皇后问题一共有 92 组解（即 92 个不同的皇后串）。给出一个数 b ，要求输出第 b 个串。串的比较是这样的：皇后串 x 置于皇后串 y 之前，当且仅当将 x 视为整数时比 y 小。

提交网址：

<http://t.cn/AiOuOazs>

小结

本章首先介绍了搜索的有关知识点，然后介绍了如何把在查找空间中进行的搜索转换为对状态的搜索，最后根据对状态搜索的不同方式，介绍了各具特点的宽度优先搜索和深度优先搜索。希望读者对这两种搜索方法多加练习，掌握它们后，你的机试水平会明显地上一个台阶。

第 10 章 数据结构二

本章介绍机试中考查的一些非线性数据结构，包括二叉树、二叉排序树、优先队列和散列表等较为高级的数据结构。

10.1 二叉树

之前介绍的向量、队列和栈都属于线性序列，在这些结构中，元素之间存在着线性次序。树则不然，树中的元素并没有强烈的线性关系，不过可以通过对其进行一定的约束，例如进行遍历，在树的元素之间确定某种线性次序。

树的结构有诸多变体，它们在各种应用中发挥着重要作用。作为树的特例的二叉树（Binary Tree），虽然看似简单，但不失一般性。因为已经被证明，任何有根、有序的多叉树，都可通过等价变换转换为二叉树。也就是说，二叉树是大部分纷繁复杂的树结构的简化版，学好了二叉树，再学习其他树结构时就能做到游刃有余。

下面先给出二叉树的递归定义：二叉树要么为空，要么由根结点（Root）、左子树（Left Subtree）和右子树（Right Subtree）构成，而左右两个子树又分别是一棵二叉树。于是，如果要建立一棵二叉树，那么最简单的方式便是按照二叉树的定义进行递归建树，每个结点的定义如下：

```
struct TreeNode {  
    ElementType data;           //数据  
    TreeNode *leftChild;        //左子树  
    TreeNode *rightChild;       //右子树  
};
```

对于二叉树来说，机试中常考的是其各种遍历方法，根据遍历每个结点的左子树 L、结点本身 N、右子树 R 的顺序不同，可将二叉树的遍历方法分为前序遍历（NLR）、中序遍历（LNR）和后序遍历（LRN），其中的序是指根结点在何时被访问。

1. 前序遍历（PreOrder）

- （1）如果结点为空，则直接返回。
- （2）访问根结点。
- （3）前序遍历左子树。

(4) 前序遍历右子树。

前序遍历代码如下：

```
void PreOrder(TreeNode* root) {  
    if (root == NULL) {  
        return;  
    }  
    Visit(root->data);  
    PreOrder(root->leftChild);  
    PreOrder(root->rightChild);  
    return;  
}
```

2. 中序遍历 (InOrder)

(1) 如果结点为空，则直接返回。

(2) 中序遍历左子树。

(3) 访问根结点。

(4) 中序遍历右子树。

中序遍历代码如下：

```
void InOrder(TreeNode* root) {  
    if (root == NULL) {  
        return;  
    }  
    InOrder(root->leftChild);  
    Visit(root->data);  
    InOrder(root->rightChild);  
    return;  
}
```

3. 后序遍历 (PostOrder)

(1) 如果结点为空，则直接返回。

(2) 后序遍历左子树。

(3) 后序遍历右子树。

(4) 访问根结点。

后序遍历代码如下：

```
void PostOrder(TreeNode* root) {  
    if (root == NULL) {  
        return;  
    }  
    PostOrder(root->leftChild);
```



```

    PostOrder(root->rightChild);
    Visit(root->data);
    return;
}

```

从上面的示例代码可以看出，在前序、中序和后序这三种遍历方法中，左、右子树的遍历顺序是固定的，区别仅在于访问根结点的顺序不同。

除前序、中序和后序这三种遍历方式外，如果二叉树按照其高度一层层地遍历所有的结点，那么这种遍历方法是层次遍历。层次遍历需要借助一个队列来实现。先将二叉树的根结点入队，在队伍非空的情况下访问队首结点，若它有左子树，则将左子树根结点入队；若它有右子树，则将右子树根结点入队。如此往复，直到队列为空为止。

```

void LevelOrder(TreeNode* root) {
    queue<TreeNode*> myQueue;
    if (root != NULL) {
        myQueue.push(root);
    }
    while (!myQueue.empty()) {
        TreeNode* current = myQueue.front();
        myQueue.pop();
        visit(current->data);
        if (current->leftChild != NULL) {
            myQueue.push(current->leftChild);
        }
        if (current->rightChild != NULL) {
            myQueue.push(current->rightChild);
        }
    }
    return;
}

```

例题 10.1 二叉树遍历（清华大学复试上机题）

题目描述：

编一个程序，读入用户输入的一串先序遍历字符串，根据此字符串建立一棵二叉树（以指针方式存储）。例如，先序遍历字符串 `ABC##DE#G##F####`，其中“#”表示空格，空格字符代表空树。建立这棵二叉树后，再对二叉树进行中序遍历，输出遍历结果。

输入：

输入包括一行字符串，长度不超过 100。

输出：

可能有多组测试数据，对于每组数据，输出将输入字符串建立二叉树后中序遍历的序列，每个字符后面都有一个空格。每个输出结果占一行。

样例输入：

abc##de#g##f###

样例输出：

c b e g d f a

提交网址：

<http://t.cn/AiKuUTlX>

【分析】

本题主要考查的知识点是二叉树的建立和二叉树的中序遍历。因此，本题只需按照题目的要求，递归地建立一棵二叉树，等这棵二叉树建立好后，再递归地进行中序遍历，问题便可以得到解答。

代码 10.1

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

struct TreeNode {
    char data;
    TreeNode* leftChild;
    TreeNode* rightChild;
    TreeNode(char c): data(c), leftChild(NULL), rightChild(NULL) {}
};

TreeNode* Build(int& position, string str) {
    char c = str[position++];           //当前字符
    if (c == '#') {                     //返回空树
        return NULL;
    }
    TreeNode* root = new TreeNode(c);  //创建新结点
    root -> leftChild = Build(position, str); //创建左子树
    root -> rightChild = Build(position, str); //创建右子树
    return root;
}
```

```

}

void InOrder(TreeNode* root) {                //中序遍历
    if (root == NULL) {
        return;
    }
    InOrder(root->leftChild);
    printf("%c ", root->data);
    InOrder(root->rightChild);
    return;
}

int main() {
    string str;
    while (cin >> str) {
        int position = 0;                    //标记字符串处理位置
        TreeNode* root = Build(position, str);
        InOrder(root);
        printf("\n");
    }
    return 0;
}

```

需要注意，建树 *Build* 函数中 *position* 参数的类型是整型引用，而非整型变量。

例题 10.2 二叉树遍历（华中科技大学复试上机题）

题目描述：

二叉树的前序、中序、后序遍历的定义。前序遍历：对任意一棵子树，首先访问根，然后遍历其左子树，最后遍历其右子树；中序遍历：对任意一棵子树，首先遍历其左子树，然后访问根，最后遍历其右子树；后序遍历：对任意一棵子树，首先遍历其左子树，然后遍历其右子树，最后访问根。给定一棵二叉树的前序遍历和中序遍历，求其后序遍历（提示：给定前序遍历与中序遍历能够唯一地确定后序遍历）。

输入：

两个字符串，其长度 n 均小于等于 26。
第一行为前序遍历，第二行为中序遍历。
二叉树中的结点名称以大写字母 A, B, C, ... 表示，最多 26 个结点。

输出：

输入样例可能有多组，对于每组测试样例，

输出一行，为后序遍历的字符串。

样例输入：

```
ABC
BAC
FDXEAG
XDEFAG
```

样例输出：

```
BCA
XEDGAF
```

提交网址：

<http://t.cn/AiKgDfLU>

【分析】

本题考查的是给定前序遍历与中序遍历可以唯一地确定一棵二叉树这个知识点。在前序遍历中，第一个结点必定是二叉树的根结点；而在中序遍历中，该根结点必定可以将中序遍历的序列拆分为两个子序列，前一个子序列就是根结点左子树的中序遍历序列，而后一个子序列就是根结点右子树的中序遍历序列。如此递归地进行下去，便可以唯一地确定该二叉树。等二叉树建立好后，再进行后序遍历，问题就可以得到解答。

值得注意的是，如果题目给定的是后序遍历与中序遍历，那么也可以唯一地确定一棵二叉树，后序遍历的最后一个结点就如同前序遍历的第一个结点，必定是二叉树的根结点。该根结点也可以将中序遍历序列拆分为两个子序列。因此，也可以采用类似的方法递归地建立唯一的一棵二叉树。

但是，如果题目给定的是先序遍历和后序遍历，那就无法唯一地确定一棵二叉树；但是，如果题目中再附加说明这是一棵满二叉树，即每个结点都只有零个或两个子结点的二叉树，那么只给定先序遍历和后序遍历也可以唯一地确定一棵二叉树。希望读者通过学习本题，能够了解各种能够唯一确立二叉树的情况。

代码 10.2

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

struct TreeNode {
    char data;
    TreeNode* leftChild;
    TreeNode* rightChild;
```

```

TreeNode(char c): data(c), leftChild(NULL), rightChild(NULL) {}
};

TreeNode* Build(string str1, string str2) {
    if (str1.size() == 0) { //返回空树
        return NULL;
    }
    char c = str1[0]; //当前字符
    TreeNode* root = new TreeNode(c); //创建新节点
    int position = str2.find(c); //寻找切分点
    root -> leftChild = Build(str1.substr(1, position), str2.substr(0, position));
    root -> rightChild = Build(str1.substr(position + 1), str2.substr(position + 1));
    return root;
}

void PostOrder(TreeNode* root) { //后序遍历
    if (root == NULL) {
        return;
    }
    PostOrder(root->leftChild);
    PostOrder(root->rightChild);
    printf("%c", root->data);
    return;
}

int main() {
    string str1, str2;
    while (cin >> str1 >> str2) {
        TreeNode* root = Build(str1, str2);
        PostOrder(root);
        printf("\n");
    }
    return 0;
}

```

10.2 二叉排序树

二叉排序树也称二叉搜索树 (Binary Search Tree)，是一种特殊的二叉树。一棵非空的二叉

排序树具有如下的特征：

1. 若左子树非空，则左子树上所有结点关键字的值均小于根结点关键字的值。
2. 若右子树非空，则右子树上所有结点关键字的值均大于根结点关键字的值。
3. 左右子树本身也是一棵二叉排序树。

如果各个数字插入的顺序不同，那么得到的二叉排序树的形态很可能不同，所以不同的插入顺序对二叉排序树的形态有重要影响。但是，根据上述二叉排序树的特点可知，左子树结点值 $<$ 根结点值 $<$ 右子树结点值；因此，如果对二叉排序树进行中序遍历，那么其遍历结果必然是一个升序序列，这也是二叉排序树名称的由来。通过建立一棵二叉排序树，就能对原本无序的序列进行排序，并实现序列的动态维护。

例题 10.3 二叉排序树（华中科技大学复试上机题）

题目描述：

二叉排序树也称二叉查找树。它可以是一棵空树，也可以是一棵具有如下特性的非空二叉树：

1. 若左子树非空，则左子树上所有结点的关键字值均不大于根结点的关键字值。
2. 若右子树非空，则右子树上所有结点的关键字值均不小于根结点的关键字值。
3. 左、右子树本身也是一棵二叉排序树。

现在给你 N 个关键字值各不相同的结点，要求你按顺序将它们插入一个初始为空树的二叉排序树，每次插入成功后，求相应父结点的关键字值，若没有父结点，则输出-1。

输入：

输入包含多组测试数据，每组测试数据两行。

第一行，一个数字 N ($N \leq 100$)，表示待插入的结点数。

第二行， N 个互不相同的正整数，表示要顺序插入结点的关键字值，这些值不超过 10^8 。

输出：

输出共 N 行，每次插入结点后，该结点对应的父结点的关键字值。

样例输入：

```
5
2 5 1 3 4
```

样例输出：

```
-1
2
2
5
3
```


提交网址:

<http://t.cn/Ai9PAkkv>

【分析】

本题考查的是二叉排序树的建树，但本题和之前两题中二叉树建树的过程有所不同，之前两题是给定某种遍历序列，通过该遍历序列来建树。而本题是给定插入元素的顺序，因此需要通过依次将元素插入二叉排序树来完成建树。不过本题在建树的基础上，还需要输出父结点的值，若采取递归建树的方式，可不必等树完全建好后再输出每个结点的父结点，而可以在插入时用一变量记录父结点的值，在建树的同时输出父结点的值。

代码 10.3

```
#include <iostream>
#include <cstdio>

using namespace std;

struct TreeNode {
    int data;
    TreeNode* leftChild;
    TreeNode* rightChild;
    TreeNode(int x): data(x), leftChild(NULL), rightChild(NULL) {}
};

TreeNode* Insert(TreeNode* root, int x, int father) {
    if (root == NULL) { //创建新结点
        root = new TreeNode(x);
        printf("%d\n", father); //输出父结点
    } else if (x < root->data) { //插入左子树
        root->leftChild = Insert(root->leftChild, x, root->data);
    } else { //插入右子树
        root->rightChild = Insert(root->rightChild, x, root->data);
    }
    return root;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        TreeNode* root = NULL; //建立空树
```

```

        for (int i = 0; i < n; ++i) {                                //逐个插入
            int x;
            scanf("%d", &x);
            root = Insert(root, x, -1);
        }
    }
    return 0;
}

```

例题 10.4 二叉排序树（华中科技大学复试上机题）

题目描述：

输入一系列整数，建立二叉排序树，并进行前序、中序、后序遍历。

输入：

输入第一行包括一个整数 n ($1 \leq n \leq 100$)。

接下来的一行包括 n 个整数。

输出：

可能有多组测试数据，对于每组数据，根据题目所给数据建立一棵二叉排序树，并对二叉排序树进行前序、中序和后序遍历。每种遍历结果输出一行。每行最后一个数据之后有一个空格。输入中可能有重复元素，但输出的二叉树遍历序列中重复元素不用输出。

样例输入:

```
5
1 6 5 9 8
```

样例输出:

```
1 6 5 9 8
1 5 6 8 9
5 8 9 6 1
```

提交网址:

```
http://t.cn/AiKDOL5V
```

【分析】

本题考查的仍是二叉排序树的建树，并在此基础上考查其三种遍历方法。因此，可以像上题那样，通过依次插入元素的方式，建立一棵二叉排序树。当这棵二叉排序树建成后，再按照前序、中序和后序对其进行遍历，问题便可获得解答。

代码 10.4

```
#include <iostream>
#include <cstdio>

using namespace std;

struct TreeNode {
    int data;
    TreeNode* leftChild;
    TreeNode* rightChild;
    TreeNode(int x): data(x), leftChild(NULL), rightChild(NULL) {}
};

TreeNode* Insert(TreeNode* root, int x) {
    if (root == NULL) { //创建新结点
        root = new TreeNode(x);
    } else if (x < root->data) { //插入左子树
        root->leftChild = Insert(root->leftChild, x);
    } else if (root->data < x) { //插入右子树
        root->rightChild = Insert(root->rightChild, x);
    }
}
```

```

    return root;
}

void PreOrder(TreeNode* root) { //前序遍历
    if (root == NULL) {
        return;
    }
    printf("%d ", root->data);
    PreOrder(root->leftChild);
    PreOrder(root->rightChild);
    return;
}

void InOrder(TreeNode* root) { //中序遍历
    if (root == NULL) {
        return;
    }
    InOrder(root->leftChild);
    printf("%d ", root->data);
    InOrder(root->rightChild);
    return;
}

void PostOrder(TreeNode* root) { //后序遍历
    if (root == NULL) {
        return;
    }
    PostOrder(root->leftChild);
    PostOrder(root->rightChild);
    printf("%d ", root->data);
    return;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        TreeNode* root = NULL; //建立空树
        for (int i = 0; i < n; ++i) { //逐个插入
            int x;

```

```
scanf("%d", &x);
root = Insert(root, x);
}
PreOrder(root);
printf("\n");
InOrder(root);
printf("\n");
PostOrder(root);
printf("\n");
}
return 0;
}
```

习题 10.1 二叉搜索树（浙江大学复试上机题）

判断两序列是否为同一二叉搜索树序列。

提交网址：

<http://t.cn/Ai9PUJtK>

10.3 优先队列

第 5 章介绍过的队列结构可以用来解决很多日常的问题。例如，采用队列可以模拟在银行排队等待接受服务的客户，队列可以保证先到的客户优先接受服务，即实现“先进先出”规则。

然而，在很多场景下，队列这种“先进先出”的简单规则并不能很好地满足问题的需求。例如，去医院就诊时，假设新来的患者病情特别严重，随时都有生命危险，那么此时就应安排该患者优先就诊，而不是等待前面的患者都就诊完后再接受治疗，这样只会让该患者承受更多的痛苦，甚至耽误治疗的最佳时机。因此，医院应该灵活变通，优先治疗病情严重的患者。

从数据结构的角度来看，不论是病情的严重程度，还是到医院时间的早晚，这些信息都可以进行量化和比较，因此可以将它们的大小视为优先级（Priority）。而这种能够将数据按照事先规定的优先级次序进行动态组织的数据结构称为优先队列（Priority Queue）。在优先队列中，每个元素都被赋予了优先级，访问元素时，只能访问当前队列中优先级最高的元素，即最高级先出（First-In Greatest-Out）规律。

1. STL-priority_queue

在正式介绍优先队列的应用之前，先介绍标准库中的优先队列模板。优先队列底层是用二叉堆实现的。但读者无须关心其内部的实现细节，只需学会如何使用即可。

（1）priority_queue 的定义

为了使用 `priority_queue` 标准模板，应在代码中添加头文件，添加格式为 `#include`

<queue>。值得注意的是，优先队列仍然在队列头文件内。定义一个优先队列 `priority_queue` 的写法是 `priority_queue<typename> name`，其中 `typename` 是优先队列元素的类型，它可以为任意的数据类型，`name` 为定义优先队列的名字。

(2) `priority_queue` 的状态

`priority_queue` 中通常作为判断的状态有两个：一个是返回当前优先队列是否为空的 `empty()`，另一个是返回当前优先队列元素个数的 `size()`。

(3) `priority_queue` 元素的添加或删除

定义一个优先队列后，要向其添加新元素，或删除已有的元素，就要使用 `push()` 和 `pop()` 来实现元素的入队和出队操作。

(4) `priority_queue` 元素的访问

由于优先队列对访问元素的限制，优先队列只能通过 `top()` 访问当前队列中优先级最高的元素。

示例代码

```
#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

priority_queue<int> myPriorityQueue;

int main() {
    printf("the size of myPriorityQueue: %d\n", myPriorityQueue.size());
    myPriorityQueue.push(20);
    myPriorityQueue.push(100);
    myPriorityQueue.push(30);
    myPriorityQueue.push(50);
    printf("the top of myPriorityQueue: %d\n", myPriorityQueue.top());
    printf("the size of myPriorityQueue: %d\n", myPriorityQueue.size());
    int sum = 0;
    while (!myPriorityQueue.empty()) {
        printf("%d ", myPriorityQueue.top());
        sum += myPriorityQueue.top();
        myPriorityQueue.pop();
    }
    printf("sum: %d\n", sum);
    return 0;
}
```



```
}
```

示例结果

```
the size of myPriorityQueue: 0
the top of myPriorityQueue: 100
the size of myPriorityQueue: 4
100 50 30 20
sum: 200
```

2. 优先队列的应用

(1) 顺序问题

优先队列的应用场景多种多样，其中最经典的应用就是求解顺序问题。这类问题往往需要根据元素在序列中的大小顺序来进行求解。例如，输出动态维护序列中的最大值。由于按照优先级依次出队，优先队列能够很好地处理这类问题。

例题 10.5 复数集合（北京邮电大学复试上机题）

题目描述：

有一个复数集合($x+iy$)，作用在该集合上的操作有两种：

1. **Pop**，表示读出集合中复数模值最大的那个复数。若集合为空，则输出 *empty*；若集合不为空，则输出最大的那个复数并从集合中删除那个复数，再输出集合的大小 *SIZE*。
2. **Insert $a+ib$** 指令 (a, b 表示实部和虚部)，将 $a+ib$ 加入集合，输出集合的大小 *SIZE*；最初要读入一个 *int n*，表示接下来 *n* 行中的每一行都是一条命令。

输入：

输入有多组数据。

每组输入一个 n ($1 \leq n \leq 1000$)，然后再输入 n 条指令。

输出：

根据指令输出结果。

模相等的，输出 b 较小的复数。

a 和 b 都是非负数。

样例输入：

```
3
Pop
Insert 1+i2
Pop
```

样例输出：

```
empty
SIZE = 1
1+i2
SIZE = 0
```

提交网址:

<http://t.cn/Ai98yYlt>

【分析】

本题需要始终输出当前队列中模最大的那个复数，是一道考查优先队列的典型题目。

代码 10.5

```
#include <iostream>
#include <cstdio>
#include <queue>
#include <string>

using namespace std;

struct Complex {
    int real; //实部
    int imag; //虚部
    Complex(int a, int b): real(a), imag(b) {}
    bool operator< (Complex c) const { //重载小于号
        return real * real + imag * imag < c.real * c.real + c.imag * c.imag;
    }
};

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        priority_queue<Complex> myPriorityQueue;
        while (n-->) {
            string str;
            cin >> str;
            if (str == "Pop") {
                if (myPriorityQueue.empty()) {
                    printf("empty\n");
                } else {
```

```

        Complex current = myPriorityQueue.top();
        myPriorityQueue.pop();
        printf("%d+i%d\n", current.real, current.imag);
        printf("SIZE = %d\n", myPriorityQueue.size());
    }
} else {
    int a, b;
    scanf("%d+i%d", &a, &b);
    myPriorityQueue.push(Complex(a, b));
    printf("SIZE = %d\n", myPriorityQueue.size());
}
}
}
return 0;
}

```

本题的优先队列中的元素并不是整型这样的内部类型，而是需要人为定义的复数类型 *Complex*。相信读者对用结构体定义一个新的数据类型不再陌生，但程序无法比较人为定义的结构体大小，因此需要重载小于符号，即重新定义复数这个结构体的比较关系。题目中要求比较关系由复数的模决定，但代码中并没有使用复数的模来进行比较的，而是利用模的平方来进行比较的。由于模平方的大小关系与模本身的大小关系相同，因此可用较为简单的模平方进行大小比较。

(2) 哈夫曼树

在机试中最常考查优先队列的应用便是哈夫曼树（Huffman Tree）。在一棵树中，从任意一个结点到达另一个结点的通路被称为路径，该路径上所需经过的边的个数被称为该路径的长度。如果树中结点带有表示某种意义的权值，那么从根结点到达该结点的路径长度再乘以该结点的权值就被称为该结点的带权路径长度。树中所有叶子结点的带权路径长度之和为该树的带权路径长度和。给定 n 个带有权值的结点，以它们为叶子结点构造一棵带权路径长度和最小的二叉树，该二叉树即为哈夫曼树，同时也称最优树。

由于 n 个带有权值的结点构成的哈夫曼树可能不唯一，所以关于哈夫曼树的机试题往往考察的是求解最小带权路径长度和。下面回顾一下哈夫曼树的求法。

- ① 将所有结点放入集合 K 。
- ② 若集合 K 中的剩余结点数大于 1，则取出其中权值最小的两个结点，将它们两个构造成某个新结点的左右子结点，设这个新结点的权值为其两个子结点的权值和，并将该新结点放入集合 K 。
- ③ 若集合 K 中仅剩余一个结点，则该结点即为构造出的哈夫曼树数的根结点。构造过程中，所有中间结点的权值和，即为该哈夫曼树的带权路径和。

使用优先队列，可以高效地求出集合 K 中权值最小的两个元素，不过此时需要的不是优先值最大的元素，而是优先值最小的元素。

例题 10.6 哈夫曼树（北京邮电大学复试上机题）

题目描述：

哈夫曼树，第一行输入一个数 n ，表示叶结点的个数。需要用这些叶结点生成哈夫曼树，根据哈夫曼树的概念，这些结点有权值，即 `weight`，题目需要输出所有结点的值与权值的乘积之和。

输入：

输入有多组数据。

每组第一行输入一个数 n ，接着输入 n 个叶结点（叶结点的权值不超过 100， $2 \leq n \leq 1000$ ）。

输出：

输出权值。

样例输入：

```
5
1 2 2 5 9
```

样例输出：

```
37
```

提交网址：

<http://t.cn/AiCuQMki>

【分析】

本题是考查哈夫曼树的经典题目，只需依照步骤求解即可，不过优先队列默认采用的是大顶堆，即优先级高的先输出。如果要采用小顶堆，即优先级低的先输出，那么就需要按照 `priority_queue<typename, vector<typename>, greater<typename>> name` 的方式重新定义优先队列。

代码 10.6

```
#include <iostream>
#include <cstdio>
#include <queue>

using namespace std;

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
```

```

priority_queue<int, vector<int>, greater<int> > myPriorityQueue;
while (n-->) {
    int x;
    scanf("%d", &x);
    myPriorityQueue.push(x);
}
int answer = 0;
while (1 < myPriorityQueue.size()) {
    int a = myPriorityQueue.top();
    myPriorityQueue.pop();
    int b = myPriorityQueue.top();
    myPriorityQueue.pop();
    answer += a + b;
    myPriorityQueue.push(a + b);
}
printf("%d\n", answer);
}
return 0;
}

```

习题 10.2 查找第 K 小的数（北京邮电大学复试上机题）

查找一个数组中第 K 小的数，注意同样大小算一样大。例如，在 2, 1, 3, 4, 5, 2 中，第三小的数是 3。

提交网址：

<http://t.cn/AiCu5hcK>

习题 10.3 搬水果（吉林大学复试上机题）

在一个果园里，小明已经将所有的水果打了下来，并按水果的不同种类分成了若干堆，小明决定把所有的水果合成一堆。每一次合并，小明可以把两堆水果合并到一起，消耗的体力等于两堆水果的重量之和。当然，经过 $n-1$ 次合并之后，就变成一堆了。小明在合并水果时总共消耗的体力等于每次合并所耗体力之和。假定每个水果的重量都为 1，并且已知水果的种类数和每种水果的数目，你的任务是设计出合并的次序方案，使小明耗费的体力最少，并输出这个最小的体力耗费值。例如有 3 种水果，数目依次为 1, 2, 9。可以先将 1, 2 堆合并，新堆数目为 3，耗费体力为 3。然后将新堆与原先的第三堆合并得到更新的堆，耗费体力为 12。所以小明总共耗费的体力为 $3 + 12 = 15$ ，可以证明 15 是最小的体力耗费值。

提交网址：

<http://t.cn/AiCu5nsQ>

10.4 散列表

之前介绍的数据结构中，不论是向量这样的线性结构，还是二叉树这样的树形结构，结构中元素存放的位置与元素的关键字之间并不存在确定的关系，因此，在进行查找时需要进行关键字的比较。然而，这类建立在比较基础上的查找，其查找效率取决于比较次数，最优也只能做到 $O(\log n)$ 。

散列表是一种根据关键字（key）直接进行访问的数据结构，通过建立关键字和存储位置的直接映射关系（map），便可利用关键码直接访问元素，以加快查找的速度。由于散列表摒弃了关键码有序，因此在理想情况下可在期望的常数时间内实现所有接口操作。也就是说，就平均时间复杂度的意义而言，这些操作的复杂度都是 $O(1)$ 。

本节的目的是讲解映射这种思想在机试中的应用。

1. STL-map

在正式介绍散列表的应用之前，先介绍标准库中提供的映射模板 `map`，`map` 是一个将关键字（key）和映射值（value）形成一对映射后进行绑定存储的容器。`map` 的底层是用红黑树实现的，内部仍然是有序的，其查找效率仍然为 $O(\log n)$ 。标准库中还有一个无序映射 `unordered_map`，其底层是用散列表实现的，其期望的查找效率为 $O(1)$ 。

由于 `map` 和 `unordered_map` 的操作几乎一样，因此本节只介绍 `map` 的使用方法。在数据量不大时，`map` 的性能能够满足绝大多数题目的要求，题目中对性能的要求特别高时，只需将 `map` 改成 `unordered_map` 即可。

(1) `map` 的定义

为了使用 `map` 标准模板，应在代码中添加头文件，添加格式为 `#include <map>`。定义一个映射 `map` 的写法是 `map<typename T1, typename T2> name`，其中 `typename T1` 是映射关键字的类型，`typename T2` 是映射值的类型，`name` 是映射的名字。

(2) `map` 的状态

`map` 中通常作为判断的状态有两个：一个是返回当前映射是否为空的 `empty()`，另一个是返回当前映射元素个数的 `size()`。

(3) `map` 元素的添加或删除

定义一个映射后，若要向其添加新元素，或删除已有元素，就要使用 `insert()` 和 `erase()` 来实现特定元素的添加与删除。

(4) `map` 元素的访问

- ① 由于 `map` 内部重载了 `[]` 运算符，因此可以通过 `[key]` 的方式访问。
- ② 可以使用 `at()` 进行访问。
- ③ 还可以通过迭代器进行访问。

(5) `map` 元素操作

`map` 中常用的元素操作有查找特定元素的函数 `find()` 和将映射清空的函数 `clear()`。用函数 `find()` 查找特定元素时，若找到则返回该元素的迭代器，若未找到则返回迭代器 `end()`。

(6) `map` 迭代器操作

`map` 中常用的迭代器操作有返回映射中首元素迭代器的 `begin()`，以及返回映射尾元素之后一个位置的迭代器 `end()`。

示例代码

```
#include <iostream>
#include <cstdio>
#include <map>

using namespace std;

map<string, int> myMap;

int main() {
    myMap["Emma"] = 67;
    myMap["Benedict"] = 100;
    myMap.insert(pair<string, int>("Bob", 72));
    myMap.insert(pair<string, int>("Mary", 85));
    myMap.insert(pair<string, int>("Alice", 93));
    printf("the size of myMap: %d\n", myMap.size());
    printf("the score of Benedict: %d\n", myMap["Benedict"]);
    printf("the score of Mary: %d\n", myMap.at("Mary"));
    myMap.erase("Benedict");
    map<string, int>::iterator it; //定义迭代器
    for (it = myMap.begin(); it != myMap.end(); it++) {
        cout << "the score of " << it->first;
        cout << ": " << it->second << endl;
    }
    myMap.clear();
    if (myMap.empty()) {
        printf("myMap is empty\n");
    } else {
        printf("myMap is not empty\n");
    }
    it = myMap.find("Bob");
    if (it != myMap.end()) {
        printf("Bob is found\n");
    } else {
```

```

        printf("Bob is not found\n");
    }
    printf("the size of myMap: %d\n", myMap.size());
    return 0;
}

```

示例结果

```

the size of myMap: 5
the score of Benedict: 100
the score of Mary: 85
the score of Alice: 93
the score of Bob: 72
the score of Emma: 67
the score of Mary: 85
myMap is empty
Bob is not found
the size of myMap: 0

```

2. 映射的应用

例题 10.7 查找学生信息（清华大学复试上机题）

题目描述：

输入 N 个学生的信息，然后进行查询。

输入：

输入的第一行为 N ，即学生的个数（ $N \leq 1000$ ）。
接下来的 N 行包括 N 个学生的信息，信息格式如下：

```

01 李江 男 21
02 刘唐 男 23
03 张军 男 19
04 王娜 女 19

```

然后输入一个 M （ $M \leq 10000$ ），接下来会有 M 行，代表 M 次查询，每行输入一个学号，格式如下：

```

02
03
01
04

```

输出：

输出 M 行，每行包括一个对应于查询的学生的信息。
如果没有对应的学生信息，则输出 “No Answer!”

样例输入：

```
4
01 李江 男 21
02 刘唐 男 23
03 张军 男 19
04 王娜 女 19
5
02
03
01
04
03
```

样例输出：

```
02 刘唐 男 23
03 张军 男 19
01 李江 男 21
04 王娜 女 19
03 张军 男 19
```

提交网址：

<http://t.cn/AiCuVluY>

【分析】

初见本题时，读者肯定认为这道题应是前面关于“查找”一章的内容。本题确实属于查找类题目，但本题若按照线性查找，则肯定会超时；若按照二分查找，则需要对所有元素进行排序。然而，学生的信息本身并无大小可言，而且本题也不要求将学生的信息进行比较，只需根据学生的学号返回其信息即可，是一道考查映射的典型题目。

其实，映射的思想就是为了提高查找的效率而提出。与一般的查找策略相比，由于映射可以直接通过元素的关键字，得到它所对应的映射值，因此通过映射进行查找会更加高效和便捷。

题目要求用学号进行查找，因此将学号设为映射的关键字，将学生信息设为映射的映射值。

代码 10.7

```
#include <iostream>
#include <cstdio>
#include <map>
```

```

using namespace std;

map<string, string> student;

int main() {
    int n;
    scanf("%d", &n);
    getchar(); //吃掉回车
    for (int i = 0; i < n; ++i) {
        string str;
        getline(cin, str);
        int pos = str.find(" "); //分界点
        string key = str.substr(0, pos); //学号作为关键字
        student[key] = str; //信息作为映射值
    }
    int m;
    scanf("%d", &m);
    for (int i = 0; i < m; ++i) {
        string key;
        cin >> key;
        string answer = student[key];
        if (answer == "") { //找不到该学生
            answer = "No Answer!";
        }
        cout << answer << endl;
    }
    return 0;
}

```

例题 10.8 魔咒词典（浙江大学复试上机题）

题目描述：

哈利·波特在魔法学校的必修课之一就是学习魔咒。据说魔法世界有 100000 种不同的魔咒，波特很难全部记住，但为了对抗强敌，他必须在危急时刻能够调用任何一个需要的魔咒，所以他需要你的帮助。给你一部魔咒词典。当哈利听到一个魔咒时，你的程序必须告诉他那个魔咒的功能；当哈利需要某个功能但不知道该用什么魔咒时，你的程序要替他找到相应的魔咒。如果他要的魔咒不在词典中，那么你的程序就输出 “what?”。

输入：

首先列出词典中不超过 100000 条的不同魔咒词条，每条的格式如下：

[魔咒] 对应功能

其中“魔咒”和“对应功能”分别为长度不超过 20 和 80 的字符串，字符串中保证不包含字符“[”和“]”，且“]”和后面的字符串之间有且仅有一个空格。词典最后一行以“@END@”结束，这一行不属于词典中的词条。

词典之后的一行包含正整数 N ($N \leq 1000$)，随后是 N 个测试用例。每个测试用例占一行，或者给出“[魔咒]”，或者给出“对应功能”。输入包含多行，每行一个字符串。

输出：

每个测试用例的输出占一行，输出魔咒对应的功能，或者功能对应的魔咒。如果魔咒不在词典中，那么输出“what?”

样例输入：

```
[expelliarmus] the disarming charm
[rictusempra] send a jet of silver light to hit the enemy
[tarantallegra] control the movement of one's legs
[serpensortia] shoot a snake out of the end of one's wand
[lumos] light the wand
[obliviate] the memory charm
[expecto patronum] send a Patronus to the dementors
[accio] the summoning charm
@END@
4
[lumos]
the summoning charm
[arha]
take me to the sky
```

样例输出：

```
light the wand
accio
what?
what?
```

提交网址：

<http://t.cn/AiCufczt>

【分析】

本题也是一道考查映射的经典题目，不同于上道题中只有单向映射关系，本题不仅要求“魔咒”能够映射“功能”，而且还需要“功能”也能映射“魔咒”，即双向映射。

估计大家会想到使用两个映射：一个映射将“魔咒”当做关键字，将“功能”当做映射值；

另一个映射将“功能”当做关键字，将“魔咒”当做映射值。这样的做法固然是可行的，不过由于题目中明确指出“魔咒”和“功能”不会出现重复，因此可以将双向映射同时放在一个映射中。

代码 10.8

```
#include <iostream>
#include <cstdio>
#include <string>
#include <map>

using namespace std;

map<string, string> dictionary;

int main() {
    string str;
    while (getline(cin, str)) {
        if (str == "@END@") {
            break;
        }
        int pos = str.find("["); //分界点
        string key = str.substr(0, pos + 1); //魔咒
        string value = str.substr(pos + 2); //功能
        dictionary[key] = value;
        dictionary[value] = key;
    }
    int n;
    scanf("%d", &n);
    getchar(); //吃掉回车
    while (n--) {
        string key;
        getline(cin, key);
        string answer = dictionary[key];
        if (answer == "") { //魔咒或者功能找不到
            answer = "what?";
        } else if (answer[0] == '[') { //魔咒需要删除方括号
            answer = answer.substr(1, answer.size() - 2);
        }
        cout << answer << endl;
    }
}
```



```
}  
return 0;  
}
```

例题 10.9 子串计算（北京大学复试上机题）

题目描述：

给出一个 01 字符串（长度不超过 100），求其每个子串出现的次数。

输入：

输入包含多行，每行一个字符串。

输出：

对每个字符串，输出它所有出现次数在 1 次以上的子串和这个子串出现的次数，输出按字典序排序。

样例输入：

10101

样例输出：

0 2
01 2
1 3
10 2
101 2

提交网址：

<http://t.cn/AiCuJt15>

【分析】

本题可以利用映射，将子串当做关键字，将其出现的次数当做映射值。这样，通过遍历整个字符串中的子串，每次将子串的映射值数加 1 即可。由于 `map` 底层采用的是红黑树，因此在其内部仍然会将元素按照关键字进行排序，故输出顺序也刚好符合题目的需求。

代码 10.9

```
#include <iostream>
#include <cstdio>
#include <string>
#include <map>

using namespace std;
```

```

int main() {
    string str;
    while (cin >> str) {
        map<string, int> number;
        for (int i = 0; i <= str.size(); ++i) {
            for (int j = 0; j < i; ++j) {
                string key = str.substr(j, i - j); //每个子串
                number[key]++; //映射值加 1
            }
        }
        map<string, int>::iterator it; //定义迭代器
        for (it = number.begin(); it != number.end(); ++it) {
            if (1 < it->second) {
                cout << it->first << " " << it->second << endl;
            }
        }
    }
    return 0;
}

```

习题 10.4 统计同成绩学生人数（浙江大学复试上机题）

读入 N 名学生的成绩，将获得某一给定分数的学生人数输出。

提交网址：

<http://t.cn/AiCuM7nj>

习题 10.5 开门人和关门人（浙江大学复试上机题）

每天第一个到机房的人要把门打开，最后一个离开的人要把门关好。现有一堆杂乱的机房签到、签离记录，请根据记录找出当天开门和关门的人。

提交网址：

<http://t.cn/AiCuMO9f>

习题 10.6 谁是你的潜在朋友（北京大学复试上机题）

“臭味相投”是我们描述朋友时喜欢用的词汇。两个人是朋友通常意味着他们存在着许多共同的兴趣。然而，作为一名宅男，你发现自己与他人相互了解的机会并不太多。幸运的是，你意外地得到了一份北京大学图书馆的图书借阅记录，于是你挑灯熬夜地编程，想从中发现潜在的朋友。首先你对借阅记录进行了一番整理，把 N 个读者依次编号为 $1, 2, \dots, N$ ，把 M 本书依次编号为 $1,$

2,...,M。同时,按照“臭味相投”的原则,和你喜欢读同一本书的人,就是你的潜在朋友。你现在的任务是从这份借阅记录中计算出每个人有几个潜在朋友。

提交网址:

<http://t.cn/AiCux4f7>

小结

本章介绍了机试中常见的非线性数据结构，包括二叉树、二叉排序树、优先队列和散列表。本章的难度要大于线性数据结构的那一章，希望大家通过反复练习，彻底掌握常考的非线性数据结构。

第 11 章 图 论

本章主要讨论计算机考研机试中有关图论的各类问题，主要包括并查集、最小生成树、最短路径、拓扑排序等。在求解这些问题之前，先为读者介绍一些有关图的基本概念和术语。在此基础上介绍如何实现作为抽象数据的图结构，主要介绍邻接矩阵和邻接表这两种常见的实现方式。

11.1 概述

图结构是用来求解很多问题的有力方法。图 (Graph) 定义为 $G=(V, E)$ 。也就是说，图是由顶点 (Vertex) 集合 V 和边 (Edge) 集合 E 组成的。边集合 E 中的元素是顶点集合 V 中的某对顶点 (u, v) 。由于 V 和 E 都是有穷集合，因此将它们的模记为 $v=|V|$ 和 $e=|E|$ ，常用它们的模分析算法的复杂度。

图根据边集合 E 中的元素 (u, v) 是否有次序，分为无向图和有向图，如图 11.1 所示。在无向图中，每条边都没有方向，边 (u, v) 仅表示顶点 u 和顶点 v 之间存在边相连。而在有向图中，每条边都有方向，边 (u, v) 表示存在一条从顶点 u 指向顶点 v 的有向边。

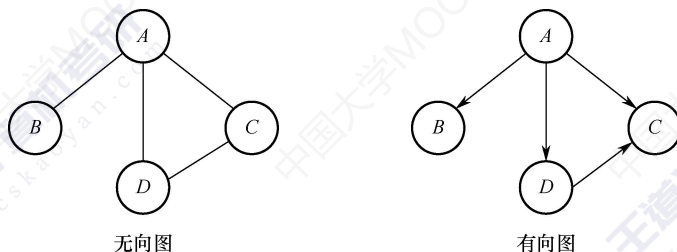


图 11.1 无向图和有向图

若存在边 $e=(u, v)$ ，则称顶点 u 和顶点 v 互为邻接 (adjacent)；而顶点 u 和顶点 v 都与边 e 关联 (incident)。在无向图中，与顶点关联的边数称为该点的度 (degree)。在图 11.1 左侧的无向图中，顶点 $\{A, B, C, D\}$ 的度为 $\{3, 1, 2, 2\}$ 。而在有向图中，度分为出度和入度，对于边 $e=(u, v)$ ， e 为 u 的出边 (outgoing edge)，同时 e 也是 v 的入边 (incoming edge)。与顶点关联的出边数称为该点的出度 (out degree)，与顶点关联的入边数称为该点的入度 (in degree)。在图 11.1 右侧的有向图中，顶点 $\{A, B, C, D\}$ 的出度为 $\{3, 0, 0, 1\}$ ，入度为 $\{0, 1, 2, 1\}$ 。

如果为图上的边添加代表某种实际意义的数值，那么称这些数值为边的权，称包含这样的

带权边的图为带权图，如图 11.2 所示。

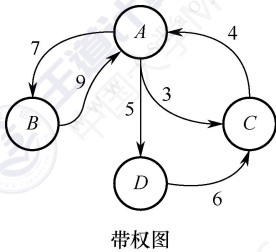


图 11.2 带权图

介绍完图的定义后，那么在计算机中应该如何表示图结构呢？常见的有邻接矩阵和邻接表这两种实现方法。

1. 邻接矩阵

邻接矩阵（adjacency matrix）是最直接的图结构实现方式，它用一个二维矩阵来表示图的信息，二维矩阵中的每个单元表述一对顶点之间的邻接关系，因此得名邻接矩阵。

对于无权图，可用 $matrix[u][v]$ 为 1 或 0 来表示 u 和 v 之间是否有边，如图 11.3 所示。

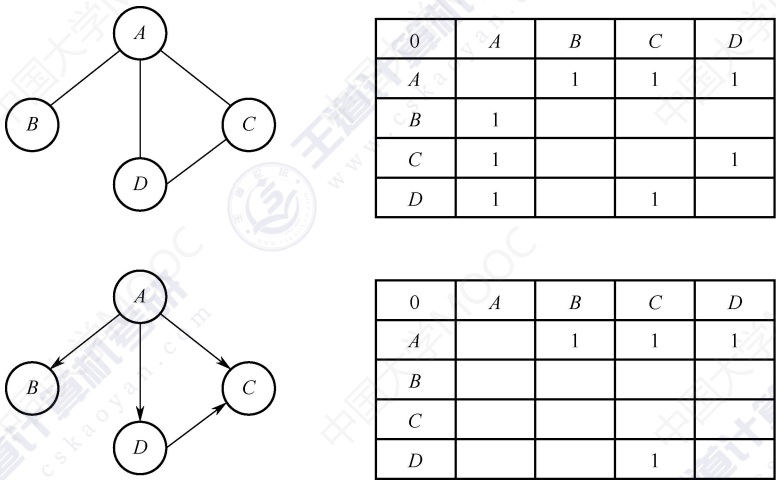


图 11.3 无权图及其邻接矩阵

对于带权图，可用 $matrix[u][v]$ 的值来代表 u 和 v 之间边的权值，对不存在的边一般取值为 ∞ ，如图 11.4 所示。图 11.3 和图 11.4 中的 3 个邻接矩阵中，空白单元对应的边不存在，取值统一标注在矩阵的左上角。

邻接矩阵的原理易懂而且用法也比较简单，在确定某对顶点之间是否存在关系时，只需访问二维数组中的相关单元即可，耗时较少。然而，邻接矩阵也存在一些缺陷。例如，若需要遍历与某顶点相邻的所有顶点，则需要依次遍历二维数组中某行的所有元素，通过其值判断是否相邻；也就是说，即使只有一个点与其相邻，也需要耗费大量的时间来遍历该行中所有的数组单元，时间利用率较低。同时，使用邻接矩阵来保存顶点个数为 n 的图时，其空间复杂度为 $O(n^2)$ 。

若所要表示的图为稀疏图时，则该矩阵将为稀疏矩阵，大量的空间会被浪费。因此，只有当表示的图为稠密图，且需要频繁地判断某特定顶点对是否相邻时，使用邻接矩阵才较为适宜。

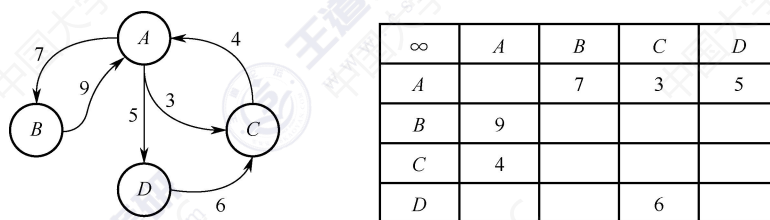


图 11.4 带权图及其邻接矩阵

2. 邻接表

邻接矩阵效率低的主要原因是，矩阵中大量单元对应的边并不存在，而这是由于一开始便分配了空间的静态空间管理策略导致的。为提高效率，可以用动态分配的链表来代替静态分配的空间。这便是邻接表（adjacency list）的思想。邻接表为图中的每个顶点建立一个单链表，单链表中保存与该顶点相邻的所有顶点及其相关信息，如图 11.5 所示。

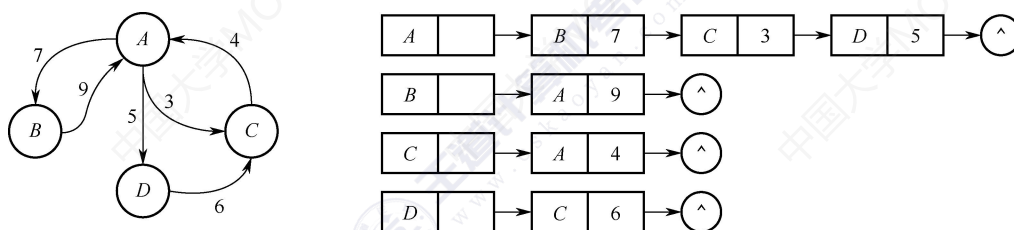


图 11.5 带权图及其邻接表

在遍历与某个特定顶点相邻的顶点时，邻接表的效率非常高。与邻接矩阵不同，邻接表无需遍历不存在关系的顶点，遍历的顶点个数即为有效的顶点个数，大大节省了时间。同时，其空间复杂度为 $O(n + e)$ ，与邻接矩阵相比，空间利用率较高。但是，与邻接矩阵相比，当邻接表需要判断顶点 u 与 v 间是否存在关系时，就会显得比较烦琐——它需要遍历 u 和 v 的所有邻接顶点，才能判定它们之间是否存在关系。因此，当问题中存在大量遍历邻接顶点的操作而较少判断两个特定顶点的关系时，使用邻接表较为适宜。

尽管邻接表访问单条边的效率不高，但擅长批量处理一个顶点的所有关联边。大部分图论算法都是以批处理的形式进行的。因此，总体而言，邻接表的效率要高于邻接矩阵，所以本章中的代码多以邻接表的方式实现。

虽然叫邻接表，但在实际编码过程中通常并不采用链表的方式来实现，而采用向量来实现。

11.2 并查集

本节讨论图论问题中的一种常用数据结构——并查集（Union Find）。并查集用于处理一些不交集（Disjoint Sets）的合并和查询问题。读者应该比较了解集合的概念，如数字集合 $A\{1, 2, 3, 4\}$ ， $B\{5, 6, 7\}$ ， $C\{8, 0\}$ 。并查集有如下两个功能：一是判断任意两个元素是否属于同一个集合，二是按照要求合并不同的集合。

首先，将集合在逻辑上表示为树结构，每个结点都指向其父结点，而树中的元素并无顺序之分，只要在同一棵树上，便说明在同一个集合中，如图 11.6 所示。

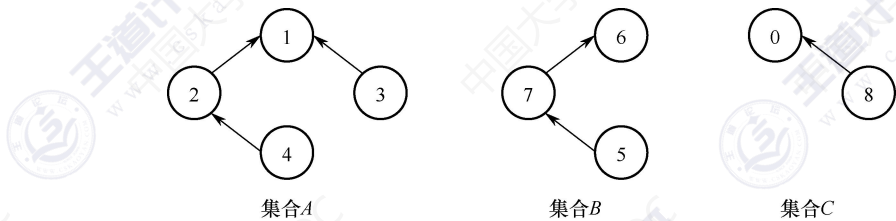


图 11.6 集合的树结构

并查集有两个操作，分别是查找（Find）和合并（Union）。

查找：确定元素属于哪个集合。这种方法的步骤是：不断向上查找，直到找到它的根结点，之后根据根结点是否相同来判断两个元素是否属于同一集合，如图 11.7 所示。

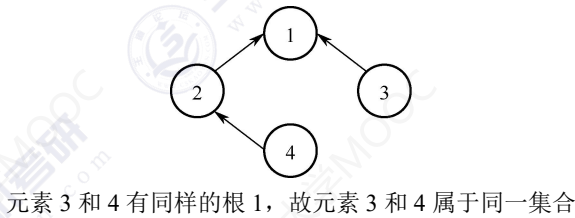


图 11.7 查找

合并：将两个子集合并成同一个集合。这种方法的步骤是：将一棵树作为另外一棵树的子树，从而使得两棵树变成一棵更大的树，如图 11.8 所示

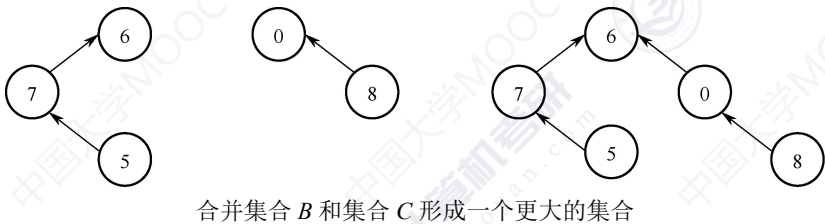


图 11.8 合并

但是，采用这种合并策略而不加以任何约束，可能会造成某些致命的问题。如前文所述，对集合的操作主要是通过查找树的根结点来实现的，在并查集中最主要的操作是查找某个结点所在

树的根结点，通过不断查找结点的父亲结点，直到找到一个不存在父亲结点的结点为止，该结点即为根结点。这个过程所需耗费的时间和该结点到树根的距离有关，即和树的高度有关。在合并两棵树的过程中，若只是简单地将两棵树合并而不采取任何措施，那么树高可能会逐渐增加，查找根结点的耗时也会相应地逐渐增大，极端情况下该树可能会退化成单链表，在单链表上查找根结点的操作将会变得非常耗时。如图 11.9 所示，树的不同形态对查找效率影响巨大。

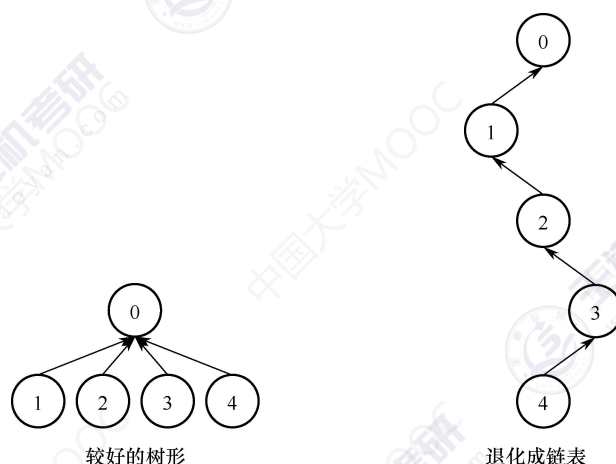


图 11.9 树的不同形态对查找效率的影响

为了避免因为树的退化而产生额外的时间消耗，在合并两棵树时就不能任由其发展，而应加入一定的约束和优化，使其尽可能保持较低的树高。为了达到这一目的，可以在查找某个特定结点的根结点的同时，将其与根结点之间的所有结点都直接指向根结点，这个过程被称为路径压缩，如图 11.10 所示。

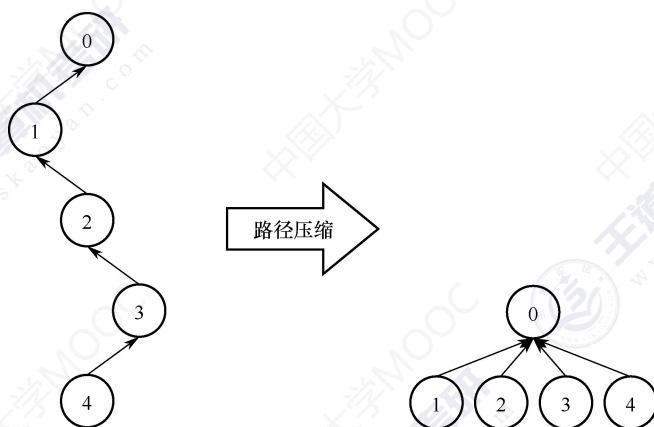


图 11.10 路径压缩

完成路径压缩的工作后，树的形态会发生巨大改变，如树高大大降低，而该树表示的集合信息却未发生任何改变，所以其在保证集合信息不变的情况下，大大优化了树结构，为后续查找工作节约了大量时间。

此外，在合并两棵树时，总是将高度较低的树，作为高度较高的树的子树进行合并。因为影响查找效率的是树高，高度较低的树作为高度较高的树的子树时，不会增加合并后的树的高度，从而提高了之后的查找效率。

例题 11.1 畅通工程（浙江大学复试上机题）

题目描述：

某省调查城镇交通状况，得到现有城镇道路统计表，表中列出了每条道路直接连通的城镇。省政府“畅通工程”的目标是使全省任何两个城镇间都可以实现交通（但不一定有直接的道路相连，只要互相间接通过道路可达即可）。问最少还需要建设多少条道路？

输入：

测试输入包含若干测试用例。每个测试用例的第一行给出两个正整数，分别是城镇数目 N ($N < 1000$) 和道路数目 M ；随后的 M 行对应 M 条道路，每行给出一对正整数，分别是该条道路直接连通的两个城镇的编号。为简单起见，城镇从 1 到 N 编号。

注意：两个城市之间可以有多条道路相通，也就是说

3 3

1 2

1 2

2 1

这种输入也是合法的。

当 N 为 0 时，输入结束，该用例不被处理。

输出：

对每个测试用例，在一行中输出最少还需要建设的道路数目。

样例输入：

4 2

1 3

4 3

3 3

1 2

1 3

2 3

5 2

1 2

3 5

999 0

0

样例输出：

1

0

2

998

提交网址:

<http://t.cn/AiOvBHj9>

【分析】

本题描述的是一个实际的问题。在这个问题中，可以将城镇抽象为顶点，将有道路连通的城镇划分到同一个集合中，从而将问题转换为求集合的个数。因此，本题可以使用并查集完成。初始时每个城镇都是孤立的，读入已经建成的道路后，将道路连接城镇所在的集合进行合并，表示这两个集合中的所有结点已经连通。对所有的道路重复该操作，最后计算所有的结点被保存到几个集合中即可。

代码 11.1

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 1000;

int father[MAXN];           //父亲结点
int height[MAXN];          //结点高度

void Initial(int n) {       //初始化
    for (int i = 0; i <= n; i++) {
        father[i] = i;     //每个结点的父亲为自己
        height[i] = 0;     //每个结点的高度为零
    }
}

int Find(int x) {           //查找根结点
    if (x != father[x]) {   //路径压缩
        father[x] = Find(father[x]);
    }
    return father[x];
}

void Union(int x, int y) {  //合并集合
    x = Find(x);
    y = Find(y);
    if (x != y) {           //矮树作为高树的子树
```



```

    if (height[x] < height[y]) {
        father[x] = y;
    } else if (height[y] < height[x]) {
        father[y] = x;
    } else {
        father[y] = x;
        height[x]++;
    }
}
return ;
}

int main() {
    int n, m;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        scanf("%d", &m);
        Initial(n);                //初始化
        while (m--) {
            int x, y;
            scanf("%d", &x);
            scanf("%d", &y);
            Union(x, y);            //合并集合
        }
        int answer = -1;
        for (int i = 1; i <= n; i++) {
            if (Find(i) == i) {
                answer++;
            }
        }
        printf("%d\n", answer);
    }
    return 0;
}

```

在图论中，并查集最常用来判断图是否为连通图，或用来求图的连通分量。连通图的定义如下：在一个无向图 G 中，若顶点 u 到顶点 v 有路径相连，则称 u 和 v 是连通的。若图中任意两点都是连通的，则图被称为连通图。而无向图 G 中的一个极大连通子图称为 G 的一个连通分

量。连通图只有一个连通分量，即其自身；非连通的无向图有多个连通分量。

例题 11.2 连通图（吉林大学复试上机题）

题目描述：

给定一个无向图和其中的所有边，判断这个图是否所有顶点都是连通的。

输入：

每组数据的第一行是两个整数 n 和 m ($0 \leq n \leq 1000$)。 n 表示图的顶点数目， m 表示图中边的数目。随后有 m 行数据，每行有两个值 x 和 y ($0 < x, y \leq n$)，表示顶点 x 和 y 相连，顶点的编号从 1 开始计算。输入不保证这些边是否重复。

输出：

对于每组输入数据，若所有顶点都是连通的，则输出 “YES”，否则输出 “NO”。

样例输入：

```
4 3
1 2
2 3
3 2
3 2
1 2
2 3
```

样例输出：

```
NO
YES
```

提交网址：

<http://t.cn/AiO77VoA>

【分析】

本题是一道求连通分量的典型例题，求解方法是不断地合并图中边相连的两个点所属的集合，最后计算出的集合个数便是图连通分量的个数。如果图的连通分量等于 1，那么该图就是一个连通图。

代码 11.2

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 1000;

int father[MAXN];           //父亲结点
int height[MAXN];          //结点高度

void Initial(int n) {       //初始化
```

```

for (int i = 0; i <= n; i++) {
    father[i] = i;           //每个结点的父亲为自己
    height[i] = 0;          //每个结点的高度为零
}

int Find(int x) {           //查找根结点
    if (x != father[x]) {   //路径压缩
        father[x] = Find(father[x]);
    }
    return father[x];
}

void Union(int x, int y) {   //合并集合
    x = Find(x);
    y = Find(y);
    if (x != y) {           //矮树作为高树的子树
        if (height[x] < height[y]) {
            father[x] = y;
        } else if (height[y] < height[x]) {
            father[y] = x;
        } else {
            father[y] = x;
            height[x]++;
        }
    }
    return ;
}

int main() {
    int n, m;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        scanf("%d", &m);
        Initial(n);         //初始化
        while (m--) {
            int x, y;

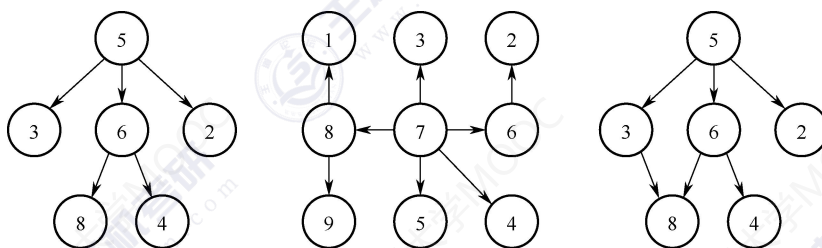
```

```

scanf("%d", &x);
scanf("%d", &y);
Union(x, y);                                //合并集合
}
int component = 0;                           //连通分量
for (int i = 1; i <= n; i++) {
    if (Find(i) == i) {                      //集合数目
        component++;
    }
}
if (component == 1) {
    printf("YES\n");
} else {
    printf("NO\n");
}
return 0;
}

```

例题 11.3 Is It A Tree? (北京大学复试上机题)



题目描述:

A tree is a well-known data structure that is either empty (null, void, nothing) or is a set of one or more nodes connected by directed edges between nodes satisfying the following properties. There is exactly one node, called the root, to which no directed edges point. Every node except the root has exactly one edge pointing to it. There is a unique sequence of directed edges from the root to each node. For example, consider the illustrations below, in which nodes are represented by circles and edges are represented by lines with arrowheads. The first two of these are trees, but the last is not.

In this problem you will be given several descriptions of collections of nodes connected by directed edges. For each of these you are to determine if the collection satisfies the definition of a tree or not.

输入:

The input will consist of a sequence of descriptions (test cases) followed by a pair of negative integers. Each test case will consist of a sequence of edge descriptions followed by a pair of zeroes. Each edge description will consist of a pair of integers; the first integer identifies the node from which the edge begins, and the second integer identifies the node to which the edge is directed. Node numbers will

always be greater than zero and less than 10000.

输出:

For each test case display the line "Case k is a tree." or the line "Case k is not a tree.", where k corresponds to the test case number (they are sequentially numbered starting with 1).

样例输入:

```
6 8 5 3 5 2 6 4
5 6 0 0
```

```
8 1 7 3 6 2 8 9 7 5
7 4 7 8 7 6 0 0
```

```
3 8 6 8 6 4
5 3 5 6 5 2 0 0
-1 -1
```

样例输出:

Case 1 is a tree.

Case 2 is a tree.

Case 3 is not a tree.

提交网址:

<http://t.cn/AiO7FyDO>

【题目大意】

树是一个众所周知的数据结构，它要么是空集，要么是由满足以下属性的结点之间的有向边连接的一个或多个结点的集合。如果只有一个结点，那么称其为根结点，没有任何有向边指向根结点。除根结点之外的每个结点都只有一个指向它的边。从根到每个结点有一个独特的有向边序列。例如，在上图中，结点由圆圈表示，边由带箭头的线条表示。其中前两个是树，但最后一个不是。

在这个问题中，你将获得由有向边连接的结点集合的描述。对于其中的每个集合，你将确定该集合是否满足树的定义。

【分析】

这道题用来判断给出的集合能够构成树。因此，不仅需要判断所有点是否属于一个集合，还需要判断各个点是否符合树的定义，而判断各点是否符合树的定义可以转换为判断它的入度是否符合要求，根结点的入度为 0，而其余点的入度为 1。只要各个结点满足入度要求，只有一个根结点，以及各个结点都属于同一集合，就可以构成一棵树。

代码 11.3

```
#include <iostream>
#include <cstdio>

using namespace std;
```

```
const int MAXN = 10000;
```

```
int father[MAXN];           //父亲结点  
int height[MAXN];          //结点高度  
int inDegree[MAXN];        //入度  
bool visit[MAXN];          //标记
```

```
void Initial() {            //初始化  
    for (int i = 0; i < MAXN; i++) {  
        father[i] = i;  
        height[i] = 0;  
        inDegree[i] = 0;  
        visit[i] = false;  
    }  
}
```

```
int Find(int x) {           //查找根结点  
    if (x != father[x]) {  
        father[x] = Find(father[x]);  
    }  
    return father[x];  
}
```

```
void Union(int x, int y) {  //合并集合  
    x = Find(x);  
    y = Find(y);  
    if (x != y) {  
        if (height[x] < height[y]) {  
            father[x] = y;  
        } else if (height[y] < height[x]) {  
            father[y] = x;  
        } else {  
            father[y] = x;  
            height[x]++;  
        }  
    }  
    return ;  
}
```



```

bool IsTree() {
    bool flag = true;
    int component = 0; //连通分量数目
    int root = 0; //根结点数目
    for (int i = 0; i < MAXN; ++i) {
        if (!visit[i]) {
            continue;
        }
        if (father[i] == i) {
            component++;
        }
        if (inDegree[i] == 0) {
            root++;
        } else if (inDegree[i] > 1) { //入度不满足要求
            flag = false;
        }
    }
    if (component != 1 || root != 1) { //不符合树定义
        flag = false;
    }
    if (component == 0 && root == 0) { //空集也是树
        flag = true;
    }
    return flag;
}

int main() {
    int x, y;
    int caseNumber = 0;
    Initial();
    while (scanf("%d%d", &x, &y) != EOF) {
        if (x == -1 && y == -1) {
            break;
        }
        if (x == 0 && y == 0) {
            if (IsTree()) {
                printf("Case %d is a tree.\n", ++caseNumber);
            } else {

```

```

        printf("Case %d is not a tree.\n", ++caseNumber);
    }
    Initial();
} else {
    Union(x, y);
    inDegree[y]++;
    visit[x] = true;
    visit[y] = true;
}
}
return 0;
}

```

习题 11.1 找出直系亲属（浙江大学复试上机题）

若 A, B 是 C 的父亲、母亲，则 A, B 是 C 的 parent， C 是 A, B 的 child；若 A, B 是 C 的（外）祖父、祖母，则 A, B 是 C 的 grandparent， C 是 A, B 的 grandchild；若 A, B 是 C 的（外）曾祖父、曾祖母，则 A, B 是 C 的 great-grandparent， C 是 A, B 的 great-grandchild。之后若再多一辈，则在关系上加一个“great-”。输入两个不超过整型定义的非负十进制整数 A 和 B ($\leq 231-1$)，输出 $A+B$ 的 m ($1 < m < 10$)。

提交网址：

<http://t.cn/AiOzTX5c>

习题 11.2 第一题（上海交通大学复试上机题）

该题的目的是要你统计图的连通分支数。

提交网址：

<http://t.cn/AiOhkgMJ>

习题 11.3 Head of a Gang（浙江大学复试上机题）

【题目大意】警察找到一个团伙头脑的方法之一是检查人们的电话。如果 A 和 B 之间有电话，那么我们说 A 和 B 是相关的。关系的权重被定义为两个人之间进行的所有电话呼叫的总时间长度。“帮派”是超过 2 个人的群集，彼此相关且总关系权重大于给定的威胁 K 。在每个群体中，总权重最大的群体是头脑。现在给出一个电话清单，你应该能够找到帮派和头脑。

提交网址：

<http://t.cn/AiOzQMBH>

11.3 最小生成树

本节讨论图论中的一个经典的问题——最小生成树（Minimum Spanning Tree, MST）。在一个无向连通图 $G(V, E)$ 中，如果存在一个连通子图，它包含原图中的所有顶点和部分边，且这个子图不存在回路，那么就称这个子图为原图的一棵生成树。在带权无向连通图中，所有生成树中边权的和最小的那一棵（或几棵）被称为该无向图的最小生成树。如图 11.11 所示。

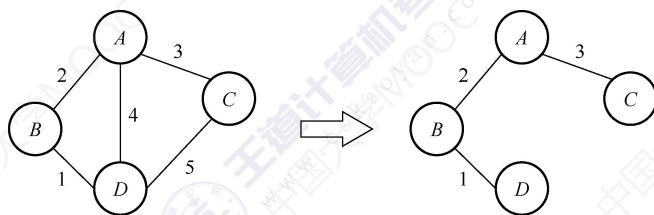


图 11.11 最小生成树

最小生成树问题在实际生活中应用广泛。例如，在通信基站之间修建通信光缆使所有的基站间可以直接或间接通信，最少需要多长的光缆？要利用最小生成树来求解实际问题，必须先学会怎样求解一个连通图的最小生成树。

构造最小生成树的算法很多，最常见的有 Kruskal 算法和 Prim 算法。由于篇幅有限，这里只介绍 Kruskal 算法，它编写起来比较简单，而且效率也非常高。

先来看一下 Kruskal 算法的原理。在要求解的连通图中，任意选择一些点属于集合 A ，剩余的点属于集合 B ，最小生成树必定存在一条权值最小的边，并且这条边的两个顶点分别属于集合 A 和集合 B 的边（即连通两个集合的边）。

可以用反证法来证明这个命题：设连通顶点集 A 和顶点集 B 的边中权值最小的一条边为 e ，在该图所有的最小生成树中都不包含该边。但在任意一棵最小生成树中必有一条边连通集合 A 和集合 B （若没有则两集合不连通，若大于一条则出现回路），设该边为 e' 。由命题假设可知， e 的权值不大于 e' 的权值，若用边 e 替换边 e' ，则替换后子图依然为原图的一棵生成树，该生成树的权值为原最小生成树的权值减去 e' 的权值后加上 e 的权值，该值将不会大于原最小生成树的权值，新的生成树也是原图的一棵最小生成树，于是就得到了一棵包含边 e 的最小生成树，这与假设矛盾，原命题得证。

Kruskal 算法的步骤如下：

- ① 初始时所有顶点属于孤立的集合。
- ② 按照边权递增顺序遍历所有边，若遍历到的边的两个顶点仍分属不同的集合（该边即为连通这两个集合的边中权值最小的那条），则确定该边为最小生成树上的一条边，并将该边两个顶点分属的集合合并。
- ③ 遍历完所有边后，若原图连通，则被选取的边和所有顶点构成最小生成树；若原图不连通，最小生成树不存在。

如以上步骤所示，在用 Kruskal 算法求解最小生成树的过程中会涉及大量的集合操作，恰好可以使用上一节中讨论的并查集来实现这些操作。

例题 11.4 还是畅通工程（浙江大学复试上机题）

题目描述：

某省调查乡村交通状况，得到的统计表中列出了任意两村庄间的距离。省政府“畅通工程”的目标是使全省任何两个村庄间都可以实现公路交通（但不一定有直接的公路相连，只要能间接通过公路可达即可），并要求铺设的公路总长度为最小。请计算最小的公路总长度。

输入：

测试输入包含若干测试用例。每个测试用例的第一行给出村庄数目 N ($N < 100$)；随后的 $N(N-1)/2$ 行对应村庄间的距离，每行给出一对正整数，分别是两个村庄的编号，以及这两个村庄间的距离。为简单起见，村庄从 1 到 N 编号。当 N 为 0 时，输入结束，该用例不处理。

输出:

对每个测试用例, 在一行中输出最小的公路总长度。

样例输入:

```
3
1 2 1
1 3 2
2 3 4
4
1 2 1
1 3 4
1 4 1
2 3 3
2 4 2
3 4 5
0
```

样例输出:

```
3
5
```

提交网址:

<http://t.cn/AiWudOC6>

【分析】

这道题是在给定所有的道路中选取一些, 使所有城市直接或间接连通, 而且使道路的总长度最短。该例是典型的最小生成树问题。可以通过将城市抽象成图上的顶点, 将道路抽象成连接点的边, 其长度即为边的权值。经过这样的抽象后, 只需求得该图的最小生成树后, 将最小生成树上的边权进行累加, 原问题便可得到求解。

代码 11.4

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 100;

struct Edge {
    int from;           //边的起点
    int to;             //边的终点
    int length;         //边的长度
    bool operator< (const Edge& e) const {
```

```

        return length < e.length;
    }
};

Edge edge[MAXN * MAXN];           //边集
int father[MAXN];                  //父亲结点
int height[MAXN];                  //结点高度

void Initial(int n) {               //初始化
    for (int i = 0; i <= n; i++) {
        father[i] = i;
        height[i] = 0;
    }
}

int Find(int x) {                   //查找根结点
    if (x != father[x]) {
        father[x] = Find(father[x]);
    }
    return father[x];
}

void Union(int x, int y) {          //合并集合
    x = Find(x);
    y = Find(y);
    if (x != y) {
        if (height[x] < height[y]) {
            father[x] = y;
        } else if (height[y] < height[x]) {
            father[y] = x;
        } else {
            father[y] = x;
            height[x]++;
        }
    }
    return ;
}

int Kruskal(int n, int edgeNumber) {

```



```

Initial(n);
sort(edge, edge + edgeNumber);    //按权值排序
int sum = 0;
for (int i = 0; i < edgeNumber; ++i) {
    Edge current = edge[i];
    if (Find(current.from) != Find(current.to)) {
        Union(current.from, current.to);
        sum += current.length;
    }
}
return sum;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        int edgeNumber = n * (n - 1) / 2;
        for (int i = 0; i < edgeNumber; ++i) {
            scanf("%d%d%d", &edge[i].from, &edge[i].to, &edge[i].length);
        }
        int answer = Kruskal(n, edgeNumber);
        printf("%d\n", answer);
    }
    return 0;
}

```

例题 11.5 继续畅通工程（浙江大学复试上机题）

题目描述：

省政府“畅通工程”的目标是使全省任何两个村庄间都可以实现公路交通（但不一定有直接的公路相连，只要能间接通过公路可达即可）。现得到城镇道路统计表，表中列出了任意两城镇间修建道路的费用，以及该道路是否已经修通的状态。现请你编写程序，计算出全省畅通需要的最低成本。

输入：

测试输入包含若干测试用例。每个测试用例的第一行给出村庄数目 N ($1 < N < 100$)；随后的 $N(N-1)/2$ 行对应村庄间道路的成本及修建状态，每行给 4 个正整数，分别是两个村庄的编号（从 1 编号到 N ），这两个村庄间道路的成本，以及修建状态：1 表示已建，0 表示未建。当 N 为 0 时输入结束。

输出：

对每个测试用例，在一行中输出最小的公路总长度。

样例输入：

```
3
1 2 1 0
1 3 2 0
2 3 4 0
3
1 2 1 0
1 3 2 0
2 3 4 1
3
1 2 1 0
1 3 2 1
2 3 4 1
0
```

样例输出：

```
3
1
0
```

提交网址：

<http://t.cn/AiW3fcfp>

【分析】

本题和上题非常相似，唯一不同的是，本题中有些道路是已经建设好的，即这些道路无须支付费用，而且已经将道路两端的城市连接了起来。因此，可将已经建好的道路的权值设为 0，再按照上一题的思路，便可得到本题的解。

代码 11.5

```
#include <iostream>
#include <cstdio>
#include <algorithm>

using namespace std;

const int MAXN = 100;

struct Edge {
    int from;                //边的起点
    int to;                  //边的终点
    int length;              //边的长度
    bool operator< (const Edge& e) const {
        return length < e.length;
    }
};

Edge edge[MAXN * MAXN];    //边集
int father[MAXN];          //父亲结点
int height[MAXN];          //结点高度

void Initial(int n) {      //初始化
    for (int i = 0; i <= n; i++) {
        father[i] = i;
        height[i] = 0;
    }
}

int Find(int x) {          //查找根结点
    if (x != father[x]) {
        father[x] = Find(father[x]);
    }
    return father[x];
}
```

```
}

void Union(int x, int y) {           //合并集合
    x = Find(x);
    y = Find(y);
    if (x != y) {
        if (height[x] < height[y]) {
            father[x] = y;
        } else if (height[y] < height[x]) {
            father[y] = x;
        } else {
            father[y] = x;
            height[x]++;
        }
    }
    return ;
}

int Kruskal(int n, int edgeNumber) {
    Initial(n);
    sort(edge, edge + edgeNumber);    //按权值排序
    int sum = 0;
    for (int i = 0; i < edgeNumber; ++i) {
        Edge current = edge[i];
        if (Find(current.from) != Find(current.to)) {
            Union(current.from, current.to);
            sum += current.length;
        }
    }
    return sum;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n == 0) {
            break;
        }
        int edgeNumber = n * (n - 1) / 2;
```

```
for (int i = 0; i < edgeNumber; ++i) {  
    int status;  
    scanf("%d%d%d%d", &edge[i].from, &edge[i].to,  
        &edge[i].length, &status);  
    if (status == 1) {  
        edge[i].length = 0;  
    }  
}  
int answer = Kruskal(n, edgeNumber);  
printf("%d\n", answer);  
}  
return 0;  
}
```

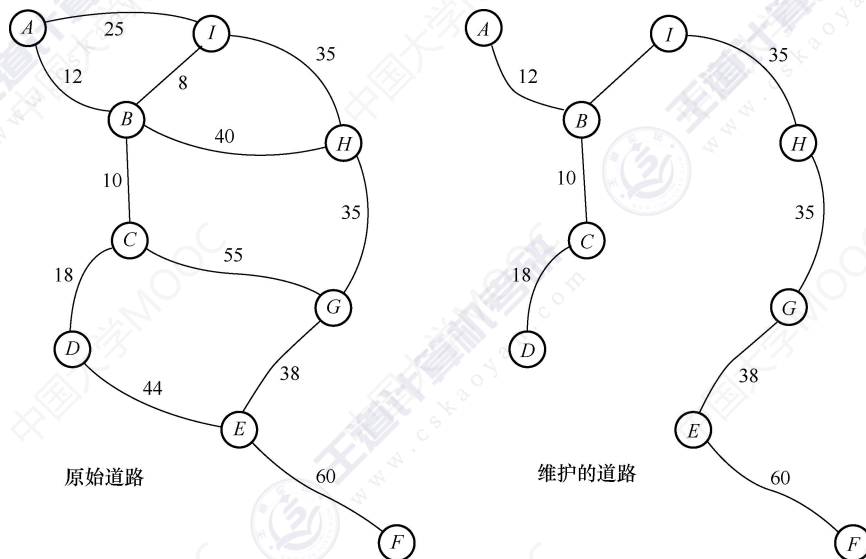
习题 11.4 Freckles (北京大学复试上机题)

【题目大意】在 *Dick Van Dyke* 节目中有一集的内容是，小 Richie 将爸爸背上的雀斑连接起来，形成了自由钟的形状。只可惜其中一个雀斑竟然是一个伤疤，所以里普利的订婚落空了。考虑爸爸 Dick 的背部是一个有各种雀斑位置 (x, y) 的平面。你的工作是告诉 Richie 如何连接点，以尽量减少使用的墨水量。Richie 通过在两点之间绘制直线来连接这些点。当 Richie 完成时，必须有一系列的连线，这些连线能让任何一个雀斑到达任何其他雀斑。

提交网址：

<http://t.cn/AiW3Hbqr>

习题 11.5 Jungle Roads (北京大学复试上机题)



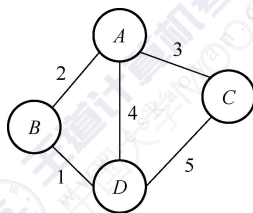
【题目大意】热带岛屿 Lagrishan 的长老现在面临一个问题。几年前，在村庄之间的道路修建上花费了一大笔外援资金。但从林会无情地覆盖道路，因此大型公路网络的维护成本太高。长老理事会必须选择一些道路放弃对其的维护。下方的地图显示了现在正在使用的所有道路以及维护它们的每个月的成本 *accms*。当然，即使路线不像以前那么长，也需要在现有的道路上保证所有村庄之间可以互连。首席长老想告诉长老会，他们每月可以花费最少的金额来维持连接所有村庄的道路。在上面的地图中，村庄从 A 标记到 I。右边的地图显示了最便宜的道路，即每月 216 个 *aacms*。你的任务是编写一个可以解决这些问题的程序。

提交网址：

<http://t.cn/AiW33P91>

11.4 最短路径

讨论最小生成树后，下面学习图论中的另一个经典问题——最短路径问题。最短路径问题就是求图 $G(V, E)$ 中某两个特定顶点间最短的路径长度。所谓图上的路径，是指从图中一个起始顶点到一个终止顶点途中经过的所有顶点序列，路径的长度即所经过的边权和，如图 11.12 所示。



从结点 B 到结点 C 的最短路径为 $\{B, A, C\}$ ，长度为 5

图 11.12 最短路径及其长度

最短路径问题在实际中应用广泛。例如，求某两个城市间的最短行车路线长度。要求解这类问题，就要根据图的特点和问题的特征选择不同的算法。

本节介绍求最短路径的一个经典算法——Dijkstra 算法，它由荷兰图灵奖获得者、计算机科学家 Dijkstra 于 1959 年提出。该算法能够有效地计算出某个特定顶点（称为源点），到其余所有顶点的最短路径，即它能够很好地解决单源最短路径问题。

Dijkstra 算法在运行过程中将顶点集合 V 分成两个集合 S 和 T 。

(1) S ：已确定的顶点集合，初始只含源点 s 。

(2) $T = V - S$ ：尚未确定的顶点集合。

算法反复从集合 T 中选择当前到源点 s 最近的顶点 u ，将 u 加入集合 S ，然后对所有从 u 发出的边进行松弛操作。

由于 Dijkstra 算法总是选择集合 T 中离最近的顶点加入集合 S ，所以说该算法使用的是贪心策略。在关于贪心策略的那一章中提到过，贪心策略并不总是能够得到最优的结果，但由于该问题具备无后效性，故使用贪心策略的 Dijkstra 算法能够保证最优结果。

由于每次都要从 T 中选择最近的顶点 u ，遍历 T 中所有顶点的时间复杂度为 $O(V)$ ，在之后对顶点 u 进行 $|V|$ 次松弛操作的时间复杂度为 $O(V)$ 。此外，在松弛过程中获得顶点距离的时间复杂度为 $O(1)$ ，更新操作最多执行 $|E|$ 次。因此，在不进行优化的情况下，Dijkstra 算法的时间复杂度为 $O(V^2 + E) = O(V^2)$ 。

然而，从 T 中选择最近的顶点 u 时，其实并不需要遍历集合 T 。可以维护一个优先队列，将集合 T 中的顶点到源点 s 的距离，作为这些点的优先级，距离越低，优先级越高。那么每次只需从优先队列中取出队首元素，即可获得当前离源点 s 最近的顶点，这样就可将选择最近顶点的时间复杂度从 $O(V)$ 降至 $O(\log V)$ ，这样的操作要进行 $|E|$ 次。建立优先队列的时间复杂度为 $O(V)$ 。因此，Dijkstra 算法的时间复杂度从 $O(V^2)$ 降至 $O((E + V) \log V) = O(E \log V)$ 。之前的讨论可以知道 $|E|$ 和 V^2 具有相同的数量级，也就是说 $O(E \log V) = O(V^2 \log V)$ 是要大于 $O(V^2)$ 的，那么为什么还说这样是优化呢？其实是因为绝大部分情况下遇到的图都是稀疏图，即边的条数 $|E|$ 远小于 V^2 。绝大多数稀疏图中 $|E|$ 是和 V 同数量级的，因此用优先队列进行优化可以大大降低算法的时间复杂度。

例题 11.6 畅通工程续（浙江大学复试上机题）

题目描述：

某省自从实行了很多年的畅通工程计划后，终于修建了很多路。不过路多了也不好，每次要从一个城镇到另一个城镇时，都有许多种道路方案可以选择，而某些方案要比另一些方案行走的距离短很多。这让行人很困扰。

现在，已知起点和终点，请你计算出要从起点到终点，最短需要行走多少距离。

输入：

本题包含多组数据，请处理到文件结束。

每组数据第一行包含两个正整数 N 和 M ($0 < N < 200, 0 < M < 1000$)，分别代表现有城镇的数目和已修建的道路的数目。城镇分别以 $0 \sim N-1$ 编号。

接下来是 M 行道路信息。每一行有三个整数 A, B, X ($0 \leq A, B < N, A \neq B, 0 < X < 10000$)，表示城镇 A 和城镇 B 之间有一条长度为 X 的双向道路。

再接下来的下一行有两个整数 S, T ($0 \leq S, T < N$)，分别代表起点和终点。

输出：

对于每组数据，请在—行中输出最短需要行走的距离。若不存在从 S 到 T 的路线，则输出-1。

样例输入：

```
3 3
0 1 1
0 2 3
1 2 1
0 2
3 1
0 1 1
1 2
```

样例输出：

```
2
-1
```

【分析】

本题是一道经典的求单源最短路径问题，只需将 s 作为源点，利用 Dijkstra 算法便能够轻松求解。

代码 11.6

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <cstring>
#include <queue>
#include <limits>

using namespace std;
```

```

const int MAXN = 200;
const int INF = INT_MAX;           //无穷设为很大的数

struct Edge {
    int to;                         //终点
    int length;                     //长度
    Edge(int t, int l): to(t), length(l) {}
};

struct Point {
    int number;                     //点的编号
    int distance;                   //源点到该点距离
    Point(int n, int d): number(n), distance(d) {}
    bool operator< (const Point& p) const {
        return distance > p.distance; //距离小的优先级高
    }
};

vector<Edge> graph[MAXN];           //邻接表实现的图
int dis[MAXN];                     //源点到各点距离

void Dijkstra(int s) {
    priority_queue<Point> myPriorityQueue;
    dis[s] = 0;
    myPriorityQueue.push(Point(s, dis[s]));
    while (!myPriorityQueue.empty()) {
        int u = myPriorityQueue.top().number; //离源点最近的点
        myPriorityQueue.pop();
        for (int i = 0; i < graph[u].size(); ++i) {
            int v = graph[u][i].to;
            int d = graph[u][i].length;
            if (dis[v] > dis[u] + d) {
                dis[v] = dis[u] + d;
                myPriorityQueue.push(Point(v, dis[v]));
            }
        }
    }
    return ;
}

```

```

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        memset(graph, 0, sizeof(graph));    //图初始化
        fill(dis, dis + n, INF);            //距离初始化为无穷
        while (m--) {
            int from, to, length;
            scanf("%d%d%d", &from, &to, &length);
            graph[from].push_back(Edge(to, length));
            graph[to].push_back(Edge(from, length));
        }
        int s, t;                            //起点与终点
        scanf("%d%d", &s, &t);
        Dijkstra(s);
        if (dis[t] == INF) {                  //终点不可达
            dis[t] = -1;
        }
        printf("%d\n", dis[t]);
    }
    return 0;
}

```

例题 11.7 最短路径问题（浙江大学复试上机题）

题目描述：

给你 n 个点， m 条无向边，每条边都有长度 d 和花费 p ，给你起点 s 和终点 t ，要求输出起点到终点的最短距离及其花费。若最短距离有多条路线，则输出花费最少的。

输入：

输入 n, m ，点的编号是 $1 \sim n$ ，然后是 m 行，每行 4 个数 a, b, d, p ，表示 a 和 b 之间有一条边，且其长度为 d ，花费为 p 。最后一行是两个数 s, t ，即起点 s 和终点 t 。 n 和 m 为 0 时输入结束（ $1 < n \leq 1000, 0 < m < 100000, s! = t$ ）。

输出：

输出一行有两个数，即最短距离及其花费。

样例输入：

```

3 2
1 2 5 6
2 3 4 5
1 3
0 0

```

样例输出：

```

9 11

```

提交网址:

<http://t.cn/AilPbME2>

【分析】

在本题中，不仅要求起点到终点的最短距离，还需要在有多条最短路径时，选取花费最少的那一条。要求解这个问题，只需更改 Dijkstra 算法中的松弛操作：新的路径若在距离上优于之前的路径，或者新的路径在距离上等于之前的路径，但花费要优于之前的路径，则更新路径和花费。

代码 11.7

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <cstring>
#include <queue>
#include <climits>

using namespace std;

const int MAXN = 1001;
const int INF = INT_MAX;           //无穷设为很大的数

struct Edge {
    int to;                        //终点
    int length;                    //长度
    int price;                     //花费
    Edge(int t, int l, int p): to(t), length(l), price(p) {}
};

struct Point {
    int number;                   //点的编号
    int distance;                 //源点到该点距离
    Point(int n, int d): number(n), distance(d) {}
    bool operator< (const Point& p) const {
        return distance > p.distance; //距离小的优先级高
    }
};

vector<Edge> graph[MAXN];          //邻接表实现的图
int dis[MAXN];                   //源点到各点距离
```

```

int cost[MAXN]; //记录花费

void Dijkstra(int s) {
    priority_queue<Point> myPriorityQueue;
    dis[s] = 0;
    cost[s] = 0;
    myPriorityQueue.push(Point(s, dis[s]));
    while (!myPriorityQueue.empty()) {
        int u = myPriorityQueue.top().number; //离源点最近的点
        myPriorityQueue.pop();
        for (int i = 0; i < graph[u].size(); ++i) {
            int v = graph[u][i].to;
            int l = graph[u][i].length;
            int p = graph[u][i].price;
            if ((dis[v] == dis[u] + l && cost[v] > cost[u] + p) ||
                dis[v] > dis[u] + l) { //注意松弛操作的条件变换
                dis[v] = dis[u] + l;
                cost[v] = cost[u] + p;
                myPriorityQueue.push(Point(v, dis[v]));
            }
        }
    }
    return ;
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        if (n == 0 && m == 0) {
            break;
        }
        memset(graph, 0, sizeof(graph)); //图初始化
        fill(dis, dis + n + 1, INF); //距离初始化为无穷
        fill(cost, cost + n + 1, INF); //花费初始化为无穷
        while (m--) {
            int from, to, length, price;
            scanf("%d%d%d%d", &from, &to, &length, &price);
            graph[from].push_back(Edge(to, length, price));
            graph[to].push_back(Edge(from, length, price));
        }
    }
}

```

```

    }
    int s, t;           //起点与终点
    scanf("%d%d", &s, &t);
    Dijkstra(s);
    printf("%d %d\n", dis[t], cost[t]);
}
return 0;
}

```

最后要提醒读者，在使用 Dijkstra 算法求解单源最短路径问题时，图一定不能是负权图，即不能是图上有边的权值为负数的图。机试中考查负权图的可能性不大；若真出现了边的权值为负数的情况，则不能使用 Dijkstra 对其求解，因为 Dijkstra 算法原理在存在负权值边的图上不成立，这时就需要使用 Bellman-Ford 算法，有兴趣的读者可以自行查阅相关资料（这个知识点在机试中考查的概率不大）。

习题 11.6 最短路径（上海交通大学复试上机题）

有 N 个城市，标号从 0 到 $N-1$ ；有 M 条道路，第 K 条道路（ K 从 0 开始）的长度为 2^K 。求编号为 0 的城市到其他城市的最短距离。

提交网址：

<http://t.cn/AilPKHTx>

习题 11.7 I Wanna Go Home（北京大学复试上机题）

【题目大意】国家现在正面临着可怕的内战——国内的城市分为两派，分别支持不同的领导者。作为一个商人，M 先生不关注政治，但他实际上知道目前国内严峻的形势，你的任务是帮助他尽快回家。“为了安全起见，”M 先生说：“你的路线最多应包含一条连接两个不同营地城市的道路。”你能告诉 M 先生至少需要多长时间才能到达他的家吗？

提交网址：

<http://t.cn/AilPCAuL>

11.5 拓扑排序

本节讨论图论中的另一个经典问题——拓扑排序。设有一个有向无环图（Directed Acyclic Graph, DAG），图中有多条有向边（ U, V ）。将图中所有顶点排成一个线性序列，使得在该序列中顶点 U 都排列在顶点 V 之前。满足该要求的顶点序列，被称为满足拓扑序列。求这个序列的过程，被称为拓扑排序，如图 11.13 所示。

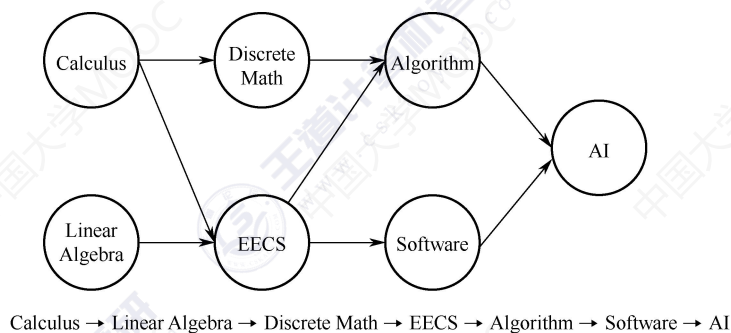


图 11.13 拓扑排序

了解拓扑序列的定义后,就可以知道前文中要将拓扑排序限定在一个有向无环图上的原因。若图无向,则边的两个顶点等价,不存在先后关系;若图有向,但存在一个环路,则该环中所有顶点无法判定先后关系。

对于一个 DAG 图来说,求其拓扑排序的方法很多,下面介绍一种最常见的方法:

- ① 从 DAG 图中选择入度为 0 的顶点,并输出。
- ② 从图中删除该入度为 0 的顶点及所有以它为起点的边。
- ③ 重复 (1) 和 (2) 直到当前图为空,或者图不存在入度为 0 的顶点。前者输出的序列就是拓扑序列;后者说明图中有环,不存在拓扑序列。

例题 11.8 Legal or Not

题目描述:

ACM-DIY is a large QQ group where many excellent acmers get together. It is so harmonious that just like a big family. Every day, many “holy cows” like HH, hh, AC, ZT, lcc, BF, Qinz and so on chat on-line to exchange their ideas. When someone has questions, many warm-hearted cows like Lost will come to help. Then the one being helped will call Lost “master”, and Lost will have a nice “prentice”. By and by, there are many pairs of “master and prentice”. But then problem occurs: there are too many masters and too many prentices, how can we know whether it is legal or not?

We all know a master can have many prentices and a prentice may have a lot of masters too, it's legal. Nevertheless, some cows are not so honest, they hold illegal relationship. Take HH and 3xian for instant, HH is 3xian's master and, at the same time, 3xian is HH's master, which is quite illegal! To avoid this, please help us to judge whether their relationship is legal or not.

Please note that the “master and prentice” relation is transitive. It means that if A is B's master and B is C's master, then A is C's master.

输入:

The input consists of several test cases. For each case, the first line contains two integers, N (members to be tested) and M (relationships to be tested) ($2 \leq N, M \leq 100$). Then M lines follow, each contains a pair of (x, y) which means x is y 's master and y is x 's prentice.

The input is terminated by $N = 0$.

TO MAKE IT SIMPLE, we give every one a number $(0, 1, 2, \dots, N - 1)$. We use their numbers instead of their names.

输出:

For each test case, print in one line the judgement of the messy relationship.

If it is legal, output “YES”, otherwise “NO”.

样例输入:

```
3 2
0 1
1 2
2 2
0 1
1 0
0 0
```

样例输出:

```
YES
NO
```

【题目大意】

ACM-DIY 是一个庞大的 QQ 群，许多优秀的 ACMER 聚集在一起。它就像大家庭一样和谐。每天有许多“神牛”如 HH, hh, AC, ZT, lcc, BF, Qinz 等通过在线聊天，交流他们的想法。当有人有疑问时，很多像 Lost 这样热心的大牛会来帮忙。然后被帮助的人将称呼 Lost 为“师傅”，而 Lost 将会有有一个“徒弟”。久而久之，就有了许多对“师傅与徒弟”。但问题出现了：出现的这么多师傅和徒弟，我们怎么知道他们之间的关系是否合法？

我们都知道一个师傅可以有許多徒弟，并且徒弟也可能有很多师傅，这是合法的。尽管如此，有些大牛并不那么诚实，他们保持着非法的关系。以 HH 和 3xian 为例子，HH 是 3xian 的师傅，同时 3xian 是 HH 的师傅，这是非法的！为避免这种情况，请帮助我们判断他们的关系

是否合法。

注意，“师傅与徒弟”关系是可传递的。这意味着如果 A 是 B 的师傅，B 是 C 的师傅，那么 A 就是 C 的师傅。

【分析】

若将群中的所有人都抽象成图上的顶点，将所有的师徒关系都抽象成有向边，这个实际问题就转化为判断图上是否有环，即判断该图是否为有向无环图。无论何时，当需要判断某个图是否是向无环图时，都应立刻联想到求拓扑排序。若一个图存在符合拓扑次序的顶点序列，则该图为有向无环图；反之，该图为非有向无环图。

为了保存在拓扑排序过程中不断出现的入度为 0 的顶点，可以使用一个队列来进行存储。每当出现一个新的入度为 0 的顶点，就将其放入队列；若需要找到一个入度为 0 的顶点，则可从队头取出。值得一提的是，这里使用队列仅仅是为了保存入度为 0 的顶点，而与队列先进先出的性质无关。若读者愿意，也可以使用栈来保存。

代码 11.8

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <queue>

using namespace std;

const int MAXN = 500;

vector<int> graph[MAXN];
int inDegree[MAXN];           //入度

bool TopologicalSort(int n) {
    queue<int> node;
    for (int i = 0; i < n; ++i) {
        if (inDegree[i] == 0) {
            node.push(i);
        }
    }
    int number = 0;           //拓扑序列顶点个数
    while (!node.empty()) {
        int u = node.front();
        node.pop();
        number++;             //拓扑序列顶点加 1
        for (int i = 0; i < graph[u].size(); ++i) {
            int v = graph[u][i];
```

```

        inDegree[v]--; //后继顶点入度减 1
        if (inDegree[v] == 0) {
            node.push(v);
        }
    }
}
return n == number; //判断能否产生拓扑排序
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        if (n == 0 && m == 0) {
            break;
        }
        memset(graph, 0, sizeof(graph)); //初始化图
        memset(inDegree, 0, sizeof(inDegree)); //初始化入度
        while (m--) {
            int from, to;
            scanf("%d%d", &from, &to);
            graph[from].push_back(to);
            inDegree[to]++;
        }
        if (TopologicalSort(n)) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}

```

例题 11.9 确定比赛名次

题目描述:

有 N 个比赛队 ($1 \leq N \leq 500$), 依次以编号 $1, 2, 3, \dots, N$ 进行比赛, 比赛结束后, 裁判委员会要将所有参赛队伍从前往后依次排名, 但现在裁判委员会不能直接获得每个队的比赛成绩, 只知道每场比赛的结果, 即 P_1 赢 P_2 , 用 P_1, P_2 表示, 排名时 P_1 在 P_2 之前。现在请你编写程序确定排名。

输入:

输入有若干组, 每组中的第一行为两个数 N ($1 \leq N \leq 500$) 和 M ; 其中 N 表示队伍的个数, M 表示接着有 M 行的输入数据。接下来的 M 行数据中, 每行也有两个整数 P_1, P_2 , 表示 P_1 队赢了 P_2 队。

输出:

给出一个符合要求的排名。输出时队伍号之间有空格, 最后一名后面没有空格。

其他说明: 符合条件的排名可能不是唯一的, 此时要求输出时编号小的队伍在前; 输入数据保证是正确的, 即输入数据确保一定能有一个符合要求的排名。

样例输入:

```
4 3
1 2
2 3
4 3
```

样例输出:

```
1 2 4 3
```

【分析】

本题不再是判断图是否为有向无环图, 而是明确告知给定的就是有向无环图, 并要求输出拓扑序列, 并在拓扑序列不唯一时, 输出编号小的在前的序列。

因此, 对于本题, 并不需要记录拓扑序列的顶点个数来判断图是否为有向无环图。而需要将整个拓扑序列记录下来, 这时应想到可以用向量进行线性的动态记录。由于需要输出编号小的在前的序列, 因此这时不能再队列或栈来存储入度为 0 的顶点。可以采用优先队列进行存储, 这样就能够保证编号小的顶点先输出。

代码 11.9

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <queue>

using namespace std;

const int MAXN = 501;

vector<int> graph[MAXN];
int inDegree[MAXN];           //入度

vector<int> TopologicalSort(int n) {
    vector<int> topology;      //拓扑序列
    priority_queue<int, vector<int>, greater<int>> > node;
    for (int i = 1; i <= n; ++i) {
```

```

        if (inDegree[i] == 0) {
            node.push(i);
        }
    }
    while (!node.empty()) {
        int u = node.top();
        node.pop();
        topology.push_back(u);           //加入拓扑序列
        for (int i = 0; i < graph[u].size(); ++i) {
            int v = graph[u][i];
            inDegree[v]--;
            //后继顶点入度减 1
            if (inDegree[v] == 0) {
                node.push(v);
            }
        }
    }
    return topology;
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        memset(graph, 0, sizeof(graph));
        memset(inDegree, 0, sizeof(inDegree));
        while (m--) {
            int from, to;
            scanf("%d%d", &from, &to);
            graph[from].push_back(to);
            inDegree[to]++;
        }
        vector<int> answer = TopologicalSort(n);
        for (int i = 0; i < answer.size(); ++i) {
            if (i == 0) {
                printf("%d", answer[i]);
            } else {
                printf(" %d", answer[i]);
            }
        }
        printf("\n");
    }
}

```



```
}  
return 0;  
}
```

11.6 关键路径

本节讨论最后一个经典的图论问题——关键路径。在介绍关键路径之前，必须先介绍 AOE 网。在日常生活中，人们常用有向图来描述和分析工程的计划与实施过程，工程常分为多个活动（Activity），在带权有向图中，若以顶点表示事件，以有向边表示活动，以边上的权值表示该活动持续的时间，则这样的图称为 AOE 网（Activity On Edge NetWork）。注意，这和之前遇到有向图有区别，之前的有向图都是用顶点代表活动的，而 AOE 网是用边代表活动的。

AOE 网上工程的开始顶点称为源点，如图 11.14 中的 V_1 点，该点的入度为 0；工程结束顶点称为汇点，如图 11.14 中的 V_6 点，该点的出度为 0。

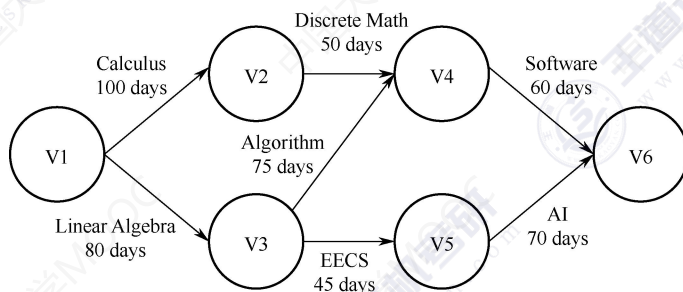


图 11.14 关键路径

在 AOE 网中，有些活动是可以并行进行的，从源点到汇点的有向路径可能有多条，并且这些路径的长度可能不同。完成不同路径上的活动所需的时间虽然不同，但只有所有路径上的活动都完成了，整个工程才能算结束。因此，从源点到汇点的所有路径中，具有最大路径长度的路径称为关键路径。把关键路径上的活动称为关键活动。

在代表工程的 AOE 网上，每个活动都有最早开始时间，即该活动的前序活动都已完成，可以进行该活动的时间；相应地，每个活动都有最晚开始时间，即该活动的后序活动要按时完成，该活动必须开始的时间。就好比 7 月一放暑假，便可以开始写暑假作业，这就是最早开始时间。如果写暑假作业要 1 个月，而 9 月开学，要在开学前完成暑假作业，必须从 8 月开始写暑假作业，这就是最晚开始时间。

对于工程中非重要的活动，可以允许些许的拖延，即最早开始时间和最晚开始时间不同，可以拖到最晚开始时间再做，因为即使拖到最晚开始时间也不会影响整个工程的完成时间。而对于工程中重要的活动却是不允许拖延的，即最早开始时间和最晚开始时间相同，因为关键活动的拖延会影响整个工程的完成时间。

这样一来，就将求关键路径转换成求每个活动的最早开始时间和最晚开始时间。之后判断二者是否相等，便可以获得关键路径上的关键活动。要如何求最早开始时间和最晚开始时间呢？

- ① 对于一个活动而言，其全部先序活动的最晚完成时间便是该活动的最早开始时间。
- ② 对于一个活动而言，其全部后序活动的最早开始时间减去该活动需要花费的时间，便是该活动的最晚开始时间。

这时出现了一个新问题：要计算一个活动的最早开始时间和最晚开始时间，就需要知道它的前序活动和后序活动，那么如何确定活动的先后次序呢？这时就要用到上一节介绍的拓扑排

序。

例题 11.10 Instructions Arrangement

题目描述：

Ali has taken the Computer Organization and Architecture course this term. He learned that there may be dependence between instructions, like WAR (write after read), WAW, RAW.

If the distance between two instructions is less than the Safe Distance, it will result in hazard, which may cause wrong result. So we need to design special circuit to eliminate hazard. However the most simple way to solve this problem is to add bubbles (useless operation), which means wasting time to ensure that the distance between two instructions is not smaller than the Safe Distance.

The definition of the distance between two instructions is the difference between their beginning times. Now we have many instructions, and we know the dependent relations and Safe Distances between instructions. We also have a very strong CPU with infinite number of cores, so you can run as many instructions as you want simultaneously, and the CPU is so fast that it just cost 1ns to finish any instruction.

Your job is to rearrange the instructions so that the CPU can finish all the instructions using minimum time.

输入：

The input consists several test cases.

The first line has two integers N, M ($N \leq 1000, M \leq 10000$), means that there are N instructions and M dependent relations.

The following M lines, each contains three integers X, Y, Z , means the Safe Distance between X and Y is Z , and Y should run after X . The instructions are numbered from 0 to $N - 1$.

输出：

Print one integer, the minimum time the CPU needs to run.

样例输入：

5 2

1 2 1

3 4 1

样例输出：

2

提示：

In the 1st ns, instruction 0, 1 and 3 are executed;

In the 2nd ns, instruction 2 and 4 are executed.

So the answer should be 2.

【题目大意】

阿里本学期选修了“计算机组成原理”这门课程。他了解到指令之间可能存在依赖关系，如 WAR（写入后读取）、WAW 和 RAW。

如果两个指令之间的距离小于安全距离，那么会导致危险，这可能会导致错误的结果。因此，我们需要设计特殊的电路来消除危险。然而，解决此问题的最简方法是添加空指令（无用指令），即增加无用操作以确保两条指令之间的距离不小于安全距离。

两条指令之间的距离的定义是它们的开始时间之差。

现在有很多条指令，我们知道指令之间的依赖关系和安全距离。我们还有一个非常强

大的 CPU，具有无限数量的内核，因此你可以根据需要同时运行多个指令，并且 CPU 速度非常快，只需花费 1ns 即可完成任何指令。

你的工作是重新排列指令，以便 CPU 可以使用最短的时间完成所有指令。

【分析】

若将指令的先后依赖关系抽象成有向边，则这个实际问题就转化为判断图上最长路径的长度，即关键路径长度。可以根据拓扑序列逐一求出每个活动的最早开始时间，再根据拓扑序列的逆序列求出每个活动的最晚开始时间。所有活动中最早开始时间和最晚开始时间相同的活动为关键活动，而所有活动中最早开始时间的最大值便是关键路径的长度。

值得注意的是，题目中指出每个指令的执行时间为 1ns，也就是说，所有源点的最早开始时间应该初始化为 1，而非通常的 0。

代码 11.10

```
#include <iostream>
#include <cstdio>
#include <queue>
#include <vector>
#include <cstring>
#include <climits>

using namespace std;

const int MAXN = 1001;
const int INF = INT_MAX;

struct Edge {
    int to; // 终点
    int length; // 距离
    Edge(int t, int l): to(t), length(l) {}
};

vector<Edge> graph[MAXN];
int earliest[MAXN]; // 最早开始时间
int latest[MAXN]; // 最晚开始时间
int inDegree[MAXN]; // 入度

void CriticalPath(int n) {
    vector<int> topology; // 拓扑序列
    queue<int> node;
    for (int i = 0; i < n; ++i) {
        if (inDegree[i] == 0) {
```

```

        node.push(i);
        earliest[i] = 1;           //初始化为 1
    }
}
while (!node.empty()) {
    int u = node.front();
    topology.push_back(u);
    node.pop();
    for (int i = 0; i < graph[u].size(); ++i) {
        int v = graph[u][i].to;
        int l = graph[u][i].length;
        earliest[v] = max(earliest[v], earliest[u] + l);
        inDegree[v]--;
        if (inDegree[v] == 0) {
            node.push(v);
        }
    }
}
for (int i = topology.size() - 1; i >= 0; --i) {
    int u = topology[i];
    if (graph[u].size() == 0) {
        latest[u] = earliest[u];           //汇点的最晚开始时间初始化
    } else {
        latest[u] = INF;                   //非汇点的最晚开始时间初始化
    }
    for (int j = 0; j < graph[u].size(); ++j) {
        int v = graph[u][j].to;
        int l = graph[u][j].length;
        latest[u] = min(latest[u], latest[v] - l);
    }
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        memset(graph, 0, sizeof(graph));
        memset(earliest, 0, sizeof(earliest));
        memset(latest, 0, sizeof(latest));
    }
}

```



```

memset(inDegree, 0, sizeof(inDegree));
while (m--) {
    int from, to, length;
    scanf("%d%d%d", &from, &to, &length);
    graph[from].push_back(Edge(to, length));
    inDegree[to]++;
}
CriticalPath(n);
int answer = 0;
for (int i = 0; i < n; ++i) {
    answer = max(answer, earliest[i]);
}
printf("%d\n", answer);
}
return 0;
}

```

对于最早开始时间的初始化，如果题目没有特殊要求，那么所有结点都初始化为 0。对于最晚开始时间的初始化，如果题目没有特殊要求，那么汇点的最晚开始时间初始化为它的最早开始时间，其余结点初始化为 INF。

例题 11.11 p3 (清华大学复试上机题)

题目描述：

小 H 为了完成一篇论文，一共要完成 n 个实验。其中第 i 个实验需要 a_i 的时间去完成。小 H 可以同时进行若干实验，但存在一些实验，只有当它的若干前置实验完成时，才能开始进行该实验。同时我们认为小 H 在一个实验的前置实验都完成时，就能马上开始该实验。

为了让小 H 尽快完成论文，需要知道在最优的情况下，最后一个完成的实验什么时候完成。

小 H 还想知道，在保证最后一个实验尽快完成的情况下（即保证上一问的答案不变），他想知道每个实验最晚可以什么时候开始。

设第 i 个实验最早可能的开始时间为 f_i ，不影响最后一个实验完成时间的最晚开始时间为 g_i ，请

证明 $\prod_{i=0}^n (g_i - f_i + 1)$ 除以 $10^9 + 7$ 所得的余数能够保证题目有解。

输入：

从标准输入读入数据。

第一行输入两个整数 n, m 。

第二行输入 n 个正整数 $a_1, a_2, \dots, a_n$ ，描述完成每个实验所需要的时间。

接下来读入 m 行，每行读入两个整数 u, v ，表示编号为 u 的实验是编号为 v 的实验的前置实验。

对于所有的输入数据，都满足 $1 \leq n \leq 10^5$ ， $1 \leq m \leq 5 \times 10^5$ ， $1 \leq a_i \leq 10^6$ 。

输出：

输出到标准输出。

第一行输出一个整数，表示最晚完成的实验的时间。

第二行输出一个整数表示 $\prod_{i=0}^n (g_i - f_i + 1)$ 除以 $10^9 + 7$ 所得的余数。

样例输入:

```
7 5
11 20 17 10 11 17 17
5 4
6 1
7 3
2 4
2 1
```

样例输出:

```
34
7840
```

【分析】

本道题也是一道求关键路径的典型题目，要求出最早开始时间和最晚开始时间。值得注意的是，本题的数据会超出 *int* 的范围，故要用 *long long* 来处理。

代码 11.11

```
#include <iostream>
#include <cstdio>
#include <queue>
#include <vector>
#include <cstring>
#include <climits>

using namespace std;

const int MAXN = 1e5 + 7;
const int INF = INT_MAX;
const int MOD = 1e9 + 7;

vector<int> graph[MAXN];
int inDegree[MAXN];           //入度
long long earliest[MAXN];     //最早开始时间
long long latest[MAXN];       //最晚开始时间
long long time[MAXN];         //花费时间

long long CriticalPath(int n) {
    vector<int> topology;      //拓扑序列
    queue<int> node;
    for (int i = 1; i <= n; ++i) {
```

```

        if (inDegree[i] == 0) {
            node.push(i);
        }
    }
    long long totalTime = 0;           //总耗时
    while (!node.empty()) {
        int u = node.front();
        topology.push_back(u);
        node.pop();
        for (int i = 0; i < graph[u].size(); ++i) {
            int v = graph[u][i];
            earliest[v] = max(earliest[v], earliest[u] + time[u]);
            inDegree[v]--;
            if (inDegree[v] == 0) {
                node.push(v);
                totalTime = max(totalTime, earliest[v] + time[v]);
            }
        }
    }
    for (int i = topology.size() - 1; i >= 0; --i) {
        int u = topology[i];
        if (graph[u].size() == 0) {
            latest[u] = totalTime - time[u];
        } else {
            latest[u] = INF;
        }
        for (int j = 0; j < graph[u].size(); ++j) {
            int v = graph[u][j];
            latest[u] = min(latest[u], latest[v] - time[u]);
        }
    }
    return totalTime;
}

int main() {
    int n, m;
    while (scanf("%d%d", &n, &m) != EOF) {
        memset(graph, 0, sizeof(graph));
        memset(earliest, 0, sizeof(earliest));
    }
}

```

```

memset(latest, 0, sizeof(latest));
memset(inDegree, 0, sizeof(inDegree));
memset(time, 0, sizeof(time));
for (int i = 1; i <= n; ++i) {
    scanf("%lld", &time[i]);
}
while (m--) {
    int from, to;
    scanf("%d%d", &from, &to);
    graph[from].push_back(to);
    inDegree[to]++;
}
long long totalTime = CriticalPath(n);
long long answer = 1;
for (int i = 1; i <= n; ++i) {
    answer *= latest[i] - earliest[i] + 1;
    answer %= MOD;
}
printf("%lld\n%lld\n", totalTime, answer);
}
return 0;
}

```

小结

本章着重讨论了图论的相关问题、算法原理和编码。首先介绍了图的定义和在计算机中的表示形式，随后介绍了求解集合查询与合并问题的并查集，最后讨论了最小生成树、最短路径、拓扑排序和关键路径这几个最基本、最经典的图论问题。图论在机试中考查的难度较大，需要读者很好地掌握图论的基本方法和性质。

第 12 章 动态规划

本章介绍另一个非常重要的算法思想——动态规划。动态规划通常用于求解最优解问题，动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干子问题，先求解子问题，然后从这些子问题的解得到原问题的解。与分治法不同的是，适合于用动态规划求解的问题，经分解得到的子问题往往不是互相独立的。若用分治法来解这类问题，则分解得到的子问题数目太多，有些子问题会被重复计算多次。而动态规划的做法是将已解决子问题的答案保存下来，在需要子问题答案的时候便可直接获得，而不需要重复计算，这样就可以避免大量的重复计算，提高效率。

12.1 递推求解

首先来看一个经典的问题——斐波那契数列，第 8 章在讲解分治法时介绍过这个问题。斐波那契数列的递归求解如下。

```
int Fibonacci(int n) {  
    if (n == 1 || n == 0) {                //递归出口  
        return n;  
    } else {                                //递归调用  
        return Fibonacci(n - 1) + Fibonacci(n - 2);  
    }  
}
```

事实上，这一求解方式会产生很多不必要的运算，如计算 $Fibonacci(7)$ 时要计算 $Fibonacci(6)$ 和 $Fibonacci(5)$ ，而计算 $Fibonacci(6)$ 时要计算 $Fibonacci(5)$ 和 $Fibonacci(4)$ ，大家发现 $Fibonacci(5)$ 会被计算两次。其实，如果在计算完 $Fibonacci(5)$ 后将其结果存储起来，求 $Fibonacci(6)$ 时就不需要再求解一次 $Fibonacci(5)$ 的值，如图 12.1 所示。

为避免重复计算，可用一个数组 dp 来记录计算的中间结果，用 $dp[n]$ 表示 $Fibonacci(n)$ 。将 $dp[0]$ 和 $dp[1]$ 分别初始化为 $dp[0]=0$ 和 $dp[1]=1$ 后，按照递推公式 $dp[i] = dp[i-1] + dp[i-2]$ 便可获得斐波那契数列。

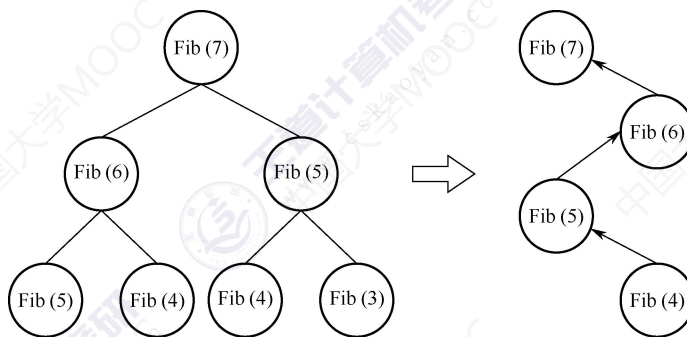


图 12.1 斐波那契数列的递归求解

例题 12.1 N 阶楼梯上楼问题（华中科技大学复试上机题）

题目描述：

N 阶楼梯上楼问题：一次可以走两阶或一阶，问有多少种上楼方式（要求采用非递归）。

输入：

输入包括一个整数 N ($1 \leq N < 90$)。

输出：

可能有多组测试数据，对于每组数据，输出当楼梯阶数是 N 时上楼方式的个数。

样例输入：

4

样例输出：

5

提交网址：

<http://t.cn/Aij9Fr3V>

【分析】

这个问题其实是斐波那契数列问题的一个变体。假设 $dp[n]$ 为 n 阶楼梯上楼方式的总数。首先，当数据规模较小时可直接得到答案，如 $dp[0]=0$, $dp[1]=1$ 。其次，当 n 大于 2 时，考虑每种上台阶方式的最后一步，由于只有两种行走的方法，因此它只可能是从 $n-1$ 阶走到 n 阶，或从 $n-2$ 阶走到 n 阶。分别考虑这两种走法，即将此时所有的上楼方式按照最后一步走法的不同分成两类，分别确定这两类的上楼方式的数目。经 $n-1$ 阶到达 n 阶，因为其最后一步是确定的，所以上楼方式的数目与原问题中到达 $n-1$ 阶的方式的数目相同，即 $dp[n-1]$ ；同理，经 $n-2$ 阶到达 n 阶，其上楼方式的数目与原问题中到达 $n-2$ 阶的方式的数目相同，即 $dp[n-2]$ 。这样，就确定了达到 n 阶楼梯的上楼方式的总数目为 $dp[n-1]$ 和 $dp[n-2]$ 的和，即 $dp[n]=dp[n-1]+dp[n-2]$ 。这便是数列的递推关系。由初始值 $dp[0]=0$, $dp[1]=1$ ，就能推出所有 $dp[n]$ 的值。

代码 12.1

```
#include <iostream>
#include <cstdio>
```



```
using namespace std;

const int MAXN = 91;

long long dp[MAXN];

int main() {
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i < MAXN; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    int n;
    while (scanf("%d", &n) != EOF) {
        printf("%lld\n", dp[n]);
    }
    return 0;
}
```

读者会发现，计算过程中已经记录了计算过的内容，需要再用到已经计算过的内容时，无须重复计算，可以直接利用之前的结果，这样就大幅度降低了无效的重复计算，把计算斐波那契数列问题的时间复杂度从指数级别的 $O(2^n)$ 降低到了线性级别的 $O(n)$ 。

习题 12.1 吃糖果（北京大学复试上机题）

小明的妈妈从外地出差回来，给他带了一盒好吃又精美的巧克力（盒内共有 N 块巧克力， $20 > N > 0$ ）。妈妈告诉小明每天可以吃一块或两块巧克力。假设小明每天都吃巧克力，问小明共有多少种不同的吃完巧克力的方案。例如，如果 $N = 1$ ，那么小明第 1 天就吃掉它，共有 1 种方案；如果 $N = 2$ ，那么小明可以第 1 天吃 1 块，第 2 天吃 1 块，也可以第 1 天吃 2 块，共有 2 种方案；如果 $N = 3$ ，那么小明第 1 天可以吃 1 块，剩 2 块，也可以第 1 天吃 2 块剩 1 块，所以小明共有 $2 + 1 = 3$ 种方案；如果 $N = 4$ ，那么小明可以第 1 天吃 1 块，剩 3 块，也可以第 1 天吃 2 块，剩 2 块，共有 $3 + 2 = 5$ 种方案。现在给定 N ，请你写程序求小明吃巧克力的方案数目。

提交网址：

<http://t.cn/AiQsVyKz>

12.2 最大连续子序列和

最大连续子序列和是动态规划中最经典的问题之一。这个问题讨论的是，在一个给定的序列 $\{A_1, A_2, \dots, A_n\}$ 中，找出一个连续子序列 $\{A_i, \dots, A_j\}$ ，使得这个连续子序列的和最大，输出这个最大的序列和。

对于这个问题，最先想到的肯定是枚举左右两个端点，以便获得所有的连续子序列，之后依次计算并比较每个子序列的序列和，这种方法的时间复杂度为 $O(n^3)$ 。

如果采用记录前缀和的方法预处理 $S_i = A_1 + \cdots + A_i$ ，便能在常数时间内获得给定子序列的序列和，不过这样仍然需要枚举左右两个端点，以便获得所有的连续子序列，这种方法的时间复杂度为 $O(n^2)$ 。

这两种方法都没有错误，但由于时间复杂度过高，当数据量较大时无法很好地完成任务。接下来介绍动态规划法，其时间复杂度与线性方法的 $O(n)$ 相同。

首先设置一个数组 $dp[]$ ，令 $dp[i]$ 表示以 $A[i]$ 作为末尾的连续序列的最大和。于是，通过设置这么一个数组，最大连续子序列和便是数组 dp 中的最大值。

由于 $dp[i]$ 是以 $A[i]$ 作为末尾的连续序列的最大和，因此只有两种情况：

- ① 最大和的连续序列只有一个元素，即 $A[i]$ 本身，也就是说 $dp[i] = A[i]$ 。
- ② 最大和的连续序列有多个元素，即从前面的某个 $A[j]$ 开始，一直到 $A[i]$ 结束，也就是 $dp[i] = A[j] + \cdots + A[i-1] + A[i]$ 。如何获得 $A[j] + \cdots + A[i-1]$ 呢？再回头看一下 dp 的定义， $dp[i]$ 表示以 $A[i]$ 作为末尾的连续序列的最大和，此时 $dp[i-1]$ 就是 $A[j] + A[j+1] + \cdots + A[i-1]$ 的值，即 $dp[i] = dp[i-1] + A[i]$ 。

由于只有这两种情况，于是得到状态转移方程 $dp[i] = \max\{A[i], dp[i-1] + A[i]\}$ 。

只需将 i 从小到大枚举并依次遍历，即可得到整个 dp 数组。接着输出该数组中的最大值，即求得最大连续子序列的和。

例题 12.2 最大序列和（清华大学复试上机题）

题目描述：

给出一个整数序列 S ，其中有 N 个数，定义其中一个非空连续子序列 T 中所有数的和为 T 的“序列和”。对于 S 的所有非空连续子序列 T ，求最大的序列和。变量条件： N 为正整数， $N \leq 1000000$ ，结果序列和在区间 $(-2^{63}, 2^{63} - 1)$ 内。

输入：

第一行为一个正整数 N ，第二行为 N 个整数，表示序列中的数。

输出：

输入可能包括多组数据，对于每组输入数据，仅输出一个数，表示最大序列和。

样例输入：

```
5
1 5 -3 2 4
6
1 -2 3 4 -10 6
4
-3 -1 -2 -5
```

样例输出:

```
9
7
-1
```

提交网址:

<http://t.cn/AiYSLQMU>

【分析】

本题是一道求最大连续子序列和的典型例题，只需按照上面分析的方法进行编码，便可以轻松地计算出其最大连续子序列和。但要注意的一点是，本题的数据较大，需要用到 `long long` 类型。

代码 12.2

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 1000000;

long long arr[MAXN];
long long dp[MAXN];

long long MaxSubsequence(int n) {
    long long maximum = 0;
    for (int i = 0; i < n; ++i) {
        if (i == 0) {
            dp[i] = arr[i];
        } else {
            dp[i] = max(arr[i], dp[i - 1] + arr[i]);
        }
        maximum = max(maximum, dp[i]);
    }
    return maximum;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
```

```

scanf("%lld", &arr[i]);
}
long long answer = MaxSubsequence(n);
printf("%lld\n", answer);
}
return 0;
}

```

了解如何利用动态规划来求最大连续子序列和后，即了解一维的情况后，下面了解如何处理这个问题的二维情况，即如何求一个二维矩阵的最大子矩阵和。

假设原二维矩阵中最大子矩阵所在的行是从 i 到 j ，那么只会出现下面这两种情况：

- ① 当 $i = j$ 时，求最大子矩阵和就转换成了求第 i 行元素的最大连续子序列和。
- ② 当 $i \neq j$ 时，把从第 i 行到第 j 行的所有行的元素加起来，得到只有一行的一维数组，

这个一维数组的最大连续最子序列和便是最大子矩阵和。

只需从小到大枚举并依次遍历 i 和 j ，接着输出所有子矩阵中的最大值，即求得最大子矩阵和。

例题 12.3 最大子矩阵（北京大学复试上机题）

题目描述：

已知矩阵的大小定义为矩阵中所有元素的和。给定一个矩阵，你的任务是找到最大的非空（大小至少是 1×1 ）子矩阵。

比如，如下 4×4 的矩阵

```

0 -2 -7 0
9 2 -6 2
-4 1 -4 1
-1 8 0 -2

```

的最大子矩阵是

```

9 2
-4 1
-1 8

```

这个子矩阵的大小是 15。

输入：

输入是一个 $N \times N$ 的矩阵。输入的第一行给出 N ($0 < N \leq 100$)。

再后面的若干行中，依次（首先从左到右给出第一行的 N 个整数，再从左到右给出第二行的 N 个整数……）给出矩阵中的 $2N$ 个整数，整数之间由空白字符分隔（空格或空行。已知矩阵中整数的范围都在 $[-127, 127]$ 内。

输出：

测试数据可能有多组，对于每组测试数据，输出最大子矩阵的大小。

样例输入：

```

4
0 -2 -7 0

```

```
9 2 -6 2
-4 1 -4 1
-1 8 0 -2
```

样例输出:

```
15
```

提交网址:

```
http://t.cn/AiO7FyDO
```

【分析】

本题是一道求最大子矩阵和的典型例题，只需按照上面分析的方法进行编码，便可计算出原始矩阵的最大子矩阵和。不过当 $i \neq j$ 时，不必将原始矩阵中的第 i 行到第 j 行的所有行的元素次累加起来得到一维数组，而事先用一个辅助二维矩阵记录原始矩阵从上到下加起来的累加矩阵，于是求从第 i 行到第 j 行的一维数组只需将辅助矩阵进行一行减法便可得到，而不需要逐行进行多次累加。

代码 12.3

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 100;

int matrix[MAXN][MAXN];           //原始矩阵
int total[MAXN][MAXN];             //辅助矩阵
int arr[MAXN];                     //一维数组
int dp[MAXN];

int MaxSubsequence(int n) {
    int maximum = 0;
    for (int i = 0; i < n; ++i) {
        if (i == 0) {
            dp[i] = arr[i];
        } else {
            dp[i] = max(arr[i], dp[i - 1] + arr[i]);
        }
        maximum = max(maximum, dp[i]);
    }
    return maximum;
}
```

```
int MaxSubmatrix(int n) {
    int maximal = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = i; j < n; ++j) {
            for (int k = 0; k < n; ++k) {          //获得一维数组
                if (i == 0) {
                    arr[k] = total[j][k];
                } else {
                    arr[k] = total[j][k] - total[i - 1][k];
                }
            }
            int current = MaxSubsequence(n);
            maximal = max(maximal, current);
        }
    }
    return maximal;
}

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                scanf("%d", &matrix[i][j]);
            }
        }
        for (int i = 0; i < n; ++i) {                //初始化辅助函数
            for (int j = 0; j < n; ++j) {
                if (i == 0) {
                    total[i][j] = matrix[i][j];
                } else {
                    total[i][j] = total[i - 1][j] + matrix[i][j];
                }
            }
        }
        int answer = MaxSubmatrix(n);
        printf("%d\n", answer);
    }
}
```

```
return 0;  
}
```

习题 12.2 最大连续子序列（浙江大学复试上机题）

给定其中有 K 个整数的序列 $\{N_1, N_2, \dots, N_K\}$ ，其任意连续子序列都可以表示为 $\{N_i, N_{i+1}, \dots, N_j\}$ ，其中 $1 \leq i \leq j \leq K$ 。最大连续子序列是所有连续子序列中元素和最大的一个，例如给定序列 $\{-2, 11, -4, 13, -5, -2\}$ ，其最大连续子序列为 $\{11, -4, 13\}$ ，最大和为 20。现在增加一个要求，即还需要输出该子序列的第一个和最后一个元素。

提交网址：

<http://t.cn/AiYoUkjP>

12.3 最长递增子序列

最长递增子序列 (Longest Increasing Subsequence, LIS) 是动态规划中最经典的问题之一, 该问题描述的是在一个已知序列 $\{A_1, A_2, \dots, A_n\}$ 中, 取出若干元素 (不必连续) 组成一个新的序列 $\{A_x, \dots, A_y\}$, 新序列中的各个数之间依旧保持原序列中的先后顺序, 此时称新序列 $\{A_x, \dots, A_y\}$ 为原序列的一个子序列。若对子序列中的任意下标 $x < y$ 有 $A_x < A_y$, 则称该子序列为原序列的一个递增子序列。最长递增子序列问题就是求给定序列的所有递增子序列中最长的那个子序列长度。

对于这个问题, 最先想到的肯定是枚举所有子序列, 逐一判断子序列是否为递增序列, 并在所有递增序列中求其最大值。对于有 n 个元素的序列, 可以通过选择是否需要某个元素来枚举出所有的子序列, 但这种方法的时间复杂度为 $O(2^n)$, 这样的时间复杂度是万万不能接受的。接下来介绍求解这个问题的动态规划方法, 其时间复杂度仅为 $O(n^2)$ 。

首先设置一个数组 $dp[]$, 令 $dp[i]$ 表示以 $A[i]$ 作为末尾的最长递增子序列的长度。于是, 通过设置这么一个数组, 最长递增子序列的长度便是数组 dp 中的最大值。

由于 $dp[i]$ 是以 $A[i]$ 作为末尾的最长递增子序列的长度, 因此只有两种情况:

- ① $A[i]$ 之前的元素都比 $A[i]$ 大, 即最长递增子序列只有 $A[i]$ 本身, 即 $dp[i] = 1$ 。
- ② $A[i]$ 之前的元素 $A[j]$ 比 $A[i]$ 小, 此时只需将 $A[i]$ 添加到以 $A[j]$ 作为末尾的最长递增子序列, 便能构成一个新的递增子序列。可是, 如何求出以 $A[j]$ 作为末尾的最长递增子序列的长度呢? 回头看一下 dp 的定义, $dp[i]$ 表示以 $A[i]$ 作为末尾的最长递增子序列的长度, 于是此时 $dp[j]$ 就是以 $A[j]$ 作为末尾的最长递增子序列的长度, 即这个新的子序列的长度为 $dp[i] = dp[j] + 1$ 。只需将 i 之前的元素逐一遍历, 便可获得以 $A[i]$ 作为末尾的最长递增子序列的长度 $dp[i]$ 。

从这两种情况可以得到状态转移方程 $dp[i] = \max\{1, dp[j] + 1 \mid j < i \ \&\& \ A_j < A_i\}$ 。

例题 12.4 拦截导弹 (北京大学复试上机题)

题目描述:

某国为了防御敌国的导弹袭击, 开发了一种导弹拦截系统。但这种导弹拦截系统有一个缺陷: 虽然它的第一发炮弹能够到达任意的高度, 但以后每发炮弹都不能高于前一发的高度。某天, 雷达捕捉到敌国的导弹来袭, 并观测到导弹依次飞来的高度, 请计算这套系统最多能拦截多少导弹。拦截来袭导弹时, 必须按来袭导弹袭击的时间顺序, 不允许先拦截后面的导弹, 再拦截前面的导弹。

输入:

每组输入有两行:

第一行, 输入雷达捕捉到的敌国导弹的数量 k ($k \leq 25$)。

第二行, 输入 k 个正整数, 表示 k 枚导弹的高度, 按来袭导弹的袭击时间顺序给出, 以空格分隔。

输出:

每组输出只有一行，包含一个整数，表示最多能拦截多少枚导弹。

样例输入：

```
8
300 207 155 300 299 170 158 65
```

样例输出：

```
6
```

提交网址：

<http://t.cn/AiYCeV3m>

【分析】

由题意不难看出，要求最多能够拦截多少枚导弹，即在按照袭击顺序排列的导弹高度中求其最长不增子序列。所谓不增子序列，即子序列中排在前面的数字不比排在后面的数字小。求最长不增子序列的原理与求最长递增子序列的原理完全一致，只是状态转移方程相应地发生了一些变化。

状态转移方程为 $dp[i] = \max\{1, dp[j] + 1 \mid j < i \ \&\& \ A_j \geq A_i\}$ 。

代码 12.4

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 25;

int height[MAXN];           //导弹高度
int dp[MAXN];

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            scanf("%d", &height[i]);
        }
        int answer = 0;
        for (int i = 0; i < n; ++i) {
            dp[i] = 1;           //初始化为 1
            for (int j = 0; j < i; ++j) {
                if (height[i] <= height[j]) {
                    dp[i] = max(dp[i], dp[j] + 1);
                }
            }
        }
        printf("%d\n", dp[n-1]);
    }
}
```

```

    }
    answer = max(answer, dp[i]);           //dp 数组的最大值
}
printf("%d\n", answer);
}
return 0;
}

```

由上例的分析过程可知，最长递增子序列问题的求解思想，不仅可以用来求解单纯的最长递增子序列问题，而且可以经过类比来求解其他大小关系确定的子序列问题。

例题 12.5 最大上升子序列和（北京大学复试上机题）

题目描述：

对于一个数的序列 b_i ，当 $b_1 < b_2 < \dots < b_s$ 时，我们称这个序列是上升的。对于给定的一个序列 $(a_1, a_2, \dots, a_n)$ ，我们可以得到一些上升的子序列 $(a_{i_1}, a_{i_2}, \dots, a_{i_k})$ ，其中 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ ，对于序列 $(1, 7, 3, 5, 9, 4, 8)$ ，有它的一些上升子序列，如 $(1, 7)$, $(3, 4, 8)$ 等。这些子序列中序列和最大为 18，为子序列 $(1, 3, 5, 9)$ 的和。你的任务是对于给定的序列，求出最大上升子序列和。

注意，最长的上升子序列不一定是最大的，比如序列 $(100, 1, 2, 3)$ 的最大上升子序列和为 100，而最长上升子序列为 $(1, 2, 3)$ 。

输入：

输入包含多组测试数据。

每组测试数据由两行组成。第一行是序列的长度 N ($1 \leq N \leq 1000$)。第二行给出序列中的 N 个整数，这些整数的取值范围都在 0 到 10000 之间（可能重复）。

输出：

对于每组测试数据，输出其最大上升子序列和。

样例输入：

```

7
1 7 3 5 9 4 8

```

样例输出：

```

18

```

提交网址：

<http://t.cn/AiYNAGD3>

【分析】

本题求的是最大上升子序列和，其原理与求最长递增子序列的原理完全一致，只是状态转移方程相应地发生了一些变化，原来 $dp[i]$ 是以 $A[i]$ 作为末尾的最长递增子序列的长度，现在将 $dp[i]$ 变成了以 $A[i]$ 作为末尾的最大上升子序列和。

对应的两种情况如下：

- ① $A[i]$ 之前的元素都比 $A[i]$ 大，即最大上升子序列和为 $A[i]$ 本身，即 $dp[i] = A[i]$ 。
- ② $A[i]$ 之前有元素 $A[j]$ 比 $A[i]$ 小，此时只需将 $A[i]$ 添加到以 $A[j]$ 作为末尾的递增子序列便能构成一个新的递增子序列，而这个新的子序列和为 $dp[i] = dp[j] + A[i]$ 。只需将 i 之

前的元素逐一遍历，便可获得以 $A[i]$ 作为末尾的最大上升子序列和 $dp[i]$ 。
状态转移方程为 $dp[i] = \max\{A[i], dp[j] + A[i] \mid j < i \ \&\& \ A_j < A_i\}$

代码 12.5

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 1000;

int arr[MAXN];
int dp[MAXN];

int main() {
    int n;
    while (scanf("%d", &n) != EOF) {
        for (int i = 0; i < n; ++i) {
            scanf("%d", &arr[i]);
        }
        int answer = 0;
        for (int i = 0; i < n; ++i) {
            dp[i] = arr[i]; //初始化为 arr[i]
            for (int j = 0; j < i; ++j) {
                if (arr[j] < arr[i]) {
                    dp[i] = max(dp[i], dp[j] + arr[i]);
                }
            }
            answer = max(answer, dp[i]); //dp 数组的最大值
        }
        printf("%d\n", answer);
    }
    return 0;
}
```

习题 12.3 合唱队形（北京大学复试上机题）

N 位同学站成一排，音乐老师要请其中的 $N - K$ 位同学出列，使得剩下的 K 位同学不交换位置就能排成合唱队形。合唱队形是指这样的一种队形：设 K 位同学从左到右依次编号为 $1, 2, \dots, K$ ，他们的身高分别为 $T_1, T_2, \dots, T_K$ ，则他们的身高满足 $T_1 < T_2 < \dots < T_i, T_i > T_{i+1} > \dots > T_K, 1 \leq i \leq K$ 。你的任务是，已知所有 N 位同学的身高，计算最少需要几位同学出列，才能使得剩下的同学排成

合唱队形。

提交网址：

<http://t.cn/AiYNyHPe>

12.4 最长公共子序列

本章介绍另一个经典的动态规划问题——最长公共子序列 (Longest Common Subsequence, LCS)。与上节中数字序列的子序列定义相同, 在字符串 S 中按照其先后顺序依次取出若干字符, 并将它们排列成一个新的字符串, 这个字符串就称为原字符串的子串。

最长公共子序列问题描述的是, 给定两个字符串 S_1 和 S_2 , 求一个最长公共子串, 即求字符串 S_3 , 它同时为 S_1 和 S_2 的子串, 且要求它的长度最长, 并确定这个长度。这个问题称为最长公共子序列问题。

对于这个问题, 最先想到的肯定是先枚举 S_1 和 S_2 的所有子序列, 之后逐一判断子序列之间是否相同。对于长度为 n 的字符串 S_1 和长度为 m 的字符串 S_2 , 可以通过选择是否需要某个字符来枚举出所有的子序列, 但这个方法的时间复杂度为 $O(2^{n+m})$, 这样的时间复杂度是万万不能接受的。接下来介绍求解这个问题的动态规划方法, 其时间复杂度只有 $O(nm)$ 。

首先设置一个二维数组 $dp[i][j]$, 令 $dp[i][j]$ 表示以 $S_1[i]$ 作为末尾和以 $S_2[j]$ 作为末尾的最长公共子序列的长度。因此, 通过设置这么一个二维数组, 最长公共子序列的长度便是数组 $dp[n][m]$ 的值。

$dp[i][j]$ 表示以 $S_1[i]$ 作为末尾和以 $S_2[j]$ 作为末尾的最长公共子序列的长度, 根据 $S_1[i]$ 和 $S_2[j]$ 的关系可分为两种情况:

- ① $S_1[i] = S_2[j]$, 即 S_1 中的第 i 个字符和 S_2 中的第 j 个字符相同, 此时必定存在一个最长公共子串以 $S_1[i]$ 和 $S_2[j]$ 结尾, 其他部分等价于 S_1 中前 $i-1$ 个字符和 S_2 中前 $j-1$ 个字符的最长公共子串, 于是这个子串的长度比 $dp[i-1][j-1]$ 多 1, 即 $dp[i][j] = dp[i-1][j-1] + 1$ 。
- ② $S_1[i] \neq S_2[j]$, 此时最长公共子串长度为 S_1 中前 $i-1$ 个字符和 S_2 中前 j 个字符的最长公共子串长度与 S_1 中前 i 个字符和 S_2 中前 $j-1$ 个字符的最长公共子串长度的较大者, 即在两种情况下得到的最长公共子串都不会因为其中一个字符串又增加了一个字符长度而发生改变, 也就是 $dp[i][j] = \max\{dp[i-1][j], dp[i][j-1]\}$ 。

从这两种情况可以得到状态转移方程:

$$\begin{aligned} dp[i][j] &= dp[i-1][j-1] + 1 & S_1[i] &= S_2[j] \\ dp[i][j] &= \max\{dp[i-1][j], dp[i][j-1]\} & S_1[i] &\neq S_2[j] \end{aligned}$$

而对于边界情况, 如果两个字符串中的其中一个为空串, 那么公共字符串的长度必定为 0, 于是可以轻松得到:

$$\begin{aligned} dp[i][0] &= 0 \quad (0 \leq i \leq n) \\ dp[0][j] &= 0 \quad (0 \leq j \leq m) \end{aligned}$$

由这样的状态转移, 只需依次遍历 i 和 j 便能求得各个 $dp[i][j]$ 的值, 其时间复杂度为 $O(nm)$ 。

最终 $dp[n][m]$ 中保存的值即为两个原始字符串的最长公共子序列长度。

例题 12.6 Common Subsequence

题目描述:

A subsequence of a given sequence is the given sequence with some elements (possibly none) left out. Given a sequence $X = (x_1, x_2, \dots, x_m)$ another sequence $Z = (z_1, z_2, \dots, z_k)$ is a subsequence of X if there exists a strictly increasing sequence $(i_1, i_2, \dots, i_k)$ of indices of X such that for all $j = 1, 2, \dots, k$, $x_{i_j} = z_j$. For example, $Z = (a, b, f, c)$ is a subsequence of $X = (a, b, c, f, b, c)$ with index sequence $(1, 2, 4, 6)$. Given two sequences X and Y the problem is to find the length of the maximum-length common subsequence of X and Y .

输入:

The program input is from a text file. Each data set in the file contains two strings representing the given sequences. The sequences are separated by any number of white spaces. The input data are correct.

输出:

For each set of data the program prints on the standard output the length of the maximum-length common subsequence from the beginning of a separate line.

样例输入:

```
abcfbc abfcab
programming contest
abcd mnp
```

样例输出:

```
4
2
0
```

【题目大意】

已知一个序列的子序列由该序列的元素组成（可为空）。给定序列 $X = (x_1, x_2, \dots, x_m)$ ，如果存在严格增加的指数序列 $(i_1, i_2, \dots, i_k)$ 使得对所有 X 的下标 $j = 1, 2, \dots, k$ 有 $x_{i_j} = z_j$ ，那么规定序列 $Z = (z_1, z_2, \dots, z_k)$ 是 X 的子序列。例如， $Z = (a, b, f, c)$ 是 $X = (a, b, c, f, b, c)$ 的子序列， Z 序列的元素在 X 序列中的下标为 $(1, 2, 4, 6)$ 。给定两个序列 X 和 Y ，问题是找到 X 和 Y 的最大长度公共子序列的长度。

【分析】

本题是一道最朴素的求解两个字符串的最长公共子串长度问题，只需按照上述分析方法进行编码便可解决。

但要注意的是，字符串最好从下标 1 而非 0 开始输入，因为这样处理有利于边界情况的分析和初始化。

代码 12.6

```
#include <iostream>
#include <cstdio>
#include <cstring>

using namespace std;

const int MAXN = 1001;

char s1[MAXN];
char s2[MAXN];
int dp[MAXN][MAXN];

int main() {
    while (scanf("%s%s", s1 + 1, s2 + 1) != EOF) { // 从下标 1 开始输入
        int n = strlen(s1 + 1);
        int m = strlen(s2 + 1);
        for (int i = 0; i <= n; ++i) {
            for (int j = 0; j <= m; ++j) {
                if (i == 0 || j == 0) { // 边界情况初始化
                    dp[i][j] = 0;
                    continue;
                }
                if (s1[i] == s2[j]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        printf("%d\n", dp[n][m]);
    }
    return 0;
}
```

习题 12.4 Coincidence (上海交通大学复试上机题)

【题目大意】找到两个字符串的最长公共子串。

提交网址:

<http://t.cn/AiY03R05>

12.5 背包问题

本节讨论动态规划问题中另一个十分常见并且在机试中重点考查的问题——背包问题。背包问题的变体繁多且复杂。根据考研机试的实际需要，本节主要讨论 0-1 背包、完全背包和多重背包三类背包问题。读者对背包问题的其他变体感兴趣时，可自行查阅相关资料。

1. 0-1 背包

0-1 背包问题描述的是，有 n 件物品，每件物品的重量为 $w[i]$ ，其价值为 $v[i]$ ，现在有个有容量为 m 的背包，如何选择物品使得装入背包物品的价值最大。

对于这个问题，最先想到的肯定是枚举所有物品的排列组合，再从中找出价值最大的那个组合。同样，可以通过选择是否需要某个物品来枚举出所有的排列组合，这个方法的时间复杂度为 $O(2^n)$ ，这样的时间复杂度是万万不能接受的。接下介绍求解这个问题的动态规划方法，其时间复杂度只有 $O(nm)$ 。

首先设置一个二维数组 $dp[i][j]$ ，令 $dp[i][j]$ 表示前 i 个物品装进容量为 j 的背包能获得的最大价值。通过设置这么一个二维数组，数组 $dp[n][m]$ 的值就是 0-1 背包问题的解。

只考虑第 i 件物品时，可将情况分为是否放入第 i 件物品两种：

- ① 对于容量为 j 的背包，如果不放入第 i 件物品，那么这个问题就转换成将前 $i-1$ 个物品放入容量为 j 的背包的问题，即 $dp[i][j] = dp[i-1][j]$ 。
- ② 对于容量为 j 的背包，如果放入第 i 件物品，那么当前背包的容量就变成了 $j - w[i]$ ，并得到这个物品的价值 $v[i]$ 。之后这个问题就转化成将前 $i-1$ 个物品放入容量为 $j - w[i]$ 的背包的问题，即 $dp[i][j] = dp[i-1][j - w[i]] + v[i]$ 。

而从这两种情况可以得到状态转移方程：

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j - w[i]] + v[i])$$

转移时要注意 $j - w[i]$ 的值是否为非负值，若为负则代表背包当前的容量无法放入第 i 件物品，不能进行转移。

而对于边界情况，如果装入 0 件物品，那么无论给定背包的容量有多大，能够获得的价值必定为 0。同样，如果背包的容量为 0，那么也无法装入任何物品，此时能够获得的价值也必定为 0。于是可以轻松地得到

$$\begin{aligned} dp[i][0] &= 0 \quad (0 \leq i \leq n) \\ dp[0][j] &= 0 \quad (0 \leq j \leq m) \end{aligned}$$

由这样的状态转移，只需依次遍历 i 和 j 便能求得各 $dp[i][j]$ 的值，其时间复杂度为 $O(nm)$ 。最终 $dp[n][m]$ 中保存的值即为 0-1 背包问题的解。

观察状态转移的特点，可以发现 $dp[i][j]$ 的转移仅与 $dp[i-1][j-w[i]]$ 和 $dp[i-1][j]$ 有关，即仅与二维数组中本行的上一行有关。根据这个特点，可以将原本的二维数组优化为一维数组，

并用如下方式完成状态转移：

$$dp[j] = \max(dp[j], dp[j-w[i]] + v[i])$$

上述 $dp[j-w[i]]$ 和 $dp[j]$ 与原始写法中的 $dp[i-1][j-w[i]]$ 和 $dp[i-1][j]$ 等值。为了保证状态正确转移，必须保证在每次更新中确定状态 $dp[j]$ 时， $dp[j-w[i]]$ 尚未被本次更新修改。这就需要在每次更新中，倒序地遍历所有 j 的值，因为只有这样才能保证在确定 $dp[j]$ 的值时， $dp[j-w[i]]$ 的值尚未被修改，从而完成正确的状态转移。

例题 12.7 点菜问题（北京大学复试上机题）

题目描述：

北大网络实验室经常有活动需要叫外卖，但是每次叫外卖的报销经费的总额最大为 C 元，有 N 种菜可以点，经过长时间的点菜，网络实验室对于每种菜 i 都有一个量化的评价分数（表示这个菜的可口程度） V_i ，每种菜的价格为 P_i ，问如何选择各种菜，使得在报销额度范围内能使点到的菜的总评价分数最大。

注意：由于需要营养多样化，每种菜只能点一次。

输入：

输入的第一行有两个整数 C ($1 \leq C \leq 1000$) 和 N ($1 \leq N \leq 100$)， C 代表总共能够报销的额度， N 代表能点的菜的数目。接下来的 N 行中，每行包括两个在 1 到 100 之间（包括 1 和 100）的整数，分别表示菜的价格和菜的评价分数。

输出：

输出只包括一行，这一行只包含一个整数，表示在报销额度范围内，所点的菜得到的最大评价分数。

样例输入：

```
90 4
20 25
30 20
40 50
10 18
40 2
25 30
10 8
```

样例输出：

```
95
38
```

提交网址：

<http://t.cn/AiYOrkXr>

【分析】

这个问题是典型的 0-1 背包问题，报销额度对应于背包的容量，每种菜的价格对应于商品的重量，而每种菜的评分对应于商品的价值。

代码 12.7

```

#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 1001;

int dp[MAXN];
int v[MAXN];           //物品价值
int w[MAXN];           //物品重量

int main() {
    int n, m;           //n 件物品, m 容量的背包
    scanf("%d%d", &m, &n);
    for (int i = 0; i < n; ++i) {
        scanf("%d%d", &w[i], &v[i]);
    }
    for (int i = 0; i <= m; ++i) {
        dp[i] = 0;       //初始化
    }
    for (int i = 0; i < n; ++i) {
        for (int j = m; j >= w[i]; --j) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    printf("%d\n", dp[m]);
    return 0;
}

```

习题 12.5 采药（北京大学复试上机题）

辰辰是个很有潜能、天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每株草药都需要一些时间，每株草药也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”如果你是辰辰，你能完成这个任务吗？

提交网址：

<http://t.cn/AiW33P91>

习题 12.6 最小邮票数（清华大学复试上机题）

有若干邮票，要求从中选取最少的邮票张数凑成一个给定的总值。

例如，有 1 分、3 分、3 分、3 分、4 分五张邮票，要求凑成 10 分；此时使用 3 张邮票：3 分、3 分、4 分即可。

提交网址：

<http://t.cn/AiYlwchD>

0-1 背包问题是最基本的背包问题，其他各类背包问题都是在此基础上演变而来。0-1 背包的特点是，每种物品至多只能选择一件，即在背包中该物品的数量只有 0 和 1 两种情况，这也是 0-1 背包名称的由来。

2. 完全背包

如果将 0-1 背包问题进行扩展，每种物品不只可以取一件，而是可以选择多件，这时该如何使得背包里的物品价值总和最大呢？这便得到完全背包问题：有 n 种物品，每种物品的重量为 $w[i]$ ，其价值为 $v[i]$ ，每种物品的数量均为无限个，现在有容量为 m 的背包，如何选择物品使得装入背包物品的价值最大？

同样，设置一个二维数组 $dp[i][j]$ ，令 $dp[i][j]$ 表示前 i 个物品装进容量为 j 的背包能够获得的最大价值。通过设置这么一个二维数组，数组 $dp[n][m]$ 的值就是完全背包问题的解。

和 0-1 背包一样，只考虑第 i 件物品时，可将情况分为是否放入第 i 件物品两种：

- ① 对于容量为 j 的背包，如果不放入第 i 件物品，那么这个问题和 0-1 背包一样转换成将前 $i-1$ 个物品放入容量为 j 的背包问题，即 $dp[i][j] = dp[i-1][j]$ 。
- ② 对于容量为 j 的背包，若放入第 i 件物品，则当前背包的容量就变成 $j - w[i]$ ，并得到这个物品的价值 $v[i]$ 。但由于第 i 件物品仍然可以取，所以并不是转移到 $dp[i-1][j - w[i]]$ ，而是转移到 $dp[i][j - w[i]]$ ，即 $dp[i][j] = dp[i][j - w[i]] + v[i]$ 。

而从这两种情况可以得到状态转移方程：

$$dp[i][j] = \max(dp[i-1][j], dp[i][j - w[i]] + v[i])$$

可以看到这个二维转移方程和 0-1 背包方程之间只有些许区别。而对于边界情况，和 0-1 背包问题一样，有

$$\begin{aligned} dp[i][0] &= 0 \quad (0 \leq i \leq n) \\ dp[0][j] &= 0 \quad (0 \leq j \leq m) \end{aligned}$$

由这样的状态转移，只需依次遍历 i 和 j 便能求得各个 $dp[i][j]$ 的值，其时间复杂度为 $O(nm)$ 。最终 $dp[n][m]$ 中保存的值即为完全背包问题的解。

观察状态转移的特点，可以发现 $dp[i][j]$ 的转移仅与 $dp[i][j - w[i]]$ 和 $dp[i-1][j]$ 有关，和 0-1 背包一样，即仅与二维数组中本行的上一行有关。根据这个特点，可将原本的二维数组优化为一维数组，并用如下方式完成状态转移：

$$dp[j] = \max(dp[j], dp[j - w[i]] + v[i])$$

可以发现这个方程和 0-1 背包的一维方程完全一样。为了保证状态正确转移，必须保证在每次更新中确定状态 $dp[j]$ 时， $dp[j - w[i]]$ 已经完成了本次的更新修改。这就需要在每次更新中，正序地遍历所有 j 的值，因为只有这样，才能保证在确定 $dp[j]$ 的值时， $dp[j - w[i]]$ 的值已被修改，从而完成正确的状态转移。也就是说，只需要将 0-1 背包的遍历过程从逆向变成正向，即可变成求完全背包的方式。

例题 12.8 Piggy-Bank

题目描述：

Before ACM can do anything, a budget must be prepared and the necessary financial support obtained. The main income for this action comes from Irreversibly Bound Money (IBM). The idea behind is simple. Whenever some ACM member has any small money, he takes all the coins and throws them into a piggy-bank. You know that this process is irreversible, the coins cannot be removed without breaking the pig. After a sufficiently long time, there should be enough cash in the

piggy-bank to pay everything that needs to be paid.

But there is a big problem with piggy-banks. It is not possible to determine how much money is inside. So we might break the pig into pieces only to find out that there is not enough money. Clearly, we want to avoid this unpleasant situation. The only possibility is to weigh the piggy-bank and try to guess how many coins are inside. Assume that we are able to determine the weight of the pig exactly and that we know the weights of all coins of a given currency. Then there is some minimum amount of money in the piggy-bank that we can guarantee. Your task is to find out this worst case and determine the minimum amount of cash inside the piggy-bank. We need your help. No more prematurely broken pigs!

输入:

The input consists of T test cases. The number of them (T) is given on the first line of the input file. Each test case begins with a line containing two integers E and F . They indicate the weight of an empty pig and of the pig filled with coins. Both weights are given in grams. No pig will weigh more than 10 kg, that means $1 \leq E \leq F \leq 10000$. On the second line of each test case, there is an integer number N ($1 \leq N \leq 500$) that gives the number of various coins used in the given currency. Following this are exactly N lines, each specifying one coin type. These lines contain two integers each, P and W ($1 \leq P \leq 50000$, $1 \leq W \leq 10000$). P is the value of the coin in monetary units, W is its weight in grams.

输出:

Print exactly one line of output for each test case. The line must contain the sentence "The minimum amount of money in the piggy-bank is X ." where X is the minimum amount of money that can be achieved using coins with the given total weight. If the weight cannot be reached exactly, print a line "This is impossible."

样例输入:

```
3
10 110
2
1 1
30 50
10 110
2
1 1
50 30
1 6
2
10 3
20 4
```

样例输出:

```
The minimum amount of money in the piggy-bank is 60.
The minimum amount of money in the piggy-bank is 100.
This is impossible.
```

【题目大意】

在 ACM 可以任何事情之前，必须准备预算和获得必要的财务支持。该行动的主要收入

来自不可逆转的捆绑资金（IBM）。背后的想法很简单。每当一些 ACM 成员有任何小钱时，他将所有硬币扔进存钱罐。你知道这个过程是不可逆转的，即硬币不能在不打破存钱罐的情况下取出。经过足够长的时间，存钱罐里应该有足够的现金来支付需要支付的东西。

但是存钱罐存在一个很大的问题，那就是无法确定存钱罐内有多少钱。也许打破存钱罐后会发现没有足够的钱。显然，我们希望避免这种不愉快的情况。唯一可以做的是称存钱罐的重量并试图猜测里面有多少硬币。假设我们能够确切地确定存钱罐的重量，并且我们知道给定货币中所有硬币的重量。然后通过存钱罐的重量可以保证存钱罐中的最小存款。你的任务是找出最坏的情况，并确定存钱罐内的最小存款。我们需要你的帮助，以便不再过早地打破存钱罐！

【分析】

由于每种硬币的数量都可以是任意多，所以该问题为完全背包问题。但在该例中，完全背包有两处变化：

- ① 要求的不再是最大值，而变为了最小值，于是状态方程相应地改为

$$dp[j] = \min(dp[j], dp[j - w[i]] + v[i])$$

- ② 该问题要求硬币和空储蓄罐的重量恰好达到总重量，即在背包问题中，物品要恰好将背包装满，而这个问题只需改变 $dp[j]$ 的初始值，将 $dp[0]$ 初始化为 0，将 $dp[i] (1 \leq i \leq m)$ 初始化为无穷大（不可达到）即可。

代码 12.8

```
#include <iostream>
#include <cstdio>
#include <climits>

using namespace std;

const int INF = INT_MAX / 10;
const int MAXN = 10000;

int dp[MAXN];
int v[MAXN];           //物品价值
int w[MAXN];           //物品重量

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    while (caseNumber--) {
        int e, f;
        scanf("%d%d", &e, &f);
        int m = f - e;           //背包容量
        int n;                   //物品种类
        scanf("%d", &n);
```

```

for (int i = 0; i < n; i++) {
    scanf("%d%d", &v[i], &w[i]);
}
for (int i = 1; i <= m; i++) {
    dp[i] = INF;           //注意初始化
}
dp[0] = 0;
for (int i = 0; i < n; ++i) {
    for (int j = w[i]; j <= m; ++j) {
        dp[j] = min(dp[j], dp[j - w[i]] + v[i]);
    }
}
if (dp[m] == INF) {
    printf("This is impossible.\n");
} else {
    printf("The minimum amount of money in the piggy-bank is %d.\n",
dp[m]);
}
return 0;
}

```

总之，完全背包问题的特点是每类物品可选的数量为无穷，其解法与 0-1 背包问题整体保持一致，与其不同的仅为状态更新时的遍历顺序。

3. 多重背包

如果在 0-1 背包和完全背包问题之间折中，即每种物品既不是只能取一件，又不是能取无穷件，而是最多只能取 k 件，这时该如何使得背包里的物品价值总和最大呢？这便得到多重背包问题：有 n 种物品，每种物品的重量为 $w[i]$ ，其价值为 $v[i]$ ，每种物品的数量为 $k[i]$ ，现在有个容量为 m 的背包，如何选择物品使得装入背包物品的价值最大。

与之前的背包问题都不同，每种物品可选的数量不再为无穷或一个，而是介于其中的一个确定的数 k 。可以将多重背包问题直接转化为 0-1 背包问题，即每种物品均被视为 k 种重量和价值都不同的物品，对所有的物品求 0-1 背包，其时间复杂度为

$$O\left(m \sum_{i=1}^n k_i\right)$$

由此可见，降低每种物品的数量 k_i 将会大大降低其复杂度，于是可以采用一种更有技巧性的拆分：将原数量为 k 的物品拆分为若干组，将每组物品视为一件物品，其价值和重量为该组中所有物品的价值重量总和。每组物品包含的原物品个数分别为 $2^0, 2^1, \dots, 2^{c-1}, k - 2^{c-1} + 1$ ，其中 c 是使得 $k - 2^{c-1} + 1 \geq 0$ 的最大整数。这种类似于二进制的拆分，不仅将物品数量大大降低，同时通过由若干原物品得到新物品的不同组合，可以得到 0 到 k 之间的任意件物品的价值重量和，

所以对所有这些新物品做 0-1 背包，即可得到多重背包的解。由于转化后的 0-1 背包物品数量大大降低，其时间复杂度也得到较大优化，此时为

$$O\left(m \sum_{i=0}^n \log_2(k_i)\right)$$

例题 12.9 珍惜现在，感恩生活

题目描述：

急！灾区的食物依然短缺！

为了挽救灾区同胞的生命,心系灾区同胞的你准备自己采购一些粮食来支援灾区。现在假设你一共有资金 n 元，而市场有 m 种大米，每种大米都是袋装产品，其价格不等，并且只能整袋购买。请问：你用有限的资金最多能采购多少千克粮食呢？

输入：

输入数据首先包含一个正整数 C ，表示有 C 组测试用例，每组测试用例的第一行是两个整数 n 和 m ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示经费的金额和大米的种类；然后是 m 行数据，每行包含 3 个数 p, h 和 c ($1 \leq p \leq 20, 1 \leq h \leq 200, 1 \leq c \leq 20$)，分别表示每袋大米的价格、每袋大米的质量以及对应种类大米的袋数。

输出：

对于每组测试数据，请输出能够购买大米的最大质量，你可以假设经费买不光所有的大米，并且经费你可以不用完。每个实例的输出占一行。

样例输入：

```
1
8 2
2 100 4
4 100 2
```

样例输出：

```
400
```

【分析】

本题对每个物品的数量进行了限制，并非无穷也并非只有一个，即本题为一个多重背包问题。通过对每种物品的拆分，可以使得物品数量大大减少，同时也保证拆分后物品间的组合可以组合出任意数量的物品，如此便转换成了 0-1 背包问题，之后按照 0-1 背包的处理办法即可得到多重背包问题的解。

代码 12.9

```
#include <iostream>
#include <cstring>

using namespace std;

const int MAXN = 10000;

int dp[MAXN];
```

```

int v[MAXN];           //物品价值
int w[MAXN];           //物品质量
int k[MAXN];           //物品数目
int value[MAXN];        //分解后物品价值
int weight[MAXN];       //分解后物品质量

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    while (caseNumber-->0) {
        int n, m;
        scanf("%d%d", &n, &m);           //n 件物品，m 容量的背包
        int number = 0;                   //分解后物品的数量
        for (int i = 0; i < n; ++i) {
            scanf("%d%d%d", &w[i], &v[i], &k[i]);
            for (int j = 1; j <= k[i]; j++) {
                value[number] = j * v[i];
                weight[number] = j * w[i];
                number++;
                k[i] -= j;
            }
            if (k[i] > 0) {
                value[number] = k[i] * v[i];
                weight[number] = k[i] * w[i];
                number++;
            }
        }
        for (int i = 0; i <= m; ++i) {
            dp[i] = 0;                       //初始化
        }
        for (int i = 0; i < number; ++i) {
            for (int j = m; j >= weight[i]; --j) {
                dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
            }
        }
        printf("%d\n", dp[m]);
    }
    return 0;
}

```


多重背包问题小结如下：多重背包的特征是每个物品可取的数量为一个确定的整数，通过对这个整数进行拆分，使若干物品组合成一个更大的物品，同时通过这些大物品间的组合又可组合出选择任意件物品的情况，通过这种拆分使得最后求解 0-1 背包问题时的物品数量大大减少，从而降低复杂度。

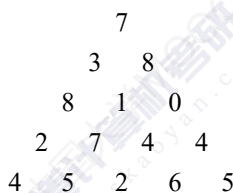
本节主要讨论了背包问题，重点指出了三类背包问题，即 0-1 背包问题、完全背包问题和多重背包问题，其中 0-1 背包问题是背包问题的基础，很多背包问题都可转换为 0-1 背包问题。

12.6 其他问题

由于动态规划问题没有固定的命题形式，之前几节也只是介绍了一些经典且常见的动态规划问题。本节结合题目分析一些动态规划问题的实例，以帮助读者深入了解动态规划。

例题 12.10 The Triangle

题目描述：



(Figure 1)

Figure 1 shows a number triangle. Write a program that calculates the highest sum of numbers passed on a route that starts at the top and ends somewhere on the base. Each step can go either diagonally down to the left or diagonally down to the right.

输入：

Your program is to read from standard input. The first line contains one integer N : the number of rows in the triangle. The following N lines describe the data of the triangle. The number of rows in the triangle is > 1 but ≤ 100 . The numbers in the triangle, all integers, are between 0 and 99.

输出：

Your program is to write to standard output. The highest sum is written as an integer.

样例输入：

```

5
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5

```

样例输出：

```

30

```

【题目大意】

图 1 显示了一个数字三角形。编写一个程序，计算从顶部开始到底部某处的路径上传递的最大数字总和。路径上的每步只能沿对角线向左下或右下滑动。

【分析】

本题要求从上到下经过的一条路径中所有值之和最大，并求出这个和。如果用二维数组 $matrix[i][j]$ 代表三角形中各个位置的数值，用 $dp[i][j]$ 代表从 (i,j) 点出发到底部路径所有值之和的最大值，那么 $dp[0][0]$ 便是问题的答案。

由于每一步只能沿对角线向左下或右下滑动，所以 $dp[i][j]$ 只能从 $dp[i+1][j]$ 和 $dp[i+1][j+1]$ 这两种状态转移得到，因此可以得到状态转移方程：

$$dp[i][j] = \max(dp[i+1][j], dp[i+1][j+1]) + matrix[i][j]$$

代码 12.10

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 100;

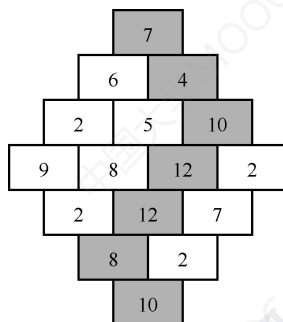
int dp[MAXN][MAXN];
int matrix[MAXN][MAXN];

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j <= i; ++j) {
            scanf("%d", &matrix[i][j]);
            dp[i][j] = matrix[i][j];
        }
    }
    for (int i = n - 1; i >= 0; --i) {
        for (int j = 0; j <= i; ++j) {
            dp[i][j] += max(dp[i + 1][j], dp[i + 1][j + 1]);
        }
    }
    printf("%d\n", dp[0][0]);
    return 0;
}
```

例题 12.11 Monkey Banana Problem

题目描述:

You are in the world of mathematics to solve the great “Monkey Banana Problem”. It states that, a monkey enters into a diamond shaped two dimensional array and can jump in any of the adjacent cells down from its current position (see figure). While moving from one cell to another, the monkey eats all the bananas kept in that cell. The monkey enters into the array from the upper part and goes out through the lower part. Find the maximum number of bananas the monkey can eat.



输入:

Input starts with an integer T (≤ 50), denoting the number of test cases.

Every case starts with an integer N ($1 \leq N \leq 100$). It denotes that, there will be $2N - 1$ rows. The i^{th} ($1 \leq i \leq N$) line of next N lines contains exactly i numbers. Then there will be $N - 1$ lines. The j^{th} ($1 \leq j < N$) line contains $N - j$ integers. Each number is greater than zero and less than 2^{15} .

输出:

For each case, print the case number and maximum number of bananas eaten by the monkey.

样例输入:

```
2
4
7
6 4
2 5 10
9 8 12 2
2 12 7
8 2
10
2
1
2 3
1
```

样例输出:

Case 1: 63

Case 2: 5

【题目大意】

你要在数学世界中解决伟大的“猴子香蕉问题”。这个问题的描述是，猴子进入一个菱形的二维矩阵，并可以从其当前位置向下跳入任何相邻的单元格（见图）。当从一处移动到另一处时，猴子会吃掉那里面的所有香蕉。猴子从上部进入矩阵并从下部出来。找到猴子可以吃的最大香蕉数量。

【分析】

本题要求从上到下经过的一条路径中所有值之和最大，并求出这个和。这个问题与上一题非常相似，只不过变成了两个三角形，由于这两个三角形的形状上下颠倒，所以它们的状态转移方程有些不同。

对于上半部分的三角形，其转移方程和上一题中的一样：

$$dp[i][j] = \max(dp[i+1][j], dp[i+1][j+1]) + matrix[i][j]$$

对于下半部分的三角形，其转移方程也是：

$$dp[i][j] = \max(dp[i+1][j], dp[i+1][j+1]) + matrix[i][j]$$

不过下半部分的三角形两端的方格只能朝一个方向移动，最左端的方格只能向右下移动，最右端的方格只能向左下移动，所以两端的方程有所不同：

$$dp[i][j] = dp[i+1][j] + matrix[i][j]$$

代码 12.11

```
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 100;

int dp[2 * MAXN][MAXN];
int matrix[2 * MAXN][MAXN];

int main() {
    int caseNumber;
    scanf("%d", &caseNumber);
    for (int cas = 1; cas <= caseNumber; ++cas) {
        int n;
        scanf("%d", &n);
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j <= i; ++j) {
                scanf("%d", &matrix[i][j]);
                dp[i][j] = matrix[i][j];
            }
        }
    }
}
```

```

    }
}
for (int i = n; i < 2 * n - 1; ++i) {
    for (int j = 0; j <= 2 * (n - 1) - i; ++j) {
        scanf("%d", &matrix[i][j]);
        dp[i][j] = matrix[i][j];
    }
}

for (int i = 2 * (n - 1) - 1; i >= n - 1; --i) {
    for (int j = 0; j <= 2 * (n - 1) - i; ++j) {
        if (j == 0) { //最左端
            dp[i][j] += dp[i + 1][j];
        } else if (j == 2 * (n - 1) - i) { //最右端
            dp[i][j] += dp[i + 1][j - 1];
        } else {
            dp[i][j] += max(dp[i + 1][j], dp[i + 1][j - 1]);
        }
    }
}

for (int i = n - 2; i >= 0; --i) {
    for (int j = 0; j <= i; ++j) {
        dp[i][j] += max(dp[i + 1][j], dp[i + 1][j + 1]);
    }
}

printf("Case %d: %d\n", cas, dp[0][0]);
}
return 0;
}

```

习题 12.7 放苹果（北京大学复试上机题）

把 M 个同样的苹果放在 N 个同样的盘子里，允许有的盘子空着不放，问共有多少种不同的分法（用 K 表示）？例如，5, 1, 1 和 1, 5, 1 是同一种分法。

提交网址：

<http://t.cn/AiQsyOnq>

习题 12.8 整数拆分（清华大学复试上机题）

一个整数总可以拆分为 2 的幂的和。例如，7 可以拆分成 $7=1+2+4$ ， $7=1+2+2+2$ ， $7=1+1+1+4$ ， $7=1+1+1+2+2$ ， $7=1+1+1+1+1+2$ ， $7=1+1+1+1+1+1+1$ ，共有 6 种

不同的拆分方式。再如，4 可以拆分成 $4=4$ ， $4=1+1+1+1$ ， $4=2+2$ ， $4=1+1+2$ ，共有 4 种不同的拆分方式。

用 $f(n)$ 表示 n 的不同拆分的种数，例如 $f(7)=6$ 。要求编写程序，读入 n （不超过 1000000），输出 $f(n)\%1000000000$ 。

提交网址：

<http://t.cn/AiQsUMoQ>

小结

本章主要介绍与动态规划相关的问题。首先介绍了递推求解、最大连续子序列和、最长递增子序列、最长公共子序列；然后着重讨论了动态规划中常见的背包问题，包括 0-1 背包问题、完全背包问题、多重背包问题；最后介绍了一些常见的其他动态规划问题。动态规划问题历来以难度高著称，虽然考研机试对动态规划的考查不会太深入，但读者也要做好求解动态规划问题的准备。